

Interazione e Multimedia

Canale A-L - Elena Giunta

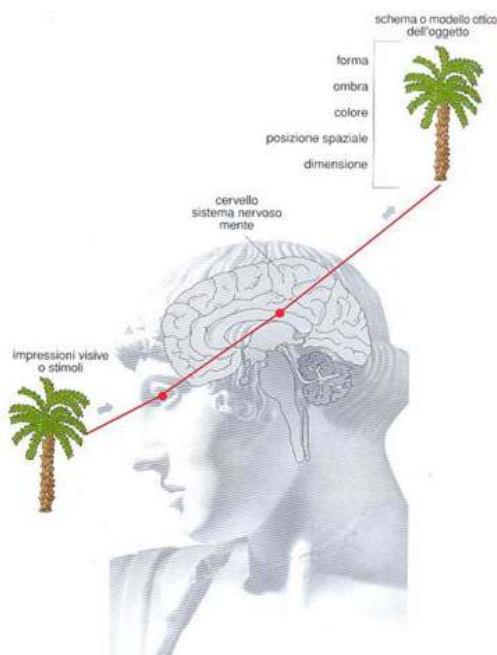
A.A. 2024-2025

L'uso delle immagini come mezzo di comunicazione è radicato nella natura umana. Nel 1920 si ebbe la prima applicazione di immagini digitali con la trasmissione di un'immagine via cavo tra New York e Londra, destinata alla stampa in *half toning* (bianco e nero). Successivamente, nel 1922, vennero aggiunti 5 e poi 15 livelli di grigio.

Fino al 1964, le immagini venivano solo trasmesse e non elaborate: la prima elaborazione digitale fu realizzata dalla NASA, che corresse una foto della Luna al computer per motivi ottici.

L'essere umano e le immagini

La visione nasce dalla collaborazione tra occhio e cervello: l'occhio percepisce gli stimoli visivi (luce, colori, forme), mentre il cervello li elabora. Analizzando aspetti come forma, ombra, colore, posizione e dimensione, il cervello crea un'immagine riconoscibile e le attribuisce significato.



Leggi della percezione visiva

La mente tende naturalmente a vedere figure chiuse, a completare contorni interrotti e a percepire forme semplici e regolari.

- **Legge della vicinanza:** gli oggetti che sono vicini tra loro tendono a essere percepiti come un gruppo o un insieme
- **Legge della chiusura:** La percezione delle figure chiuse prevale su quella delle figure aperte.
- **Legge dell'uguaglianza:** gli elementi simili tra loro tendono a essere percepiti come appartenenti allo stesso gruppo.
- **Legge della continuità:** il nostro cervello tende a percepire gli elementi posti uno di seguito all'altro in modo continuo e fluido, come una struttura unitaria.
- **Legge delle buone forme:** il cervello cerca di "chiudere" le forme e completare le immagini in modo da ottenere strutture regolari, unitarie e compatte.

Queste leggi vengono applicate negli algoritmi di inpainting, tecnica utilizzata per ripristinare o completare parti mancanti di un'immagine.



Intensità percepita

La relazione tra l'intensità fisica della luce e l'intensità percepita dall'occhio umano è un aspetto cruciale da considerare, specialmente nelle immagini digitali. L'intensità percepita è una **funzione logaritmica dell'intensità incidente**. Ciò significa che se l'intensità della luce aumenta in modo esponenziale, la percezione di questa intensità aumenta in modo logaritmico. Questo implica che:

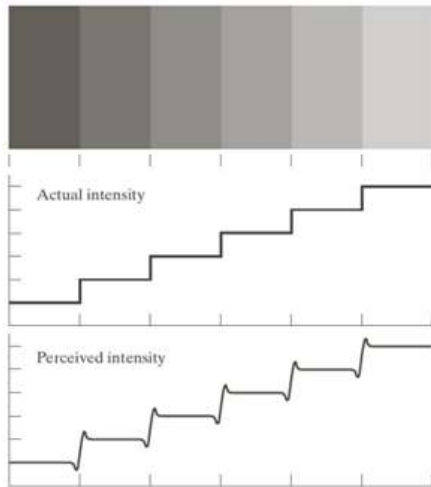
- A basse intensità, piccoli aumenti di luce possono essere percepiti in modo significativo.
- A intensità elevate, grandi aumenti di luce possono non risultare in cambiamenti altrettanto evidenti nella percezione.

Il sistema visivo umano non opera contemporaneamente su tutto il range delle intensità percepite: non riusciamo contemporaneamente a vedere le zone troppo luminose e quelle troppo buie.

Bande di Mach e contrasto simultaneo

Le **bande di Mach** illustrano come il sistema visivo interpreti le informazioni luminose in modo diverso dalla realtà fisica. Questa illusione si manifesta quando l'occhio percepisce strisce chiare e scure lungo i confini tra regioni con gradienti di luminosità, anche se in realtà non esistono: le bande hanno un'intensità costante, ma vengono percepite in maniera non

uniforme all'approssimarsi dei bordi.



Rappresentazione di un'immagine

Un'immagine digitale è descritta come una funzione bidimensionale $f(x,y)$, dove x e y sono coordinate spaziali. La funzione $f(x,y)$ descrive l'intensità luminosa dell'immagine in quel punto, che dipende sia dalla luce incidente sull'oggetto, indicata con $i(x,y)$, sia dalla luce riflessa dall'oggetto, indicata con $r(x,y)$.

La relazione può essere espressa come: $f(x,y) = i(x,y) \cdot r(x,y)$

- Il valore della luce incidente i può assumere valori: $0 < i(x, y) < \infty$ (limiti teorici)
- Il valore della luce riflessa r può assumere valori: $0 < r(x, y) < 1$ (limiti teorici)

L'indice di riflettanza è relativo al materiale ed è un numero limitato e compreso tra 0 e 1, a differenza dell'illuminazione, potenzialmente illimitato.

In teoria, il valore di $f(x,y)$ è un numero *reale continuo*, tuttavia per produrre un'immagine digitale dobbiamo convertire questa funzione continua in una forma discreta che i computer possano gestire. Questo passaggio avviene grazie alle operazioni di **campionamento** (suddivisione dell'immagine continua in pixel discreti) e **quantizzazione** (assegnare valori discreti di intensità a ciascun pixel)

Il piano XY in cui stanno le coordinate dell'immagine è detto dominio spaziale e le variabili x , y sono dette variabili spaziali o coordinate spaziali.

Immagini Vettoriali e Raster

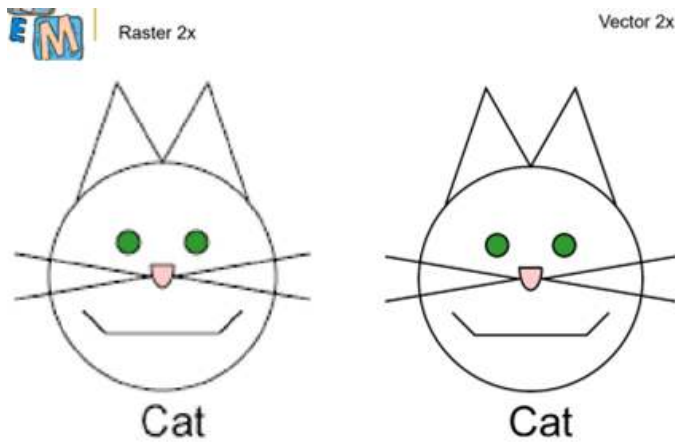
Distinguiamo principalmente due tipologie di immagini: immagini raster e immagini vettoriali

Grafica Vettoriale

Le immagini vettoriali sono caratterizzate da una serie di regole matematiche che definiscono specifiche **forme geometriche** anziché da una griglia di pixel.

Questo tipo di immagine può essere modificata senza perdita di qualità, quindi *non si creano artefatti*. Un visualizzatore di immagini vettoriali segue tali regole matematiche per mostrare

l'immagine in un specifico piano di riferimento. Le immagini possono poi essere stampate con strumenti appositi (plotter).



Grafica Raster

La **grafica raster**, o **grafica bitmap**, rappresenta le immagini come una griglia di **pixel** con un valore che indica il suo colore. Le immagini raster sono legate alla risoluzione: la qualità dipende dal numero di pixel per unità di misura (ad esempio, pixel per pollice - PPI). Se un'immagine raster viene ingrandita troppo, i singoli pixel diventano visibili, e questo provoca un effetto di **pixellatura** e perdita della qualità.

Il **BPP** (bit per pixel) valuta l'efficienza della compressione, misurando il numero di bit necessari per memorizzare un singolo pixel. Un **BPP** più basso indica una compressione più efficiente e un'immagine che occupa meno spazio.

Immagini raster come matrici

Le immagini raster sono rappresentate come matrici $M \times N$, dove ogni elemento rappresenta un pixel. Possiamo porre la matrice sul quarto piano cartesiano, convenzionalmente il primo elemento della matrice è sempre l'elemento in alto a sinistra.

- **L'asse Y** è orientato dall'alto verso il basso; quindi, la coordinata $y=0$ si trova nella parte superiore dell'immagine a sinistra, mentre i valori crescono man mano che si scende verso il basso.
- **L'asse X** è orientato da sinistra verso destra; quindi la coordinata $x=0$ è a sinistra, e i valori di x aumentano man mano che ci si sposta a destra.

Entrambe le coordinate x e y sono **positive**.

	0	1	2	...	n_2	...	N_2
0							
1							
2							
...							
n_1					x[n ₁ , n ₂]		
...							
N_1							

Tipologie di immagini

Bianco e nero

Le **immagini in bianco e nero** sono il tipo più semplice di immagine digitale, ogni pixel è rappresentato da un solo bit e può assumere come valori 0 o 1 (rispettivamente nero o bianco)

Scala di Grigio

Le **immagini in scala di grigio** sono rappresentate da valori che variano dal nero al bianco, passando attraverso sfumature di grigio.

8 bit per pixel (bpp): ogni pixel è rappresentato da 8 bit, che consentono di memorizzare un valore intero compreso tra **0 e 255**:

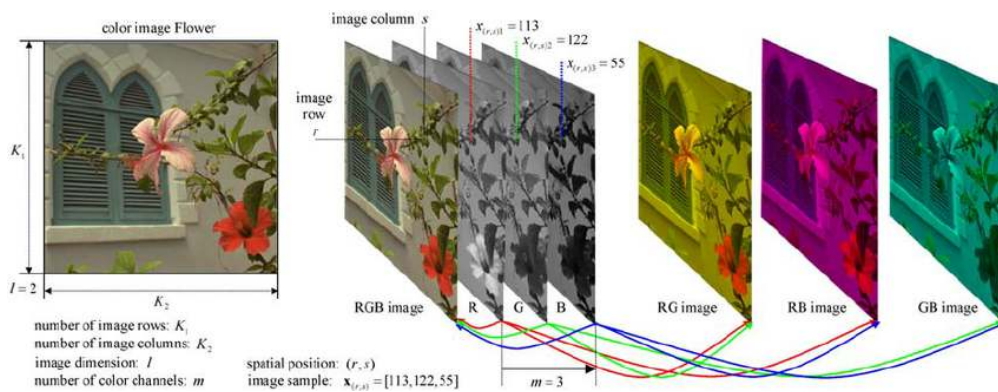
- **0** rappresenta il nero.
- **255** rappresenta il bianco.
- I valori intermedi (1-254) rappresentano le diverse sfumature di grigio.

A colori

Le **immagini a colori** usano tre canali di colore (rosso, verde e blu - RGB) per rappresentare ogni pixel.

Ogni canale è rappresentato da **8 bit**, quindi ogni pixel ha un totale di **24 bit** (8 bit per ciascuno dei tre canali). Ogni pixel è rappresentato da una terna di valori **(x, y, z)**, dove:

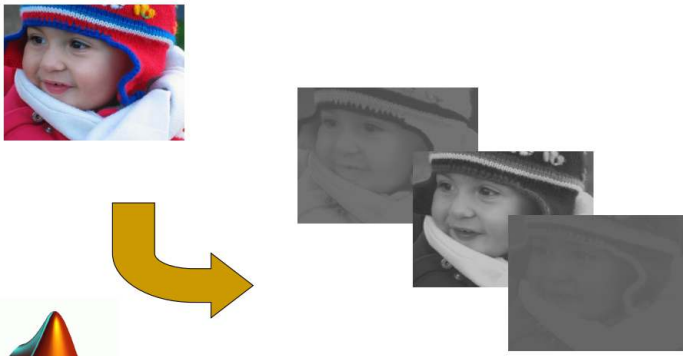
- **x** è l'intensità del rosso e varia da **0 a 255**,
- **y** è l'intensità del verde e varia da **0 a 255**,
- **z** è l'intensità del blu e varia da **0 a 255**



Quando separiamo i canali di un'immagine RGB (rosso, verde, blu), vengono visualizzati in **scala di grigio** per mostrare l'**intensità** di ciascun colore.

In questo modo, possiamo immediatamente capire quali parti dell'immagine contribuiscono di più o di meno a quel canale.

- **Bianco** indica alta intensità del colore (ad esempio, molto rosso nel canale rosso).
- **Nero** indica assenza di colore.



Operazioni sulle matrici

Su una immagine possono essere applicate le operazioni che si possono fare sulle matrici, ma non è detto che tali operazioni abbiano sempre un senso logico (ad esempio la moltiplicazione matriciale)

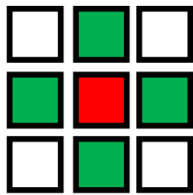
(Per le matrici algebriche vale la regola del prodotto riga per colonna, mentre nell'immagine processing si usa fare il prodotto puntuale tra due matrici, cioè il prodotto punto a punto degli elementi corrispondenti)

Neighborhood N_p

In un'immagine digitale, il vicinato N_p di un pixel indica i pixel immediatamente vicini, dove il valore di p determina il numero e la posizione dei pixel adiacenti.

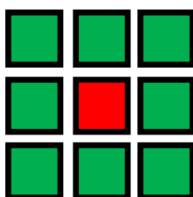
Vicinato 4-connesso:

- Comprende solo i pixel nei punti cardinali: sopra, sotto, sinistra e destra.



Vicinato 8-connesso:

- Include i pixel del vicinato 4-connesso **più** i pixel in diagonale: in alto a destra, in alto a sinistra, in basso a destra e in basso a sinistra.



Distanza Euclidea

Per calcolare la distanza fra due pixel nella diagonale:

È la distanza "in linea retta" tra due pixel, calcolata dalla formula della distanza euclidea. Se abbiamo due pixel $P1(x_1, y_1)$ e $P2(x_2, y_2)$, la distanza euclidea è:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Distanza di Manhattan (o Distanza "City Block")

Questa misura la distanza totale percorribile in una griglia solo lungo gli assi orizzontale e verticale (senza diagonali). La distanza di Manhattan tra i pixel $P1$ e $P2$ è:

$$d = |x_2 - x_1| + |y_2 - y_1|$$

Operazioni affini

Trasformazioni affini: tali operazioni mantengono inalterata l'intensità dei colori dei pixel ma ne modificano la geometria (posizione).

Con una matrice 3x3, un punto (x, y) viene rappresentato con coordinate omogenee come $(x, y, 1)$ e la trasformazione si applica così:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Traslazione

La traslazione sposta il punto di (t_x, t_y) :

Risultato: $x' = x + t_x$ e $y' = y + t_y$

$$T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix}$$

- Non è richiesta interpolazione dopo questa operazione

Scala (Scaling)

La scala modifica le dimensioni lungo gli assi con fattori s_x e s_y

$$T = \begin{bmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Risultato: $x' = x \times s_x$ e $y' = y \times s_y$

- Dopo la sua applicazione è necessaria l'interpolazione

Distorsione (Shear):

- Inclinazione lungo x di un fattore s_x o lungo y di un fattore s_y

Risultato Shear verticale asse x $\rightarrow x' = x + s_v \times y \rightarrow y$ rimane invariato

$$T = \begin{bmatrix} 1 & 0 & 0 \\ s_v & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Risultato Shear orizzontale asse y $\rightarrow y' = y + s_h \times x \rightarrow x$ rimane invariato

$$T = \begin{bmatrix} 1 & s_h & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Dopo la sua applicazione è necessaria l'interpolazione.

Rotazione

La rotazione ruota di un angolo θ . $\cos\theta$ e $\sin\theta$ sono i valori trigonometrici relativi all'angolo θ

$$x' = x \cos\theta + y \sin\theta$$

$$y' = -x \sin\theta + y \cos\theta$$

$$T = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- **Rotazioni di 90°, 180°, 270°:** Non richiedono interpolazione, poiché i pixel ruotati si allineano perfettamente con la griglia originale.
- **Rotazioni con angoli diversi:** Richiedono interpolazione, poiché i nuovi pixel non si allineano perfettamente.

Ogni trasformazione è quindi ottenuta moltiplicando la riga delle coordinate $[x, y, 1]$ (in coordinate omogenee) per la matrice di trasformazione T .

Ad esempio così:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} (1 \cdot x) + (0 \cdot y) + (0 \cdot 1) \\ (0 \cdot x) + (1 \cdot y) + (0 \cdot 1) \\ (t_x \cdot x) + (t_y \cdot y) + (1 \cdot 1) \end{bmatrix}$$

Forward mapping e Inverse mapping

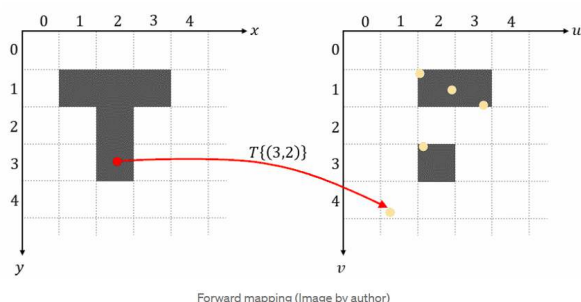
Forward Mapping e **Inverse Mapping** sono due approcci utilizzati per applicare trasformazioni geometriche (come quelle affini) su immagini.

Il modo più diretto per implementare trasformazioni spaziali è iterare su ogni pixel dell'immagine di input, calcolare la sua nuova posizione nell'immagine di output utilizzando $T\{\dots\}$ (matrice della trasformata affine) e copiare il valore del pixel nella nuova posizione.

- (v,w) rappresentano le coordinate di un pixel nell'immagine di input.
- (x,y) rappresentano le coordinate di un pixel nell'immagine di output.
- T è la matrice di trasformazione affine.

Forward mapping $\rightarrow [x \ y \ 1] = [v \ w \ 1] \cdot T$

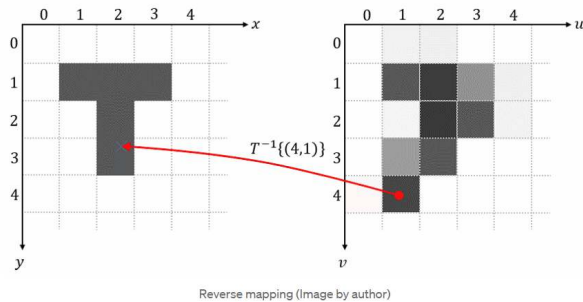
Questo metodo presenta due principali svantaggi: sovrapposizioni e buchi. Alcuni pixel dell'immagine di output ricevono più di un pixel dell'immagine di input, mentre altri pixel dell'output non ricevono alcun pixel dell'immagine di input.



Il mapping inverso itera su ciascun pixel della griglia dell'immagine di output, utilizza la trasformazione inversa $T^{-1}\{\dots\}$ per determinare la posizione corrispondente nell'immagine di input, campiona il valore del pixel in questa posizione e usa quel valore come pixel di output:

Inverse mapping $\rightarrow [x \ y \ 1] = [v \ w \ 1] \cdot T^{-1}$

Tale funzione non è affetta dalla problematica dei buchi neri ma può essere affetta da quello delle sovrapposizioni.



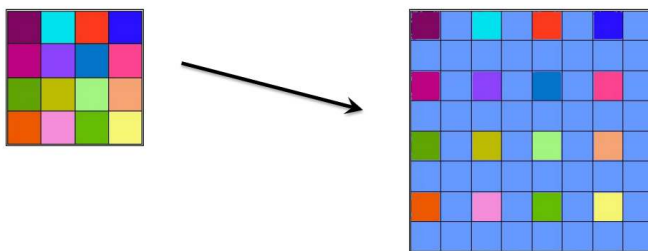
Interpolazione

Durante le trasformazioni, possono esserci pixel il cui valore non è determinato dalle formule utilizzate. Si ricorre quindi a un processo di interpolazione, il quale non migliora la qualità dell'immagine: si tratta di un metodo matematico che stima dati sconosciuti partendo da dati reali.

Zooming in

L'interpolazione è effettuata anche nei processi di zooming.

Zooming e dimensioni: Quando si applica uno zoom di $2\times$ a un'immagine di dimensioni $m \times n$, le nuove dimensioni diventano $2m \times 2n$, aumentando così il numero totale di pixel da mn a $4mn$. Lo zooming può essere effettuato in maniera differente per le singole dimensioni. Ad esempio si potrebbe raddoppiare il numero di righe e triplicare il numero di colonne.



Esistono diversi tipi di interpolazione:

Nearest neighbor - Replication

Il metodo **Nearest Neighbor** è uno degli approcci più semplici per l'interpolazione, essendo l'unica a non fare operazioni aritmetiche. Quando un'immagine viene ingrandita, i pixel vengono semplicemente replicati venendo copiati in base al fattore di zoom. Ovviamente i pixel vengono copiati con criterio (copio a partire da sinistra verso destra quello bordeaux piuttosto che quello azzurro).

La replication riporta risultati di interpolazione più scarsi.

Bilineare

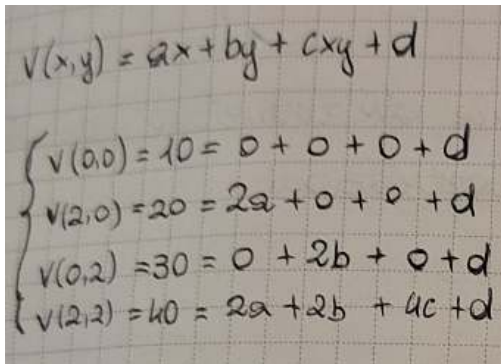
Nell'interpolazione bilineare si utilizzano i **quattro pixel più vicini** per stimare l'intensità da assegnare a ciascuna nuova posizione.

Supponiamo che (x, y) siano le coordinate della posizione cui si deve assegnare un valore di intensità e che $v(x, y)$ equivalga al valore dell'intensità.

I quattro coefficienti sono determinati dalle quattro equazioni ottenibili usando i quattro pixel più vicini al punto nella coordinata (x, y) .

$$v(x, y) = ax + by + cxy + d$$

L'interpolazione bilineare produce dei risultati migliori rispetto alla replication.



Handwritten equations for bilinear interpolation coefficients:

$$v(x, y) = ax + by + cxy + d$$
$$\begin{cases} v(0,0) = 10 = 0 + 0 + 0 + d \\ v(2,0) = 20 = 2a + 0 + 0 + d \\ v(0,2) = 30 = 0 + 2b + 0 + d \\ v(2,2) = 40 = 2a + 2b + 4c + d \end{cases}$$

Bicubica

L'interpolazione bicubica utilizza i **sedici pixel più vicini** al punto. per stimare l'intensità da assegnare a ciascuna nuova posizione. Il valore di intensità assegnato al punto (x, y) si ottiene attraverso l'equazione:

$$v(x, y) = \sum_{i=0}^3 \sum_{j=0}^3 a_{i,j} x^i y^j$$

Dove i sedici coefficienti sono determinati a partire da sedici equazioni in sedici incognite che possono essere scritte utilizzando i sedici punti più vicini a (x, y) .

L'interpolazione bicubica produce dei risultati migliori rispetto alla bilineare.

Problema dei bordi

A ridosso dei bordi dell'immagine, il numero di pixel adiacenti è minore, e non risulterà possibile usare in maniera precisa gli algoritmi di interpolazione. Due sono le possibili soluzioni:

- Non si applica nessuna soluzione, si replicano solo i valori di righe e colonne adiacenti
- Si usano algoritmi di interpolazione che prendono come riferimento un numero di pixel adiacenti

Media dei pixel

Dopo aver fatto i calcoli per media dei pixel adiacenti, si replicano le ultime righe e colonne.

output

1	1.5	2	2.5	3	3
2.5	3	3.5	4	4.5	4.5
4	4.5	5	5.5	6	6
5.5	6	6.5	7	7.5	7.5
7	7.5	8	8.5	9	9
7	7.5	8	8.5	9	9

Zooming out

Se lo zooming è fatto con un numero inferiore ad 1, si ottiene una immagine più piccola dell'originale, si ha quindi un processo di decimazione. Data una immagine $M \times N$ con uno zooming out di 0,5 si otterrà una immagine $\frac{m}{2} \times \frac{n}{2}$

Anche per lo zooming out esistono diversi metodi di applicazione:

Metodo 1: Decimazione

Conserviamo solo un pixel seguendo una regola arbitraria (quello in alto a sinistra ogni 4 pixel, in questo caso)

input

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

1	3
9	11

Metodo 2: calcolare la media

Di quattro pixel se ne calcola il valore medio.

input

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

3	5
11	13

In alternativa, si possono anche replicare i valori nelle righe e colonne al bordo.

Stima della qualità di un algoritmo

MSE (Mean Square Error): è una metrica ampiamente utilizzata per quantificare la differenza tra due immagini, o più in generale tra due segnali. In contesti come l'elaborazione delle immagini viene utilizzato per stimare l'errore tra un'immagine originale e una versione modificata (ad esempio, un'immagine compressa o ricostruita).

- Un MSE basso indica che le due immagini sono molto simili.
- L'MSE tra un'immagine e sé stessa è sempre **0**
- Il valore massimo è pari S^2 (valore massimo che un pixel può assumere).

L'MSE è calcolato come la media dei quadrati delle differenze tra i pixel corrispondenti nelle due immagini:

$$MSE = \frac{1}{MN} \sum_{x=1}^M \sum_{y=1}^N [I^*(x, y) - I(x, y)]^2$$

PSNR (Peak Signal to Noise Ratio): è un parametro usato per misurare la qualità di un'immagine compressa rispetto all'originale. Il PSNR non è il metodo migliore per valutare la qualità di un algoritmo di compressione, ma è il più diffuso. Più alto è il PSNR, migliore è la qualità dell'immagine (ovvero, meno errore ci sarà rispetto all'immagine originale).

- Per calcolare il PSNR abbiamo bisogno dell'MSE.

Il PSNR è calcolato con una delle seguenti **formule** (sono equivalenti):

$$PSNR = -10 \log_{10} \frac{MSE}{S^2} \quad PSNR = 20 \log_{10} \left(\frac{S}{\sqrt{MSE}} \right) \quad PSNR = 10 \log_{10} \left(\frac{S^2}{MSE} \right)$$

MSE e PSNR sono valori molto sensibili alle trasformazioni affini: confrontare un'immagine con la versione traslata, ruotata o ridimensionata di se stessa avrà un forte impatto su valori di MSE e PSNR.

Non sempre danno un risultato fedele a quello dato dal sistema visivo umano: due immagini distorte possono avere tipi molto diversi di errori pur avendo lo stesso MSE.

- Esercizio: Quanto vale il PSNR se l'MSE è uguale al quadrato del massimo valore di luminanza rappresentabile nell'immagine?

$$MSE = S^2$$

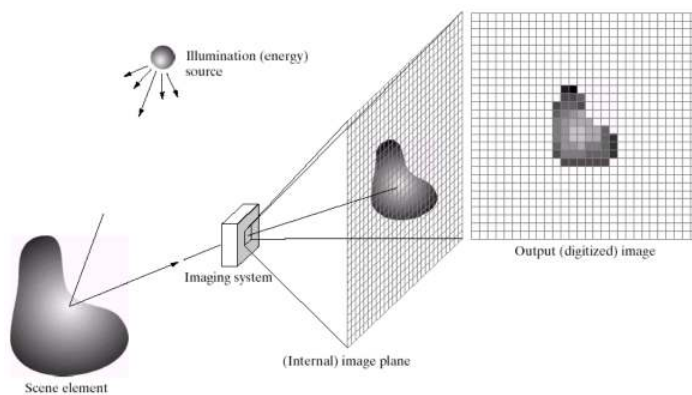
Sostituiamo nell'equazione:

$$PSNR = 10 \cdot \log_{10} \left(\frac{L^2}{L^2} \right) = 10 \cdot \log_{10}(1)$$

$$\log_{10}(1) = 0 \text{ quindi: } PSNR = 10 \cdot 0 = 0$$

Acquisizione delle immagini

Quando la luce colpisce un oggetto, una parte viene assorbita ed una parte viene riflessa, è quest'ultima a dare origine al colore percepito. Per creare una immagine digitale la luce riflessa deve essere catturata da un sensore ed essere elaborata.



Il sensore

L'energia che colpisce il sensore viene trasformata in impulso elettrico, che sarà poi convertito in digitale. In strumenti diversi vengono utilizzati sensori di tipo differente, come sensori singoli, in linea o a matrice. Questa ultima tipologia è molto diffusa nelle fotocamere digitali.

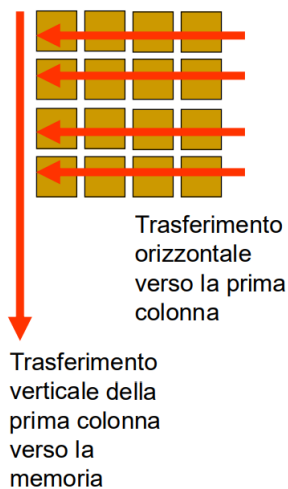
Sensori in 2D array

Nelle macchine fotografiche digitali, i sensori sono disposti su una matrice. Non è necessario spostare il sensore per effettuare una scansione. I sensori più diffusi di questo tipo sono i CCD, l'altra tipologia è chiamata CMOS, più presente nei cellulari.

CCD: Charged Coupled Device

I sensori a matrice più diffusi sono di tipo CCD (Charged Coupled Device). Questi dispositivi elettronici sono formati da tante celle, che se colpite da fotoni assumono una carica positiva. Il numero di celle per area di esposizione è misurato in MEGAPIXEL ed indica un parametro di qualità.

Dopo che le cariche sono state acquisite da una matrice di celle esse devono essere trasferite attraverso il bus in una memoria digitale. La scansione avviene in C fasi, una fase per ciascuna colonna della matrice. Ad ogni fase viene trasferita in memoria la prima colonna della matrice, nello stesso tempo tutti gli elementi (dalla seconda colonna in poi) vengono trasferiti dalla propria colonna a quella precedente.



Cattura di un'immagine a colori

Non è possibile creare un tipo di sensore capace di catturare specifici colori, né tantomeno uno che riesca a catturare l'intero spettro del visibile simultaneamente. Tuttavia, usando un consono sistema di filtri, è possibile catturare i valori delle singole componenti dei colori.

CFA: Color filter Array

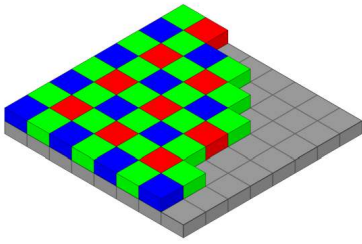
Le fotocamere digitali acquisiscono immagini a colori grazie a una **matrice di filtri colorati** (CFA, *Color Filter Array*), posizionata sopra il sensore. Il CFA filtra la luce, consentendo a ciascuna cella di registrare solo una specifica componente cromatica (rosso, verde o blu), si ottiene così una immagine a falsi colori.

Dopo l'acquisizione, un processo chiamato **demosaicizzazione** (demosaicking) ricostruisce l'immagine a colori, combinando le informazioni provenienti dai diversi pixel. La disposizione ottimale dei filtri nel CFA è cruciale per garantire una resa cromatica accurata ed efficiente.

Bayern Pattern

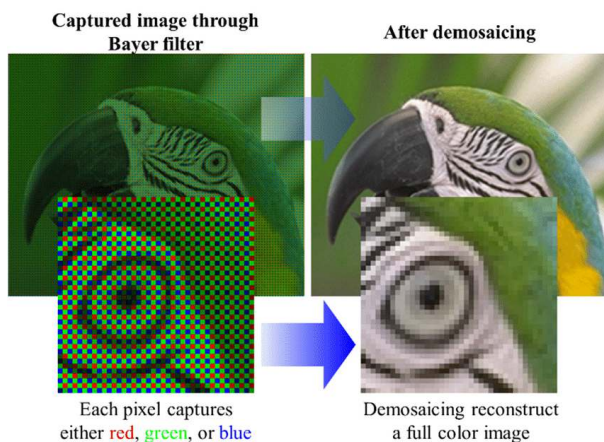
Il **Bayer Pattern** è lo schema di CFA più utilizzato nei sensori delle fotocamere digitali dal 1980. Segue un rapporto **1:2:1 per i colori Rosso (R), Verde (G) e Blu (B)** con i pixel verdi disposti in diagonale. Questa scelta deriva dal fatto che l'occhio umano è più sensibile al verde, il quale si trova al centro dello spettro visibile e contribuisce maggiormente alla percezione della luminosità.

Le immagini acquisite con il **Bayer Pattern** vengono salvate nel formato **RAW**, che conserva i dati grezzi del sensore senza alcuna elaborazione. Ciò consente di applicare processi di **demosaicizzazione**, correzione del colore e ottimizzazione dell'immagine.



Schema:

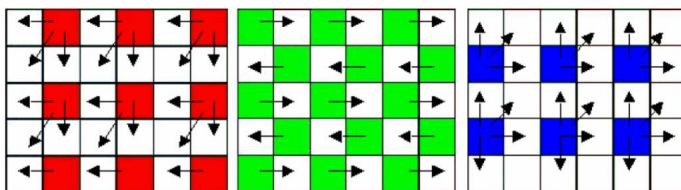
- **Rilevazione Monocromatica:** Il sensore rileva solo l'intensità della luce che passa attraverso il filtro di ogni pixel: se il pixel ha un filtro rosso, registra l'intensità della luce rossa ecc.
- **Dati grezzi in scala di grigi:** L'immagine grezza prodotta è quindi un mosaico in scala di grigi, dove ogni pixel contiene un solo valore di intensità. Questa immagine appare puntinata e non rappresenta ancora una vera immagine a colori, perché ogni pixel ha informazioni incomplete.
- Sovrapponendo il Bayer Pattern all'immagine RAW si ottiene la cosiddetta **"immagine a falsi colori"**.
- Infine, eseguendo la color interpolation si ottiene un'immagine a colori priva, dell'effetto mosaico detta **"immagine demosaicizzata"**, per tale motivo il processo di color interpolation viene anche detto **"demosaicking"**.



Nearest-neighbor interpolation

Per ogni singolo pixel gli elementi mancanti della terna vengono copiati dall'intorno.

L'interpolazione bilineare riporta i risultati più scarsi in termini di dettagli nell'immagine.



Bilineare

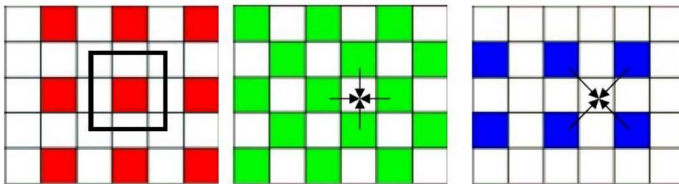
Nell'interpolazione bilineare si utilizzano i quattro pixel più vicini per stimare l'intensità da assegnare a ciascuna nuova posizione.

Per l'interpolazione bilineare il valore assegnato si ottiene mediante l'equazione:

$$v(x, y) = ax + by + cxy + d$$

L'interpolazione bilineare preserva meglio i dettagli rispetto all'interpolazione nearest-neighbor.

Red position:



Bicubica

L'interpolazione bicubica utilizza i sedici pixel più vicini al punto. Il valore di intensità assegnato al punto (x, y) si ottiene attraverso l'equazione

$$v(x, y) = \sum_{i=0}^3 \sum_{j=0}^3 a_{ij} x^i y^j$$

Dove i sedici coefficienti sono determinati a partire da sedici equazioni in sedici incognite che possono essere scritte utilizzando i sedici punti più vicini a (x, y).

L'interpolazione bicubica preserva meglio i dettagli rispetto all'interpolazione bilineare.

La risoluzione

La più piccola unità di misura di un'immagine raster è un pixel. La **risoluzione** di un'immagine raster si riferisce alla quantità di pixel utilizzati per rappresentarla, quindi il numero di pixel per unità di misura. Può essere espressa in modi diversi:

- **Pixel per unità di misura (densità di pixel)**: questo tipo di risoluzione misura la densità di pixel per un'unità di lunghezza, ad esempio **Ppcm (pixel per centimetro)** oppure **Dpi (dots per inch)**.
- **Risoluzione totale**: in questo caso, la risoluzione è espressa come il numero totale di pixel che costituiscono l'immagine. (Megapixel)

Nelle immagini la risoluzione indica il grado di qualità. Più è alta la risoluzione, più saranno i pixel in essa contenuti e di conseguenza si avrà un maggior dettaglio. La qualità dell'immagine, in termini di risoluzione, è il risultato della risoluzione dello strumento di ripresa e di quello di resa (esempio: fotocamera e monitor).

Risoluzione apparecchiatura di ripresa

È data dal numero di sensori per unità lineare di misura. Si riferisce alla capacità di cattura delle immagini o video di un dispositivo.

Risoluzione apparecchiatura di resa

È data dal numero di punti per unità lineare di misura. Si riferisce invece alla capacità di un dispositivo di visualizzare o riprodurre l'immagine o il video.

Per ottenere la massima fedeltà di resa visiva, è la risoluzione dell'immagine deve corrispondere a quella del dispositivo di resa (come uno schermo). Quando la risoluzione dell'immagine e quella del dispositivo di resa non coincidono, interviene il processo di interpolazione, che cerca di "aggiustare" l'immagine per adattarla alla risoluzione di resa, inserendo pixel nuovi o riorganizzandoli, il che potrebbe portare a una perdita di dettaglio o a distorsioni visive.

Rapporto

Il rapporto di un'immagine – **aspect ratio** – indica la proporzione tra larghezza e altezza dell'immagine. Viene espresso come una frazione (ad esempio, 16:9, 4:3).

L'occhio

La retina è una membrana che ricopre la parte posteriore dell'occhio, è formata da coni e bastoncelli che sono fotorecettori. Intercettando stimoli luminosi, producono stimoli elettrici.

I coni

I coni sono circa 6/7 milioni e sono concentrati nella fovea, la zona centrale della retina. Sono responsabili della visione fototica (dei colori e dei dettagli fini). Ogni cono è collegato ad un nervo ottico. La **quantità è minore rispetto ai bastoncelli**.

Altro da sapere sulla retina

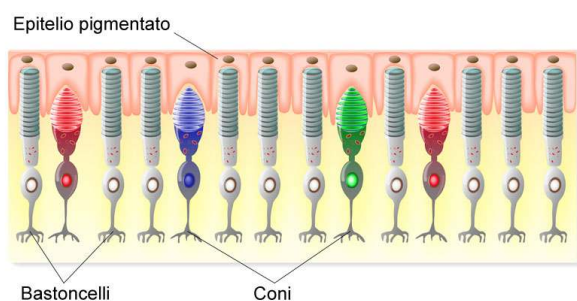
La fovea è una regione di 1,5 mm x 1,5 mm. Ha una popolazione di coni di circa 150.000 elementi per mm². Il numero di coni nella fovea è di circa 337.500 elementi. Tale numero è impressionante poiché un CCD per contenere lo stesso numero di celle necessita di una superficie di almeno 5mm x 5mm.

I bastoncelli

I bastoncelli sono circa 75/150 milioni e sono distribuiti su tutta la retina. Sono poco sensibili al colore ed sono collegati a gruppi ai nervi ottici. I **bastoncelli** sono responsabili della

visione scotopica e monocromatica (visione in condizioni di bassa luminosità e in scala di grigi). Sono in **quantità maggiore rispetto ai coni**.

STRUTTURA DELLA RETINA

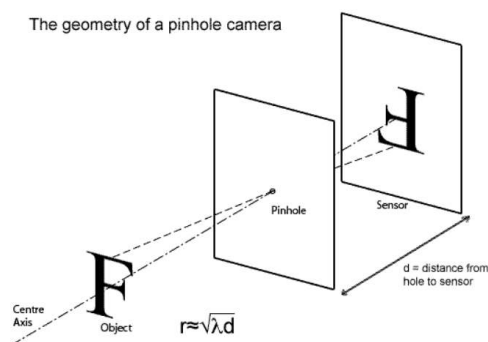


Come si forma l'immagine nell'occhio

Il modello Pinhole permette di astrarre il problema della formazione dell'immagine dell'occhio. Si tratta di un modello teorico in cui si approssima l'occhio con una scatola, nella quale all'interno su una parete viene posizionata una pellicola sensibile alla luce (simula la retina). Nella parete opposta si pratica un foro con uno spillo.

La dimensione ideale del foro dipende da due fattori principali:

1. **La distanza d** tra il foro e la pellicola (o retina), che influenzerà la quantità di luce che riesce a passare attraverso il foro.
2. **La lunghezza d'onda λ** della luce, che influisce sul comportamento delle onde luminose, specialmente in relazione agli effetti di diffrazione.



Il foro non può essere un foro puntiforme infinitesimo, in quanto non farebbe passare abbastanza fotoni per attivare il sensore.

Se il foro è molto piccolo si verificano effetti di **diffrazione**, cioè la deviazione delle onde luminose. Questi effetti di diffrazione causano una distorsione dell'immagine, poiché la luce che passa attraverso il foro si disperde in modo irregolare, sfocando l'immagine.

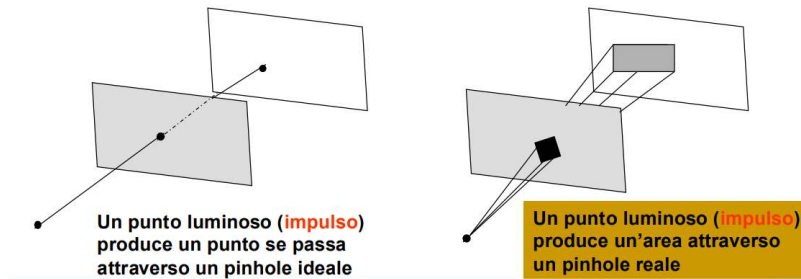
Pinhole ideale vs reale

Un **pinhole ideale** è un foro infinitesimo che permette alla luce di passare in modo perfetto, proiettando un'immagine nitida, dove ogni punto luminoso rimane un punto sulla superficie

sensibile.

Tuttavia, un **pinhole reale** ha una forma geometrica definita (come un cerchio) e non è un punto perfetto. Questo causa una **sfocatura** nell'immagine, poiché la luce che passa attraverso il foro non proviene da un singolo punto ma da un'area limitata, creando una piccola zona di proiezione invece di un punto.

La qualità dell'immagine dipende dalla dimensione del foro: troppo grande causa sfocatura, mentre troppo piccolo riduce la quantità di luce che passa, rendendo l'immagine più debole.



Point spread Function

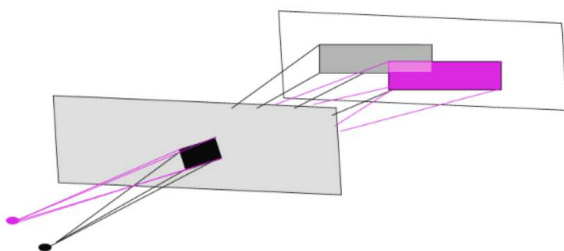
La **Point Spread Function (PSF)** descrive la diffusione di un punto luminoso quando passa attraverso il foro

- La **forma geometrica del foro** influenza la PSF. Ad esempio, un foro circolare crea una PSF circolare.
- La **dimensione del foro** determina il grado di sfocatura dell'immagine: un foro più grande lascia passare più luce ma causa una maggiore diffusione.
- La **diffrazione** (soprattutto con fori molto piccoli) amplifica l'area della PSF, riducendo la nitidezza dell'immagine.

Principio di sovrapposizione

Il **principio di sovrapposizione** descrive come le onde luminose provenienti da diverse sorgenti vicine si combinano quando passano attraverso il foro nel modello pinhole.

Più sorgenti ci sono e più la luce tende a sovrapporsi, riducendo la nitidezza dell'immagine.

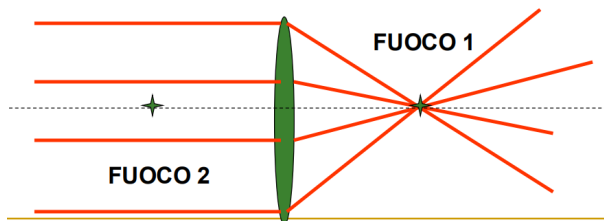


Lenti sottili

Viste le problematiche del pinhole, possiamo affermare che i fori sono inadeguati per consentire ai sensori misurazioni precise. Per questo motivo vengono usate le lenti sottili.

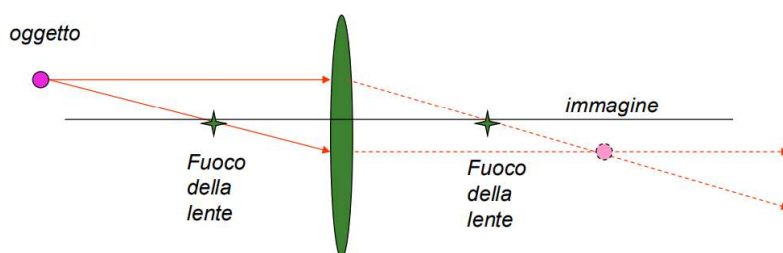
Proprietà geometriche della lente sottile

- Una lente sottile ha due fuochi equidistanti da essa.
- Raggi paralleli all'asse della lente sottile che passano attraverso essa, vengono proiettati tutti attraverso il fuoco, posto a distanza F dalla lente.
- Raggi che passano attraverso il fuoco, sono ritrasmessi tutti in direzione parallela nella direzione dell'asse della lente.



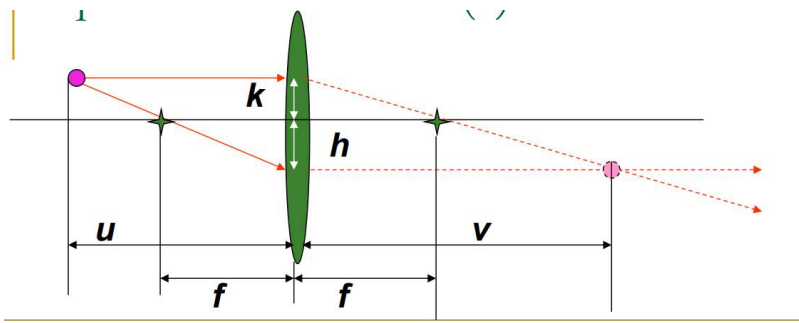
Un oggetto puntiforme luminoso emette raggi luminosi in ogni direzione, ma sappiamo per certo che:

- Solo un raggio sarà parallelo all'asse della lente ed esso verrà concentrato nel fuoco;
- Solo un raggio passerà per il fuoco ed esso verrà ritrasmesso parallelo all'asse della lente;
- Il punto in cui i due nuovi raggi si incontrano sarà il punto di formazione dell'immagine dell'oggetto;
- Spostando il piano dei sensori più avanti o più indietro del piano in cui si forma l'immagine è possibile ottenere un'immagine sfocata



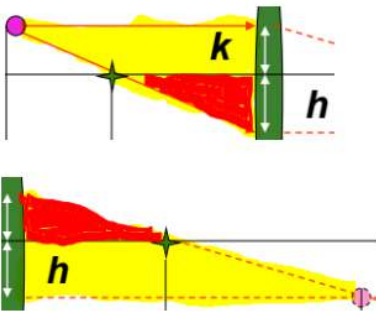
Equazione della lente sottile

Troviamo adesso un'equazione che metta in relazione i valori indicati nella seguente immagine:



Definizione delle variabili:

- u : distanza dell'oggetto dalla lente.
- v : distanza dell'immagine dalla lente.
- f : distanza focale della lente.
- h : altezza dell'immagine.
- k : altezza dell'oggetto.



Triangoli simili sul lato dell'oggetto:

- Il triangolo di base u e altezza $(h + k)$ (sul lato sinistro della lente) è simile al triangolo di base f e altezza h .
- Da questa somiglianza si ricava: $\frac{u}{h+k} = \frac{f}{h}$
Da cui otteniamo: $\frac{u \times h}{f} = h + k$

Triangoli simili sul lato dell'immagine:

- Il triangolo di base v e altezza $h + k$ (sul lato destro della lente) è simile al triangolo di base f e altezza k .
- Da questa somiglianza si ricava: $\frac{v}{h+k} = \frac{f}{k}$

Da cui otteniamo: $\frac{v \times k}{f} = h + k$

Eguagliamo: $\frac{v \times k}{f} = \frac{u \times h}{f} \rightarrow$ eliminiamo $f \rightarrow v \times k = u \times h \rightarrow$ infine ottenendo:

$$\frac{h}{v} = \frac{k}{u}$$

Da $(h + k) = \frac{(u \times h)}{f}$ dividiamo entrambi i membri per u , ottenendo: $\frac{h}{u} + \frac{k}{v} = \frac{h}{f}$

Ma $\frac{h}{u} = \frac{h}{v}$ quindi $\frac{h}{u} + \frac{h}{v} = \frac{h}{f}$

Eliminiamo il fattore comune e otteniamo definitivamente **l'equazione della lente sottile**, ovvero:

$$\frac{1}{u} + \frac{1}{v} = \frac{1}{f}$$

Ricordiamo che: u : distanza dell'oggetto dalla lente; v : distanza dell'immagine dalla lente; f : distanza focale della lente.

L'equazione della lente sottile ha alcune importanti applicazioni pratiche e implica diversi fenomeni ottici.

Se f (distanza focale della lente) si misura in metri, **la quantità 1/metro si definisce pari ad una diottria** (potenza ottica)

$$D = \frac{1}{f}$$

a) Messa a fuoco con lente fissa

Quando abbiamo una lente fissa, come nelle fotocamere, f è una quantità costante. Se la distanza dall'oggetto indicata con u aumenta, la distanza lente - sensore deve diminuire per mettere a fuoco un oggetto.

b) Messa a fuoco nell'occhio umano

Nell'occhio umano, il piano dei sensori (la retina) è fisso, quindi non può essere spostato. Invece, la messa a fuoco avviene variando la **distanza focale** della lente. I muscoli dell'occhio contraggono o rilassano il cristallino per modificare la sua curvatura, cambiando così f . La capacità del cristallino di cambiare la lunghezza focale è misurata in **diottrie**.

Il cristallino simula una lente!

c) Profondità di campo

Due oggetti a distanza u_1 e u_2 dalla lente sottile, distanze di molto superiori a f , appariranno approssimativamente sullo stesso piano (in quanto v_1 e v_2 saranno valori molto vicini); ciò non succede invece quando u_1 ed u_2 sono a distanze differenti e comparabili (meno di 30 volte la distanza della lente), allora non sono focalizzabili contemporaneamente. Si presenterà così il fenomeno della "profondità di campo"

Magnificazione

Col termine magnificazione andiamo a indicare la proprietà della lente sottile di alterare la dimensione dell'immagine di un oggetto rispetto alla sua dimensione effettiva.

Il **fattore di magnificazione** è dato da:

$$\frac{v}{u} = m$$

Moltiplichiamo per v l'equazione della lente sottile $\frac{v}{u} + 1 = \frac{v}{f}$

Ovvero $m + 1 = \frac{v}{f}$

Invertiamo: $\frac{1}{m} + 1 = \frac{f}{v}$

Moltiplichiamo per u e sostituiamo $\frac{u}{v} = 1/m$

$$\frac{1}{m} + 1 = \frac{f}{v}$$

Moltiplichiamo per m ottenendo finalmente una relazione tra il fuoco, la distanza dall'oggetto e il fattore di magnificazione.

$$f = \frac{u \times m}{m+1}$$

- Esercizio sulla diottria: Cosa rappresenta la quantità $\frac{m+1}{u \times m}$?

La quantità $\frac{m+1}{u \times m}$ rappresenta il **numero di diottrie** della lente. La formula $\frac{u \times m}{m+1}$ è la relazione inversa, che si usa per calcolare la distanza focale f della lente. **La diottria** $\frac{1}{f}$, può essere scritta come:

$$\frac{1}{f} = \frac{m+1}{u \times m}$$

- **Esercizio sul fuoco**

Data una lente sottile da dieci diottrie, se poniamo un oggetto di fronte ad essa a distanza di 1 metro, a che distanza dal sensore va posta la lente affinché si possa formare un'immagine a fuoco?

Usiamo l'equazione della lente sottile, che lega la distanza dell'oggetto u , la distanza dell'immagine v e la distanza focale f .

- La lente ha 10 diottrie, quindi $f = \frac{1}{10} \text{ m} = 0,1 \text{ m}$
- La distanza dell'oggetto è $u = 1 \text{ m}$
- Il nostro obiettivo è quindi trovare v .

Sostituendo i valori nell'equazione della lente:

$$\frac{1}{0,1} = \frac{1}{1} + \frac{1}{v} = 10 = 1 + \frac{1}{v}$$

Sottraendo 1 da entrambi i lati:

$$9 = \frac{1}{v}$$

Invertiamo l'equazione:

$$v = \frac{1}{9} \text{ m}$$

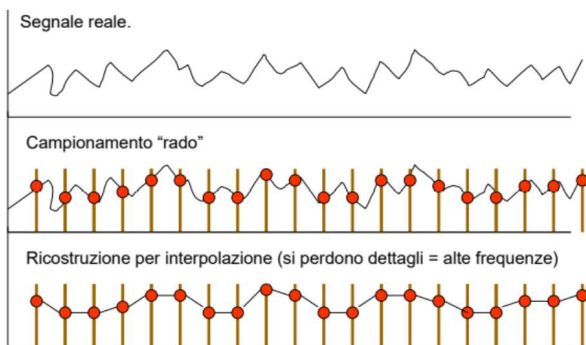
Ampiezza di campo e focale

- **Campo visivo ampio** è ottenuto con lunghezze focali corte e sensori di dimensioni maggiori.
- **Campo visivo stretto** è ottenuto con lunghezze focali lunghe o con sensori più piccoli.

Campionamento e Quantizzazione

Campionamento

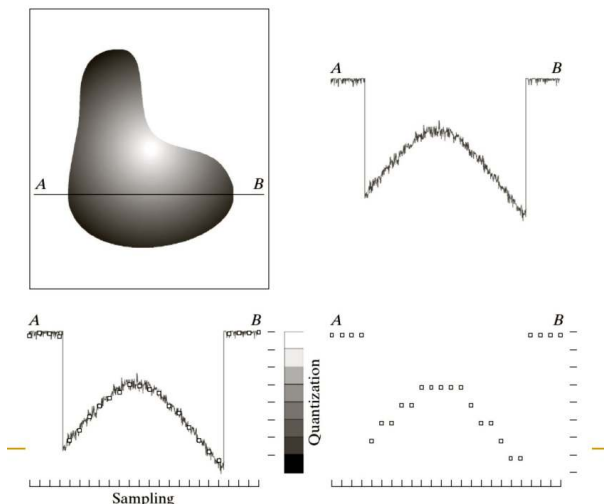
Nella teoria dei segnali il campionamento è una tecnica che consiste nel convertire un segnale continuo nel tempo oppure nello spazio in un segnale discreto, valutandone l'ampiezza con un numero finito di campioni rappresentativi del segnale cosicché da avere valori discreti anziché numeri reali. Un campionamento troppo basso può causare la perdita di dettagli ed informazioni.



(*campionamento*)

I punti mostrano valori continui in base all'intensità misurata a ciascun intervallo lungo il profilo

A-B. I campioni sono punti lungo il profilo presi a intervalli regolari. Si osserva che questi punti rappresentano un'accurata approssimazione del segnale continuo.



(*quantizzazione*)

Il grafico mostra il risultato della quantizzazione. I campioni sono stati mappati in valori discreti

I valori sono rappresentati con una scala di grigi, con una suddivisione ben visibile in livelli di intensità.

Il Nyquist rate e il teorema di Shannon

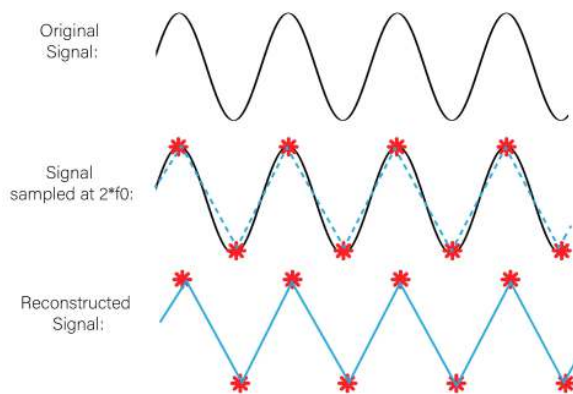
Per scegliere il giusto valore di campionamento si ricorre ad un teorema fondamentale: il teorema di Shannon. Tale teorema si basa sulla misura del Nyquist rate.

Si definisce Nyquist rate il doppio della più alta frequenza di un segnale continuo e limitato.

$$f_{Nyquist} = 2 \cdot f_{max}$$

Il teorema di Shannon afferma che, per evitare perdita di informazioni nella ricostruzione di un segnale, la frequenza di campionamento deve essere pari o maggiore al Nyquist rate.

$$f_s \geq 2 \cdot f_{max}$$



Ad esempio, se un segnale ha una frequenza massima di 2000kHz, la frequenza di campionamento minima necessaria sarà 4000kHz, ovvero il doppio di quella massima.

Sottocampionamento e Aliasing

Se viene violato il teorema di Nyquist-Shannon (sottocampionamento) possono apparire nelle immagini dettagli non presenti nell'originale, che alterano il segnale. Un esempio di quanto detto è il fenomeno di aliasing, il quale fa sì che le alte frequenze siano trattate come basse frequenze durante il campionamento.

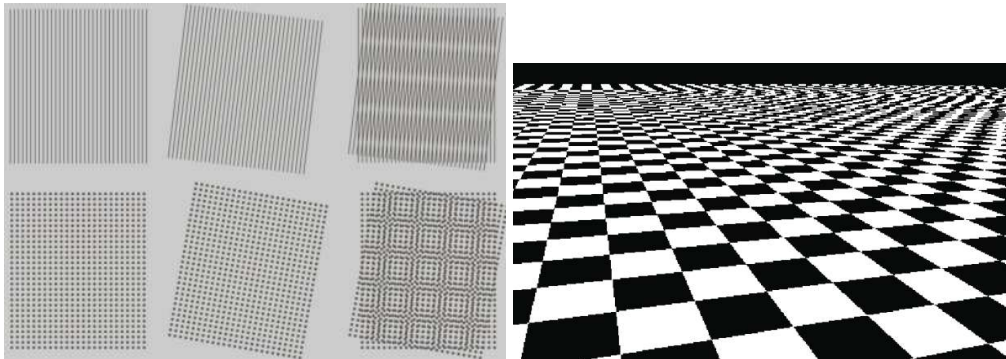
Aliasing

Quando il teorema di Nyquist-Shannon non viene rispettato a causa di un tasso di campionamento insufficiente (sottocampionamento), si verifica il fenomeno dell'**aliasing**. Questo effetto fa sì che le alte frequenze del segnale vengano interpretate erroneamente come frequenze più basse, alterando l'informazione originale. Nell'elaborazione delle immagini, l'aliasing può causare la perdita di dettagli e la comparsa di distorsioni, specialmente in prossimità di linee continue o pattern complessi.

- Esercizio: Un segnale S viene campionato con una frequenza di campionamento pari a 5Khz. Tuttavia, sul segnale campionato S' si nota la presenza di aliasing. Cosa possiamo affermare con certezza?

La frequenza di campionamento è 5 kHz. Se nel segnale campionato S' c'è aliasing, significa che nel segnale originale S c'era almeno una frequenza **superiore a 2.5 kHz**. Questa frequenza alta non è stata campionata correttamente e ha causato l'aliasing. **La massima frequenza di S è superiore a 2.5 kHz**

Un particolare artefatto derivante dall'aliasing è il **Moiré pattern**.



Per evitare tali problemi, è fondamentale applicare filtri anti-aliasing (che prevedono l'applicazione di un filtro passa-basso) o aumentare la frequenza di campionamento per rispettare il criterio di Nyquist. **Funzionamento filtri anti-aliasing:**

- Prima del campionamento si applica un filtro che attenua le frequenze superiori alla metà della frequenza di campionamento ($f_{\text{campionamento}}/2$).
- Questo filtro "smussa" il segnale, riducendo le componenti ad alta frequenza che potrebbero causare aliasing durante il campionamento

Nell'ambito dei video, un framerate basso potrebbe causare problemi relativi alla percezione del movimento degli oggetti: un vero e proprio alias temporale. Un movimento rotatorio in un verso, dato un basso framerate, potrebbe rendere in video nel verso opposto.

Quantizzazione

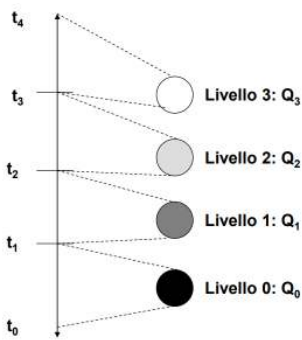
Nella teoria dei segnali, la quantizzazione è un processo che permette di passare da numeri reali, frutto delle misurazioni dei sensori, a numeri discreti e limitati all'interno di un determinato range attraverso arrotondamenti e normalizzazioni, dopo il campionamento.

I valori da quantizzare sono nel range $[a, b]$ e si vuole quantizzare su N livelli:

Si fissano $n + 1$ in $[a-b]$ tali che: $t_0 = a < t_1 < t_2 \dots < t_n < t_{n+1} = b$

Il numero x in $[a-b]$ verrà assegnato al livello di quantizzazione k tale che:

$$t_k \leq x < t_{k+1}$$



Ogni livello di quantizzazione viene associato a delle etichette numeriche memorizzate in binario.

Il numero di bit B necessari per rappresentare N livelli è dato da: $B = \log_2(N)$

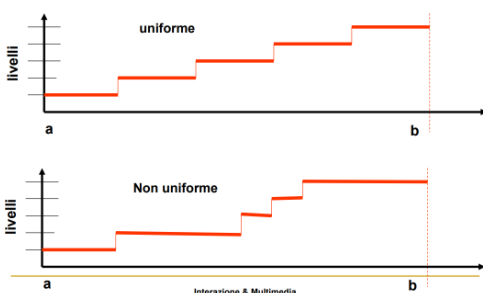
Nella **quantizzazione uniforme**, i livelli di quantizzazione sono distribuiti in modo equidistante. Questo significa che la differenza tra ogni livello adiacente è costante. Funziona bene per segnali in cui i valori sono distribuiti uniformemente.

- range in ingresso $0 \dots N-1$ (esempio 0-255), - range in uscita $0 \dots K-1$ con $K \leq N$
 Se L è il livello di ingresso rappresentato da un intero il livello L' corrispondente dopo la quantizzazione è:
 L' corrispondente dopo la quantizzazione è: $L' = \frac{L \cdot K}{N}$

Nella **quantizzazione non uniforme**, i livelli di quantizzazione non sono distribuiti in modo equidistante. Viene utilizzata una maggiore densità di livelli di quantizzazione nelle regioni in cui il segnale è più probabile che si trovi, e meno livelli nelle regioni meno probabili. La quantizzazione effettuata dagli scanner commerciali e dalla fotocamere digitali è non uniforme e logaritmica: ciò permette di assegnare più livelli nella area dei toni scuri e meno livelli nella area dei toni chiari.

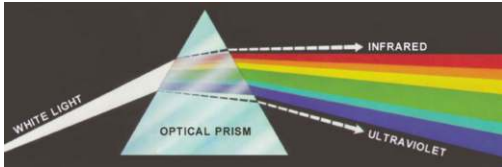
- range in ingresso $0 \dots N-1$, - range in uscita $0 \dots K-1$ con $K \leq N$.
 Se L è il livello di ingresso rappresentato da un intero il livello L' corrispondente dopo la riquantizzazione è:
 $L' = f(L, N, K) \rightarrow$ La funzione $f(L, N, K)$ definisce lo schema di riquantizzazione, e può avere le forma più varie.

Quantizzazione logaritmica: $f(L, N, K) = \frac{(\log_2(L) \cdot K)}{\log_2(N)}$ (si intende per log la parte intera del logaritmo in base 2)



Il colore

Nel 1666 Isaac Newton ha scoperto che se un raggio luminoso bianco attraversa un prisma di vetro ciò che si ottiene non è una luce bianca ma uno spettro di colori che va dal violetto al rosso. Quindi la luce può essere decomposta in onde luminose di tipo differente.

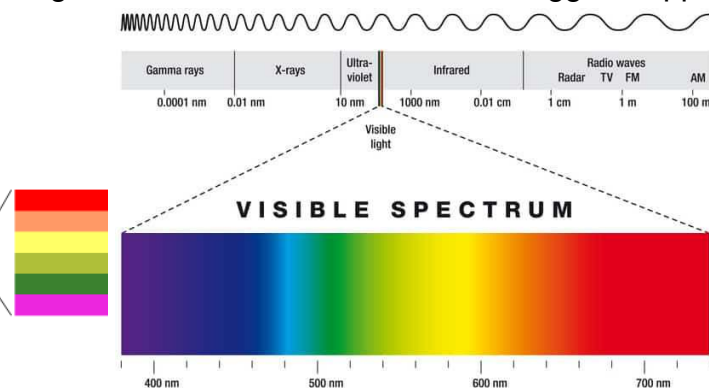


Lo spettro elettromagnetico

Il nostro **occhio** percepisce solo una piccola parte dello **spettro elettromagnetico**, chiamato **luce visibile**, che si estende tra il **violetto** e il **rosso**. I **colori** sono legati alle **lunghezze d'onda** della luce: ognuna corrisponde a una specifica regione dello spettro. Per comodità, lo spettro visibile viene diviso in sei **regioni** principali: **violetto, blu, verde, giallo, arancio e rosso**. Tuttavia, le bande di colore non sono tutte della stessa grandezza e sfumano nelle regioni adiacenti. In particolare, la **banda rossa** è la più larga. Il range va da circa **380 nm (violetto) e 750 nm (rosso)** di lunghezza d'onda.

- Quando guardiamo un oggetto, vediamo il **colore** di quella luce che l'oggetto **riflette**. Se un oggetto riflette tutte le lunghezze d'onda della luce visibile l'oggetto apparirà **bianco**.

Lung. in nanometri	Tipo radiazione
$10^{17} - 10^{13}$	Osc.elettriche
$10^{13} - 10^9$	Onde radio
$10^9 - 10^6$	Micro-onde
$10^6 - 10^3$	Infrarosso
$10^3 - 10^2$	Visibile
$10^2 - 10$	Ultravioletto
$10 - 10^{-3}$	Raggi X
$10^{-3} - 10^{-7}$	Raggi gamma e cosmici



Per descrivere la luce, si utilizzano tre concetti principali:

- 1. Radianza:** Quantità di luce emessa dalla **sorgente luminosa**
- 2. Luminanza:** la misura dell'**energia percepita** dall'occhio umano. Indica la **luminosità** di un colore, cioè quanto un colore appare chiaro o scuro. È una misura oggettiva, ma dipende dalle caratteristiche fisiche della luce che colpisce la superficie.
- 3. Brillantezza:** È una misura **percepita** di quanto chiaro o luminoso appare un colore all'occhio umano. La brillantezza dipende non solo dalla luminanza, ma anche dalla **saturazione** del colore (quanto il colore è "puro") e dalle condizioni di **illuminazione ambientale**. Quindi, la brillantezza è una percezione soggettiva che varia in base a questi fattori.

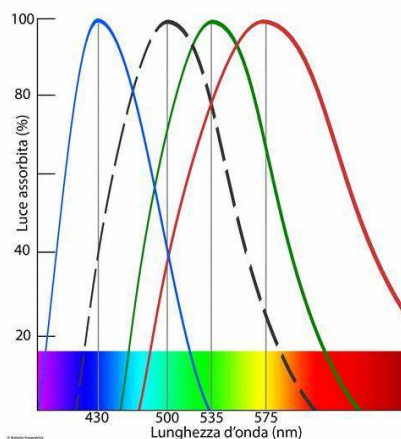
La nostra percezione dei colori dipende dai **coni**, i fotorecettori della retina sensibili a diverse lunghezze d'onda:

- **Coni S** (Short): sensibili al blu (lunghezze d'onda corte).
- **Coni M** (Medium): sensibili al verde (lunghezze d'onda medie).
- **Coni L** (Long): sensibili al rosso (lunghezze d'onda lunghe).

Spettro di assorbimento dei coni

Il grafico rappresenta le curve di assorbimento relative ai tre tipi di coni:

- **Curva S (blu)**: Mostra un picco intorno a 420 nm, indicando sensibilità alle lunghezze d'onda corte (violetto-blu).
- **Curva M (verde)**: Mostra un picco intorno a 530 nm, corrispondente alle lunghezze d'onda medie (verde).
- **Curva L (rosso)**: Mostra un picco intorno a 560-570 nm, associato alle lunghezze d'onda più lunghe (giallo-rosso).



Teoria del tristimolo (Young, 1802)

Nel 1802 Young ha teorizzato che tutti i colori si possono ottenere mescolando i tre colori fondamentali in proporzioni differenti. La teoria fu poi perfezionata e completata con la **teoria dei processi opposti**.

Il colore

Non esiste una scala fisica per misurare il colore, essendo una questione di percezione o di interpretazione soggettiva, basata sulla luce e sugli occhi e il cervello dell'osservatore.

Differenze di colore

Inoltre, la dimensione della superficie dell'oggetto influisce sull'intensità del colore: aree grandi appaiono più luminose. Anche l'angolo di osservazione e il colore dello sfondo giocano un ruolo importante, con i colori che sembrano più vivaci su sfondi scuri.

Illuminanti

Si definisce illuminante l'energia radiante con distribuzione spettrale di energia relativa definita nel campo di lunghezza d'onda capace di influenzare la visione del colore degli oggetti. Gli illuminanti A, B, C, D65, sono stati definiti dal CIE (un'organizzazione internazionale che si occupa di standardizzare la colorimetria)

Composizione dei colori

Illuminare una superficie bianca (con coefficiente di riflessione del 100%) se proiettiamo una luce su una superficie perfettamente bianca, questa rifletterà tutta la luce incidente. Quando si illumina una superficie bianca con **più luci monocromatiche contemporaneamente**, il colore che risultante sarà il risultato della **sintesi additiva** delle luci riflesse.

Basandosi sull'ipotesi che tutti i colori possano essere rappresentati come combinazioni di tre componenti pure, nel 1931 il CIE ha fissato le lunghezze d'onda standard per i tre colori primari, per poi ratificarli nel 1964.

Valori del 1931: $B = 435,8 \text{ nm}$; $R = 700 \text{ nm}$; $G = 546,1 \text{ nm}$

Valori del 1964: $B = 445 \text{ nm}$; $R = 575 \text{ nm}$; $G = 535 \text{ nm}$

Colorimetria

Queste equazioni calcolano i valori dei tristimolo X, Y, e Z, che rappresentano le componenti del colore misurate in uno spazio tridimensionale definito dai primari di luce rossa, verde e blu.

- *Color Matching Functions* $\bar{x}(\lambda)$, $\bar{y}(\lambda)$, $\bar{z}(\lambda)$: queste sono le funzioni di corrispondenza del colore definite nel sistema CIE 1931. Rappresentano la quantità di luce rossa ($\bar{x}(\lambda)$) verde ($\bar{y}(\lambda)$) e blu ($\bar{z}(\lambda)$) necessaria per riprodurre un determinato colore.
- 380 e 780 sono i limiti delle lunghezze d'onda visibili (in nm)
- $L_{e,\lambda}$ è la funzione di illuminante spettrale, che descrive la distribuzione della luce emessa dall'illuminante in funzione della lunghezza d'onda λ

$$X = \int_{380}^{780} L_{e,\lambda} \bar{x}(\lambda) d\lambda \quad Y = \int_{380}^{780} L_{e,\lambda} \bar{y}(\lambda) d\lambda = \int_{380}^{780} L_{e,\lambda} V(\lambda) d\lambda \quad Z = \int_{380}^{780} L_{e,\lambda} \bar{z}(\lambda) d\lambda$$

Dai valori del tristimolo è possibile ottenere delle coordinate di cromaticità, che descrivono le proporzioni relative dei tre componenti cromatici di un colore indipendentemente dalla sua luminosità.

Definizione di Modello di colore (o spazio dei colori)

I modelli di colore (o spazi di colore) sono sistemi utilizzati per rappresentare i colori in un formato standardizzato, facilitando così la loro specificazione, elaborazione e visualizzazione. Questi sistemi sfruttano coordinate tridimensionali/bidimensionali nel quale ogni colore è rappresentato da un punto.

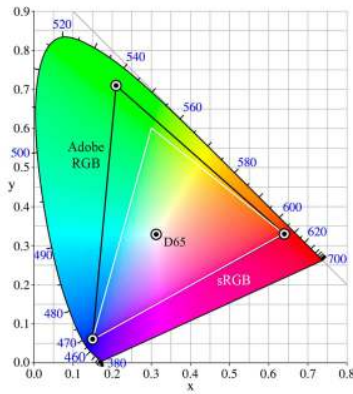
Nell'immagine processing, i modelli del colore sono interessati in più ambiti:

- Acquisizione (RGB) e restituzione (RGB, CMY) delle immagini;
- Trasmissione (YUV, YIQ) delle immagini;
- Compressione (YCbCr) delle immagini;
- Elaborazione (o analisi) delle immagini mediante trattamento del colore (RGB, HSI, HSV, LUV, ...);

Diagramma cromatico CIE xyz

Il **diagramma (o spazio) cromatico CIE 1931 xyz** è una rappresentazione bidimensionale *parziale* dello **spazio di colore XYZ**, ed è basata sulla variazione delle coordinate **x** ed **y**. Questo diagramma mostra tutte le cromaticità percepibili dall'occhio umano, senza considerare la luminanza. Il valore di **z** non è rappresentato direttamente, ma può essere calcolato con: $z = 1 - (x + y)$

- I **colori spettrali** (lunghezze d'onda visibili) si trovano lungo il bordo del diagramma.
- Il **punto di uguale energia** è situato al centro del diagramma e rappresenta il **bianco neutro**. I colori sul bordo sono **puri**, cioè privi di bianco. La **saturazione** di un colore è data dalla distanza tra il punto che lo rappresenta e il punto di uguale energia: più un colore è vicino al bordo, più è saturo.
- Se si **collegano due punti** qualsiasi sul diagramma con una linea retta, i colori situati lungo questa linea sono quelli ottenibili mescolando i due colori in proporzioni diverse.
- Se si **uniscono tre colori** sul diagramma formando un **triangolo**, i colori situati all'interno del triangolo sono quelli che possono essere generati mescolando quei tre colori in varie proporzioni. Questo **triangolo definisce il gamut** di un determinato sistema di visualizzazione, cioè l'insieme di tutti i colori che un dispositivo può rappresentare.
- I tre colori primari **Rosso (R), Verde (G) e Blu (B)** formano un triangolo che **non copre l'intera area del diagramma**: alcuni colori non possono essere riprodotti mescolando solo R, G e B. Questo rappresenta un limite del sistema RGB e mostra perché alcuni colori percepibili dall'occhio umano non possono essere visualizzati su schermi tradizionali.



Per passare dallo **spazio tridimensionale XYZ** alle coordinate bidimensionali **xyz**, si utilizzano le seguenti formule:

$$x(\text{rosso}) = \frac{X}{(X+Y+Z)}$$

$$y(\text{verde}) = \frac{Y}{(X+Y+Z)}$$

$$z(\text{blu}) = 1 - (x + y)$$

La loro somma è sempre uguale a 1!

Principale differenza

- Spazio dei colori XYZ (CIE 1931) è **tridimensionale** e include la luminanza.
- Spazio di cromaticità xyz (CIE 1931) è **bidimensionale** e descrive solo la cromaticità, senza tener conto della quantità di luce emessa o riflessa.

Lo spazio di colore CIE Lab

Il principale difetto del sistema CIE e dei modelli di colore derivati da esso tramite trasformazioni lineari o non lineari, è la **mancanza di uniformità percettiva**.

Un sistema di colore è percettivamente uniforme quando due colori che hanno la **stessa distanza numerica** in quello spazio risultano essere **ugualmente distinti per l'occhio umano**.

Alcune regioni dello spettro risultano molto più **comprese o più dilatate**.

Di conseguenza, se prendiamo due colori **C1** e **C2** con la stessa distanza numerica **ΔC**, potremmo percepirli con una differenza visiva diversa a seconda della loro posizione nello spazio di colore.

Nel 1976, la CIE ha standardizzato due **spazi di colore percettivamente uniformi** CIE Luv e **CIE Lab**:

CIE Lab

Il sistema **CIE Lab** è stato sviluppato nel **1976** dal CIE per creare uno spazio di colore **percettivamente uniforme**.

A differenza di altri modelli come il CIE Lab è **indipendente dal dispositivo**.

Luminanza e cromaticità stanno su canali separati:

- L^* : Luminanza, che varia da 0 (nero) a 100 (bianco)
- a^* : Coordinata di cromaticità che varia dal verde (-) al rosso (+)
- b^* : Coordinata di cromaticità che varia dal blu (-) al giallo (+)

È possibile passare dallo spazio di colore CIE XYZ allo spazio di colore CIELAB e viceversa con le seguenti formule:

$$(luminanza) L^* = 116 \left(\frac{Y}{Y_n} \right)^{\frac{1}{3}} - 16; \quad a^* = 500 \left[\left(\frac{X}{X_n} \right)^{\frac{1}{3}} - \left(\frac{Y}{Y_n} \right)^{\frac{1}{3}} \right]; \quad b^* = 200 \left[\left(\frac{Y}{Y_n} \right)^{\frac{1}{3}} - \left(\frac{Z}{Z_n} \right)^{\frac{1}{3}} \right]$$

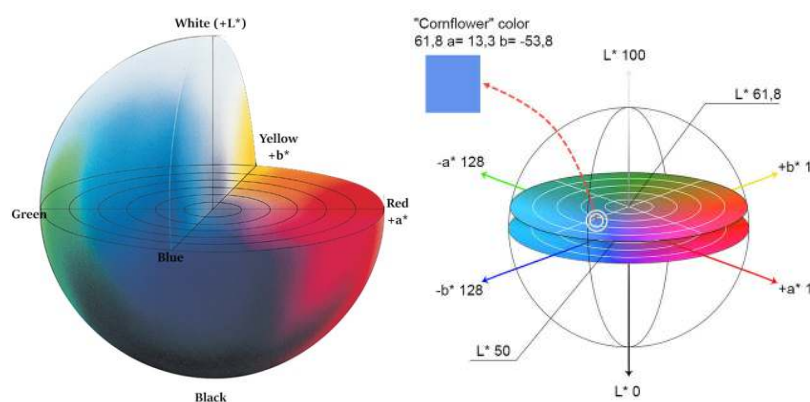
- I valori $\frac{X}{X_n}$, $\frac{Y}{Y_n}$ e $\frac{Z}{Z_n}$ sono sempre > 0.01 ;
- I valori X_n , Y_n e Z_n rappresentano il punto bianco;

Nello spazio di colore CIELAB le differenze di colore sono definite come distanze fra due punti:

$$(metrica) \Delta E_{ab}^* = \sqrt{\Delta L^{*2} + \Delta a^{*2} + \Delta b^{*2}}$$

Uguali differenze corrispondono a uguali differenze di percezione.

CIE Lab ha una forma sferica.



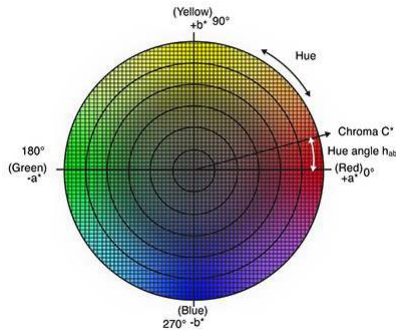
CIE L C h:

Sempre nel 1976 la CIE ha standardizzato un altro spazio di colore percettivamente uniforme, detto CIE L C h, che usa lo stesso diagramma del CIE Lab, in cui:

- L : rappresenta la luminanza o chiarezza del colore, su una scala da 0 (nero) a 100 (bianco),
- C : rappresenta la saturazione
- h : rappresenta la tinta

C e h sono coordinate polari, rispettivamente distanza radiale dal centro e angolo (espresso in gradi da 0° a 360°)

$$C^* = \sqrt{a^{*2} + b^{*2}}; \quad h^* = \tan^{-1}\left(\frac{b^*}{a^*}\right)$$



Il bianco e il nero matematico e fisico

1. Nero Matematico

- **Definizione:** Il nero matematico è rappresentato dalle **coordinate del tristimolo**:
 $X = 0, Y = 0, Z = 0$
- Nessun materiale reale possiede queste proprietà, poiché ogni materiale riflette almeno una minima quantità di luce.

2. Nero Fisico

- **Definizione:** Il nero fisico si riferisce a un materiale reale che riflette pochissima luce. Il fattore di riflessione spettrale per questi materiali è tipicamente molto basso (nell'ordine del 3-5%).

1. Bianco Teorico

Il bianco teorico è un concetto ideale in colorimetria, rappresentato da un'illuminazione con energia uguale per tutte le lunghezze d'onda. Ha coordinate tristimolo $(X, Y, Z) = (1, 1, 1)$ e coordinate cromatiche $(x, y, Y) = (\frac{1}{3}, \frac{1}{3}, 1)$, indicando un punto neutro nello spazio cromatico.

2. Bianco Fisico

Il **bianco fisico reale** dipende dall'illuminante scelto e dalle caratteristiche riflettive del materiale.

Percezione del Colore e Spettri di Radiazione

In natura, raramente osserviamo **colori puri**, ovvero luce con una singola lunghezza d'onda. I colori che vediamo sono solitamente **miscele** di diverse lunghezze d'onda. Il nostro sistema visivo non agisce come uno spettrometro, che distingue ogni lunghezza d'onda separatamente.

Piuttosto, il cervello combina le lunghezze d'onda in una singola percezione cromatica, riuscendo a mantenere una percezione costante del colore di una superficie, anche quando cambia la luce che la illumina. Questo fenomeno è noto come **costanza del colore**.

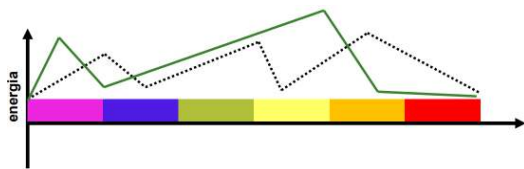
L'energia totale di una sorgente luminosa è la somma delle energie di tutte le lunghezze d'onda che la compongono.

Ogni sorgente di luce emette energia a diverse lunghezze d'onda, creando uno **spettro** che può essere analizzato in due modi principali:

- **Spettro continuo** → Contiene tutte le lunghezze d'onda in un intervallo (La luce solare, che copre tutte le lunghezze d'onda visibili)
- **Spettro discreto** → Composto da picchi a lunghezze d'onda specifiche (Lampade fluorescenti o LED, che emettono solo certe lunghezze d'onda.)

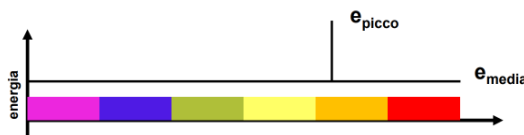
Metameri

I metameri sono coppie di spettri differenti che producono la stessa percezione cromatica, nonostante la loro diversa composizione di lunghezze d'onda.



Lo spettro tratteggiato e quello continuo producono (nel cervello) il medesimo colore

Ogni spettro ha un metamero della seguente forma:



Il modello del pittore e lo Spazio HSV (o HSI)

Modello del pittore:

Tra i vari metameri di un dato spettro se ne può individuare uno molto importante, alla base del modello dei colori detto “**modello del pittore**”.

Questo modello semplificato permette di descrivere qualsiasi colore usando solo tre parametri:

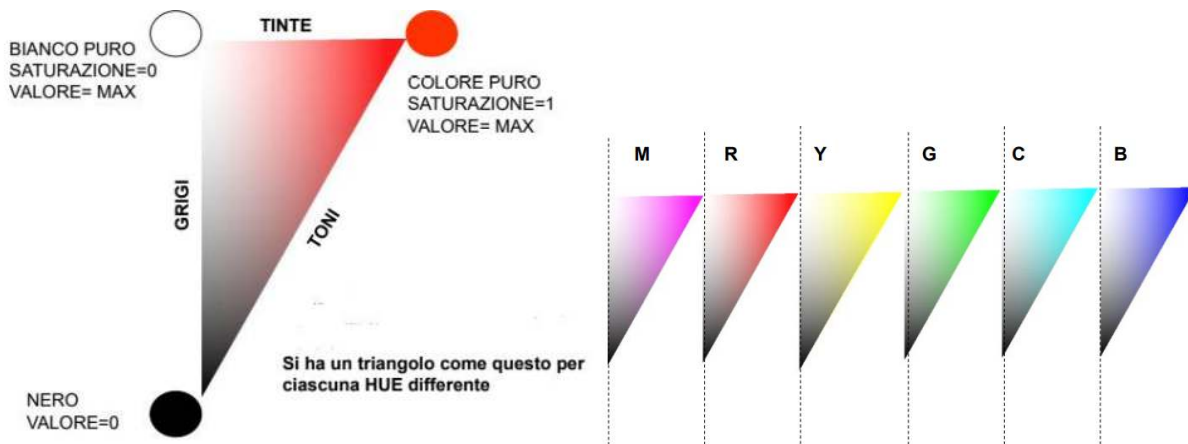
- **Hue (Tinta):**
La **lunghezza d'onda** in cui si osserva il **picco massimo** di energia nello spettro determina il colore percepito, ovvero la **tinta**.
- **Saturazione:**
La **saturazione** indica quanto un colore è puro o intenso.
Viene calcolata come:
$$\text{Saturazione} = \frac{e_{\text{picco}} - e_{\text{media}}}{e_{\text{picco}} + e_{\text{media}}}$$

Un valore maggiore di questo rapporto indica un colore più puro (meno luce bianca mescolata)

- **Luminosità (Valore):**

Dato dall'energia media e_{media} dello spettro. Se il valore di e_{media} è alto, significa che c'è più **luce bianca** mescolata al colore, rendendolo meno saturo.

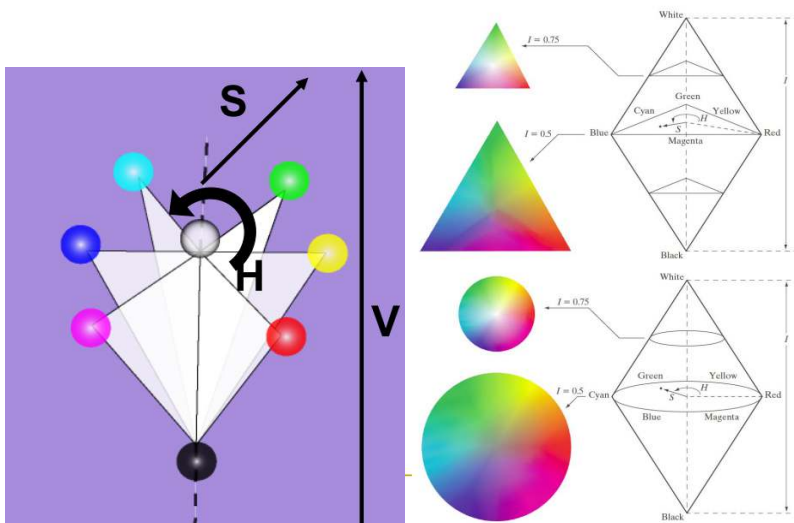
Mettendo insieme i tre concetti per ogni possibile colore si crea un triangolo.



Unendo i triangoli che rappresentano i colori delle varie tinte (corrispondenti alle diverse lunghezze d'onda) attraverso la linea dei grigi in comune, otteniamo lo **spazio HSV**, nel quale i colori vengono rappresentati in uno spazio a forma di **cono**.

Il modello **HSV** descrive i colori attraverso tre parametri:

- **H (Hue):** La tinta, o il colore percepito.
- **S (Saturation):** La saturazione, che indica l'intensità del colore. Al massimo (periferia), il colore è puro, mentre al minimo (centro), il colore è grigio.
- **V (Value):** Il valore o luminosità, che rappresenta la quantità di luce presente nel colore, variando dal nero (centro) al massimo di luminosità (periferia).



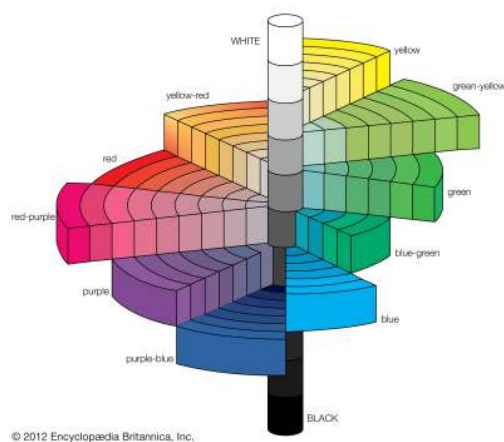
Pro: è intuitivo ed è percettivamente significato (i parametri hanno una perfetta interpretazione nella nostra percezione)

Contro: Nel modello HSV non esiste un numero definito di colori base come in RGB o CMY, inoltre; non è percettivamente uniforme.

Munsell System (H, C, V)

Sulla stessa ideologia esiste il **Munsell System**. Questo sistema di colori è costruito attorno a un **cilindro tridimensionale** che rappresenta i colori in modo percettivamente uniforme, ovvero i cambiamenti di colore sono percepiti in modo costante rispetto alle tre dimensioni del modello.

- **Tinta (Hue):** Rappresentata dall'angolo attorno all'asse centrale del cilindro.
- **Croma (Chroma):** Misura quanto un colore è "vivo" o "intenso"; più si è vicini al bordo, maggiore è la saturazione.
- **Luminosità (Value):** Corrisponde all'altezza nel cilindro; i colori più luminosi si trovano in alto, quelli più scuri in basso.



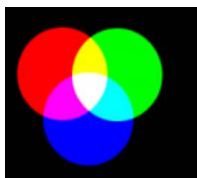
Modello RGB e sintesi additiva

Nel sistema di colori RGB, i colori si formano per sintesi additiva partendo dai tre colori primari: rosso R, verde G e blu B. La somma di tutti e tre i colori produce il **bianco** (W).

Colori Secondari:

- La combinazione di due colori primari genera i cosiddetti colori secondari, complementari dei primari RGB, che formano lo **spazio di colori CMY**:
 - Rosso + Verde = **Giallo** (Y)
 - Rosso + Blu = **Magenta** (M)
 - Verde + Blu = **Ciano** (C)

In sintesi additiva, il colore complementare di uno primario è quello che, combinato ad un secondario, produce il bianco.



Il **Giallo** è complementare del **Blu**
Il **Magenta** è complementare del **Verde**
Il **Ciano** è complementare del **Rosso**

- Sebbene RGB consenta di riprodurre una vasta gamma di colori, non riesce a coprire tutti i colori percepibili dall'occhio umano.

Pro: semplice da usare e implementare in software e hardware.

Contro: è percettivamente poco comodo, dato che è difficile capire quando si guarda un colore in natura in quali proporzioni vi contribuiscano R, G e B.

La somma tra un'immagine RGB e la sua immagine negativa è sempre una matrice di bianchi, poiché ogni componente di colore risulta essere 255

Modello CMY e sintesi sottrattiva

La **sintesi sottrattiva** è un metodo di combinazione dei colori utilizzato principalmente nella stampa e nella fotografia. Questo processo si basa sull'uso di **filtri ottici colorati**, che sono mezzi trasparenti in grado di assorbire selettivamente determinate lunghezze d'onda della luce, assumendo il colore prodotto dalla radiazione complementare di quella assorbita.

(Ad esempio, il filtro magenta assorbe la componente verde della luce, facendo passare il rosso e il blu, che combinandosi "creano" il color magenta.)

Quando si sovrappongono più filtri colorati su una **sorgente di luce bianca**, il colore risultante si ottiene tramite **sottrazione delle componenti luminose**:

- **Sovrapposizione di tutti e tre i filtri (C + M + Y):** Il filtro ciano assorbe il rosso, il magenta assorbe il verde e il giallo assorbe il blu. Di conseguenza, tutte le lunghezze d'onda della luce bianca vengono assorbite ed il risultato è la percezione di **nero (K)**.
- Sovrapponendo **due filtri** si ottiene il colore corrispondente alla componente luminosa che non viene assorbita da nessuno dei due. (esempio Ciano + Magenta = Blu)

Y + M = R
Y + C = G
M + C = B
Y + M + C = K



Poiché la combinazione di C, M e Y **non genera un nero puro**, nella stampa si aggiunge il colore **nero (K, "Key")**, ottenendo il modello **CMYK**, che garantisce una migliore qualità di stampa.

Dal modello RGB al CMY

Il modello **CMY (Ciano, Magenta, Giallo)** è ottenuto dal modello **RGB (Rosso, Verde, Blu)** tramite la formula:

$$C = 255 - R$$

$$M = 255 - G$$

$$Y = 255 - B$$

- Esercizio: Dato il colore RGB (128,128,128). Quali sono le corrispondenti coordinate CMY?

Data la formula prima citata, avremo che:

$$C = 255 - 128 = 127$$

$$M = 255 - 128 = 127$$

$$Y = 255 - 128 = 127$$

Dal modello CMY al RGB

Per convertire un colore da **CMY** a **RGB**, si utilizza la formula inversa:

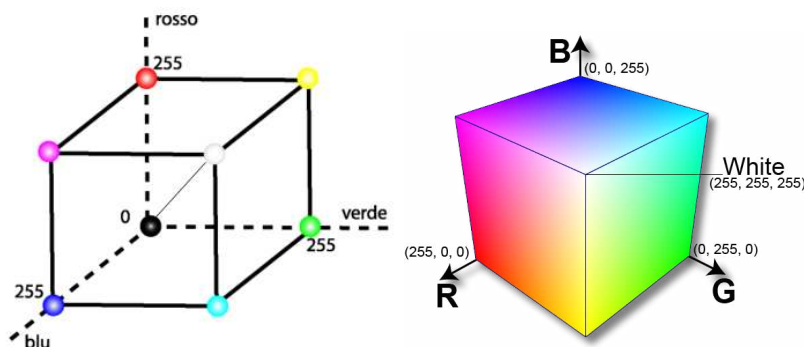
$$R = 255 - C$$

$$G = 255 - M$$

$$B = 255 - Y$$

RGB come cubo

Il modello RGB può essere rappresentato graficamente come un **cubo tridimensionale** in uno spazio cartesiano. I contributi del Red, Green e Blue sono assunti indipendenti l'uno dall'altro (e quindi rappresentanti da direzioni perpendicolari tra loro). La retta che congiunge nero e bianco è la **retta dei grigi**.



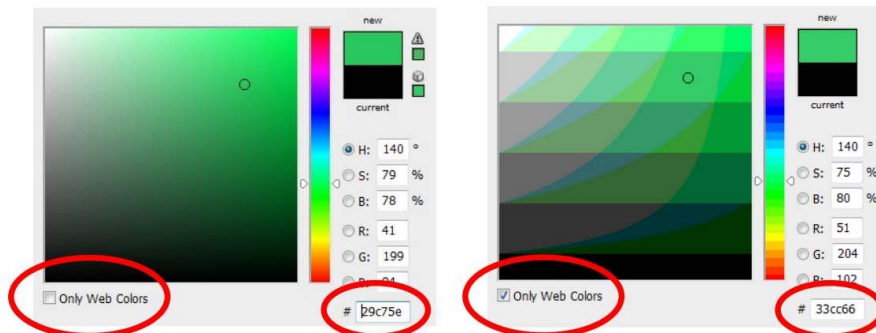
Colori sicuri per il web

I **colori sicuri per il web** sono un insieme di 216 colori RGB che sono stati scelti per garantire una **visualizzazione uniforme** su diversi dispositivi e sistemi operativi. Si utilizzano solo sei valori possibili per ogni componente RGB: **00**, **33**, **66**, **99**, **CC** e **FF**.

- Questi valori rappresentano una scala di luminosità standardizzata, passando dal minimo (**00**, che è nero) al massimo (**FF**, che è il massimo dell'intensità).

Quindi sono colori sicuri tutti quelli che in esadecimale sono scritti usando terne con questi 6 possibili lavori: ad esempio **#33CCFF**

- Avendo 6 possibili valori per ciascuna componente RGB:
 $6 \times 6 \times 6 = 216$ colori sicuri per il web



Rappresentazioni luminanza-crominanza

Gli **spazi colore** in cui una componente rappresenta la **luminanza** (ovvero la luminosità o intensità della luce percepita) e le altre due rappresentano la **crominanza** (che contengono le informazioni sui colori) vengono chiamate rappresentazioni luminanza-crominanza. Sono particolarmente importanti nella compressione delle immagini.

Lo spazio YUV

Lo spazio YUV è una *famiglia di spazi* usata per la codifica di immagini o video analogici. Lo spazio colore **YUV** separa l'immagine in:

- **Y (luminanza)**: Informazione di luminosità, che rappresenta l'intensità luminosa percepita.
- **U e V**: componenti di crominanza che catturano le differenze di colore rispetto al bianco alla stessa luminanza, descrivendo quanto un pixel si avvicina al blu o al rosso.

(Da RGB a YUV)

La luminanza Y può essere ottenuta mediante una combinazione lineare delle intensità luminose dei canali rosso, verde e blu di RGB, nella quale la componente verde ha il maggior contributo.

- $Y = 0.299 \times R + 0.587 \times G + 0.114 \times B$

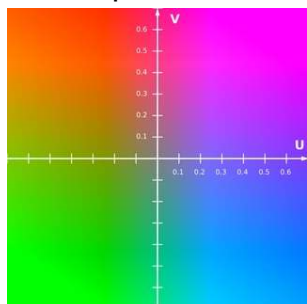
Le componenti U e V sono correlate alla componente Y

- $U = 0.6 \times (B - Y) = -0.2 \times R - 0.3 \times G + 0.5 \times B$
- $V = 0.7 \times (R - Y) = 0.5 \times R - 0.4 \times G - 0.1 \times B$

Se $R = G = B$, allora U e V valgono 0 e si ottengono solo grigi

Se i valori di R, G, e B sono compresi tra **0** e **1**:

- La luminanza **Y** sarà anch'essa compresa tra **0** e **1**.
- Le componenti di cromaticità **U** e **V** avranno valori compresi tra **-0.5** e **0.5**.



Lo standard *YCbCr*

Lo spazio *YCbCr* è una famiglia di spazi che costituiscono la controparte digitale dello spazio YUV. Il passaggio da **YUV** a **YCbCr** avviene attraverso la **normalizzazione** e **quantizzazione** dei valori, in modo da adattarli a un intervallo compreso tra **0** e **255**.

- La componente **Y** è la stessa per entrambi gli spazi colore **YUV** e **YCbCr**. Viene calcolata come:

$$Y = 0.299 \times R + 0.587 \times G + 0.114 \times B$$
- Per trasformare le componenti di cromaticità da **U** e **V** a **Cb** e **Cr**, si applica uno **shift** (spostamento) di **128** unità:

$$Cb = 128 + U$$

$$Cr = 128 + V$$

Colori e memoria

Nello schema RGB a **24 bit**, ogni pixel è rappresentato con:

- **8 bit per il canale Rosso (Red)**
- **8 bit per il canale Verde (Green)**
- **8 bit per il canale Blu (Blue)**

Quindi, un singolo pixel può rappresentare **2²⁴** (circa **16.7 milioni**) di colori, che è noto come **true color**. In questa modalità, ogni pixel ha il suo colore specifico senza alcuna limitazione nel numero di colori distinti. Tuttavia, usare 24 bit per ogni pixel può essere costoso in termini di memoria.

Per risparmiare memoria si utilizza una tecnica chiamata **colori indicizzati** o **palette** o **look-up table (LUT)**.

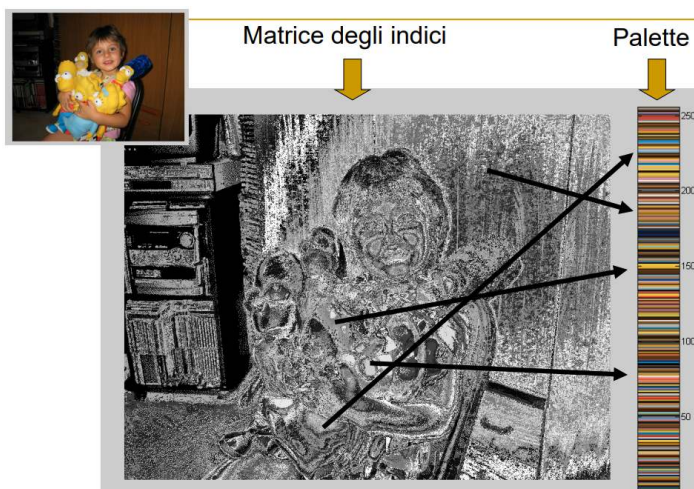
- Si crea una **palette** che contiene un insieme limitato di colori (tipicamente **256** colori).
- Ogni colore nella palette è rappresentato con una terna RGB (8 bit per componente).
- Invece di memorizzare direttamente i valori RGB per ogni pixel, si memorizza un **indice** che punta a un colore nella palette. Ogni pixel è rappresentato quindi da un valore a **8 bit** che indica il colore corrispondente nella palette.

00	00	01	00 = (255, 0, 0)
00	00	01	01 = (0, 255, 0)
11	10	00	10 = (255, 255, 255)
			11 = (0, 0, 255)

Ricordo queste "etichette" e questa tabella

Molti software adottano una palette a 256 colori. L'utilizzo di una palette *custom* sarà così definita:

- Se nell'immagine true color ci sono meno di 256 colori, alcuni di essi verranno replicati (quindi non c'è perdita di informazione);
- Se nell'immagine true color ci sono più di 256 colori, essi verranno ridotti, e verranno scelti solo 256 colori rappresentativi che garantiscano una buona qualità visiva (c'è perdita di informazione);



Re-indexing

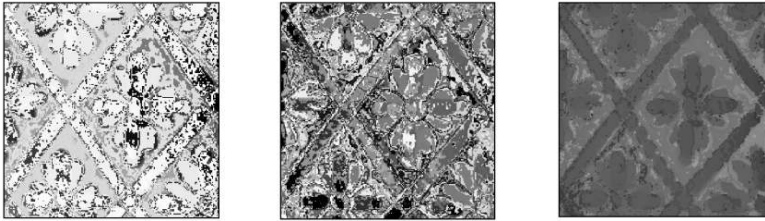
Il **re-indexing** è una tecnica utilizzata per ottimizzare la disposizione dei colori in una palette, al fine di ridurre l'entropia dell'immagine e **migliorare l'efficienza di compressione**. L'idea centrale è quella di **riordinare i colori** all'interno della palette e aggiornare gli indici associati ai pixel in modo che:

- I **colori simili** abbiano **indici numericamente vicini**.
- Le differenze tra indici di pixel adiacenti siano più piccole possibile.

Entropia di un'immagine

L'entropia di un'immagine è una misura della sua complessità o disordine. Un'immagine con **alta entropia** ha molti cambiamenti nei valori dei pixel (molti colori diversi e distribuiti casualmente), mentre un'immagine con **bassa entropia** ha meno variazioni tra pixel adiacenti.

Il problema del riordinamento ottimale della palette per ottenere l'entropia minima è **NP-hard**. Per M colori, ci sono **$M!$** possibili combinazioni di riordinamenti.



Operazioni sulle immagini

L'istogramma

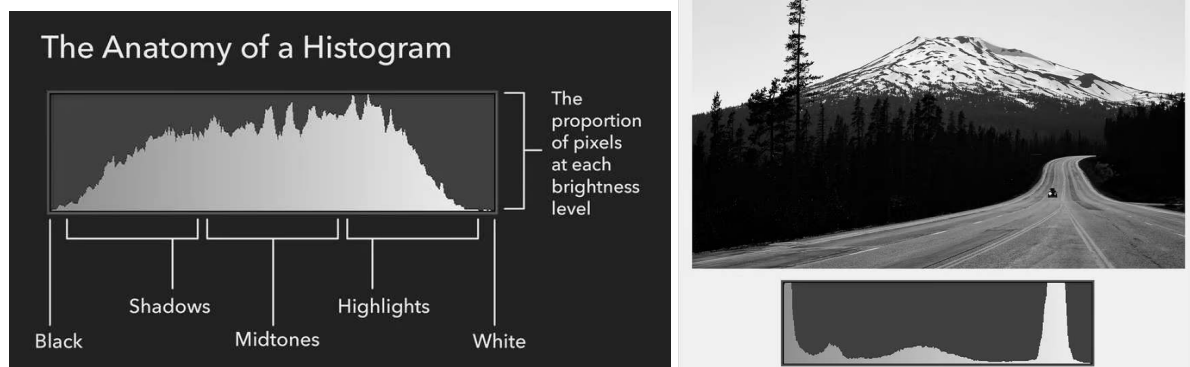
L'**istogramma di un'immagine** mostra la distribuzione dei valori di intensità dei pixel grigi in un'immagine.

Per un'immagine $I[m, n]$ si ha che per ogni livello di grigio k :

$$H(k) = \text{numero di pixel del valore } k$$

La somma di tutti gli H è $m \times n$.

L'istogramma è utile a comprendere in maniera immediata le caratteristiche dell'immagine.



- **Denso a destra:** immagine chiara, con molti pixel di alta intensità (vicini al bianco).
- **Denso a sinistra:** immagine scura, con molti pixel di bassa intensità (vicini al nero).

L'**istogramma non considera la disposizione spaziale** dei pixel: due immagini diverse possono avere lo stesso istogramma.

Contrast stretching (stretching del contrasto)

Il **contrast stretching** aumenta la dinamica di un'immagine espandendo i valori di intensità su tutto l'intervallo disponibile. Ad esempio, in un'immagine in scala di grigi con valori tra 50

e 200, il contrast stretching mappa il valore minimo a 0 (nero) e quello massimo a 255 (bianco), garantendo così che l'immagine risultante contenga sicuramente entrambi questi estremi, oltre a una distribuzione più ampia dei livelli intermedi. Si migliora così la percezione del contrasto si rendono i dettagli più evidenti.

Aritmetica sulle immagini

Applicando operazioni aritmetiche su un pixel, esso può assumere:

- a) Un valore negativo
- b) Un valore maggiore del massimo (che è di solito 255)
- c) Un valore non intero (float - facilmente, risolvibile con un'approssimazione o troncamento)

Normalizzazione

I problemi a) e b) sono detti **problemi di range**, per i quali le due soluzioni più comuni sono:

- Settare a 0 (nero) i valori negativi ed a 255 (bianco) i valori maggiori di 255
- Ri-normalizzare il range, trasformando ciascun valore mediante l'equazione:

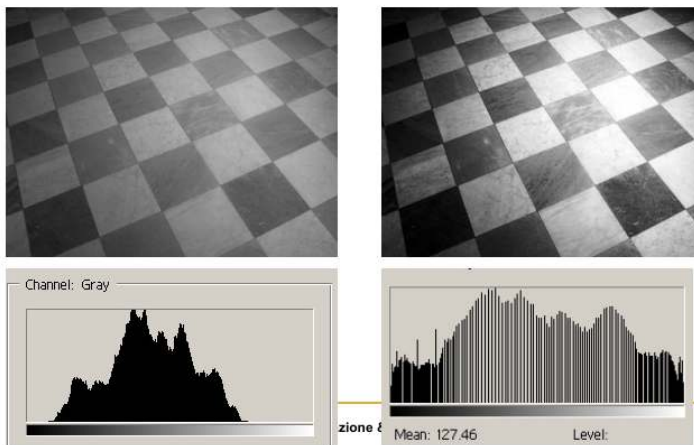
$$v_{nuovo} = 255 \cdot \frac{(v_{vecchio} - min_{osservato})}{(max_{osservato} - min_{osservato})}$$

Equalizzazione

Un'immagine è **equalizzata** quando il contributo di ogni differente tonalità di grigio è pressappoco uguale. In questo caso l'istogramma è *uniforme* o *appiattito*.

- L'equalizzazione si può ottenere tramite appositi algoritmi, ma essa non sempre migliora l'immagine.

Se due immagini diverse hanno lo stesso istogramma iniziale, dopo l'equalizzazione avranno ancora lo stesso istogramma.



Algoritmo di equalizzazione

La formula:

$$p_r(r_k) = \frac{n_k}{MN} \quad k = 0, 1, 2, \dots, L - 1$$

in cui:

- r_k è un livello di grigio
- n_k è il numero di pixel di quel livello di grigio nell'immagine $N \cdot M$

Calcola la **probabilità di frequenza** p_r di un dato livello di grigio nell'immagine. In pratica, stima quanto spesso compare un certo livello r_k rispetto al totale.

Se facciamo il plot di r_k versus $p_r(r_k)$, cioè

- **Sull'asse orizzontale (x)** riporti r_k , i valori dei livelli di grigio.
 - **Sull'asse verticale (y)** riporti $p_r(r_k)$, le probabilità associate a quei livelli di grigio.
- si ottiene l'istogramma dell'immagine

I nuovi valori di grigio dell'istogramma sono così definiti:

$$\begin{aligned} s_k &= T(r_k) = (L - 1) \sum_{j=0}^k p_r(r_j) \\ &= \frac{(L - 1)}{MN} \sum_{j=0}^k n_j \quad k = 0, 1, 2, \dots, L - 1 \end{aligned}$$

- $T(r_k)$: è la funzione di trasformazione che calcola il nuovo valore del livello di grigio s_k in funzione dei valori originali.
- $L-1$: rappresenta il massimo livello di intensità disponibile (es., 255 per immagini a 256 livelli).
- La sommatoria è la funzione di distribuzione cumulativa (CDF) dei livelli di grigio fino al livello k .
- $p_r(r_j) = n_j / MN$: è la frequenza relativa del livello di grigio r_j

- n_j : è il numero di pixel con livello di intensità r_j

Esempio

- Sia data una immagine a 3 bit (L=8) con 64x64 pixel (MN=4096) con la seguente distribuzione di intensità:

r_k	n_k	$p_r(r_k) = n_k/MN$
$r_0 = 0$	790	0.19
$r_1 = 1$	1023	0.25
$r_2 = 2$	850	0.21
$r_3 = 3$	656	0.16
$r_4 = 4$	329	0.08
$r_5 = 5$	245	0.06
$r_6 = 6$	122	0.03
$r_7 = 7$	81	0.02

21

- Applicando la formula si ha:

$$s_0 = T(r_0) = 7 \sum_{j=0}^0 p_r(r_j) = 7p_r(r_0) = 1.33$$

$$s_1 = T(r_1) = 7 \sum_{j=0}^1 p_r(r_j) = 7p_r(r_0) + 7p_r(r_1) = 3.08$$

$$s_2 = 4.55, s_3 = 5.67, s_4 = 6.23, s_5 = 6.65, s_6 = 6.86, s_7 = 7.00.$$

Arrotondando:

$$\begin{aligned} s_0 &= 1.33 \rightarrow 1 & s_4 &= 6.23 \rightarrow 6 \\ s_1 &= 3.08 \rightarrow 3 & s_5 &= 6.65 \rightarrow 7 \\ s_2 &= 4.55 \rightarrow 5 & s_6 &= 6.86 \rightarrow 7 \\ s_3 &= 5.67 \rightarrow 6 & s_7 &= 7.00 \rightarrow 7 \end{aligned}$$

$$\begin{aligned} s_0 &= 1.33 \rightarrow 1 & s_4 &= 6.23 \rightarrow 6 \\ s_1 &= 3.08 \rightarrow 3 & s_5 &= 6.65 \rightarrow 7 \\ s_2 &= 4.55 \rightarrow 5 & s_6 &= 6.86 \rightarrow 7 \\ s_3 &= 5.67 \rightarrow 6 & s_7 &= 7.00 \rightarrow 7 \end{aligned}$$

Questi sono i valori dell'istogramma equalizzato. Si osservi che ci sono solo cinque livelli distinti. Dato che $r_0 = 0$ è stato trasformato in $s_0 = 1$, ci sono 790 pixel nell'immagine dell'istogramma equalizzato con questo valore (vedi Tabella 3.1). Inoltre, in questa immagine, ci sono 1023 pixel con valore di $s_1 = 3$ e 850 pixel con il valore di $s_2 = 5$. Sia r_3 che r_4 sono stati trasformati nello stesso valore, così ci sono $(656 + 329) = 985$ pixel nell'immagine equalizzata con questo valore. In modo simile, ci sono $(245 + 122 + 81) = 448$ pixel con il valore di 7 nell'immagine equalizzata. Dividendo questi numeri per $MN = 4096$ si ottiene l'istogramma equalizzato della Figura 3.19c.

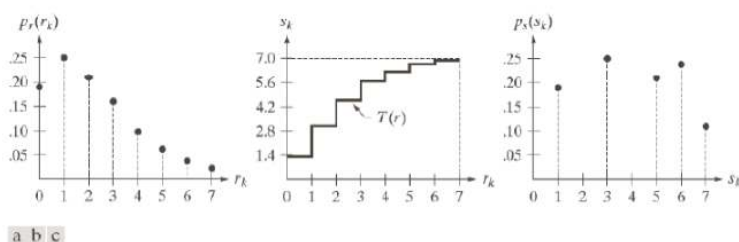


FIGURE 3.19 Illustration of histogram equalization of a 3-bit (8 intensity levels) image. (a) Original histogram. (b) Transformation function. (c) Equalized histogram.

Operazioni sulle immagini

Per semplificare lavoreremo solo su immagini a toni di grigio, tuttavia le operazioni che stiamo per descrivere si estendono alle immagini RGB lavorando separatamente sui tre canali R, G e B trattati come un'immagine a toni di grigio.

- Queste operazioni alterano i valori dei pixel di un'immagine.

L'equazione:

$$g(x, y) = T[f(x, y)]$$

descrive un'operazione nel dominio spaziale applicata a un'immagine digitale.

- Lavorare nel *dominio spaziale* significa lavorare direttamente sui pixel dell'immagine.
- $f(x, y)$ è l'immagine di ingresso
- $g(x, y)$ è l'immagine di uscita
- T è un operatore su f definito in un intorno (x, y) (che trasforma f in g)

Tipi di operazioni

La dimensione dell'intorno (x, y) definisce il carattere dell'elaborazione:

- **Puntuale:** l'intorno coincide con il pixel specifico (x, y) , senza considerare i pixel vicini;
- **Locale:** l'elaborazione dipende anche dai pixel circostanti di (x, y)
- **Globale:** l'intorno coincide con tutta l'immagine f .

Operatori puntuali

Sono operatori che, preso in input il valore di un pixel, restituiscono un valore modificato che dipende esclusivamente dal valore del pixel in ingresso.

Operazioni puntuali

- Aggiunta o sottrazione di una costante a tutti i pixel (compensazione di sotto o sovraesposizioni): $g(x, y) = f(x, y) + c$
- Inversione della scala di grigi (negativo)
- Espansione del contrasto
- Modifica (equalizzazione o specifica) dell'istogramma
- Presentazione in falsi colori

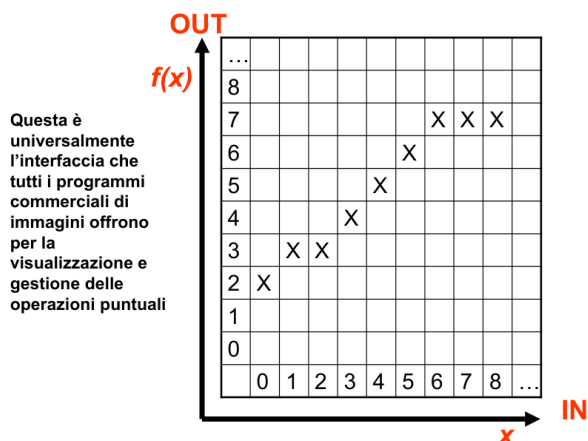
Definizione di operatore puntuale

Un **operatore puntuale** agisce su un singolo punto (pixel) dell'immagine o dato senza considerare i valori circostanti. Questo può essere rappresentato da una funzione matematica T che mappa un valore $f(x, y)$ in un nuovo valore $g(x, y)$ (con i due valori appartenenti allo stesso campo, ad esempio entrambi tra 0 e 255)

$$g(x, y) = T(f(x, y))$$

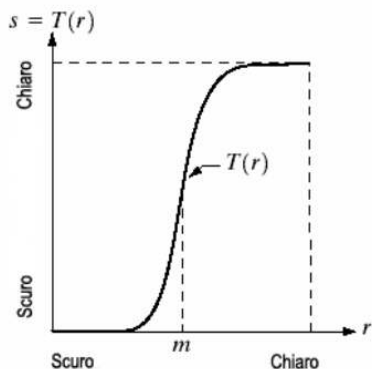
Un operatore puntuale è descritto da una tabella:

IN	0	1	2	3	4	5	6	7	...
OUT	T(0)	T(1)	T(2)	T(3)	T(4)	T(5)	T(6)	T(7)	...



LUT

Questo grafico si chiama **look-up-table** (LUT). Rappresenta una funzione di trasferimento tonale, mappa i valori di grigio in ingresso (r) nei corrispondenti valori in uscita (s), trasformando la luminosità o il contrasto di un'immagine. Descrive univocamente la risposta a un operatore puntuale.



Negativo

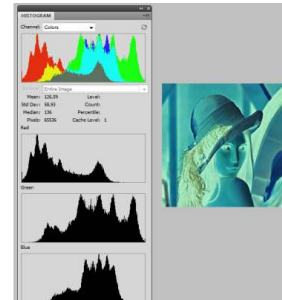
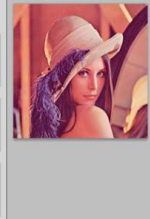
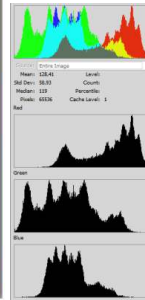
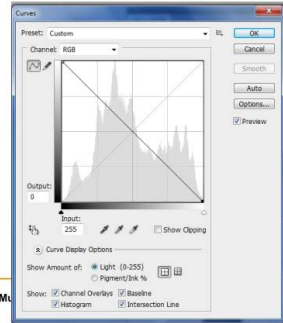
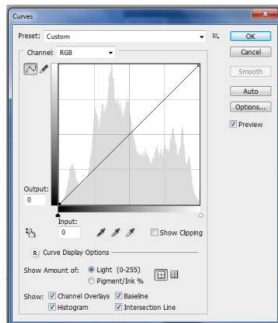
L'operazione del negativo è un'operazione puntuale, invariante per traslazione e lineare, nella quale si invertono i valori dell'intensità di ogni pixel dell'immagine originale.

- Non è lineare

Associa al valore $f(x, y)$ del pixel il valore $255 - f(x, y)$



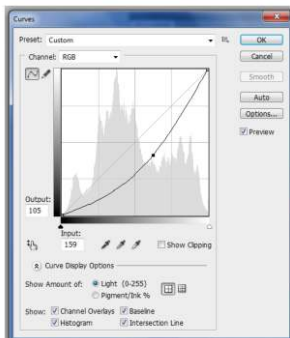
La curva cambia in:



Come si può notare, anche l'istogramma dell'immagine risulta invertito

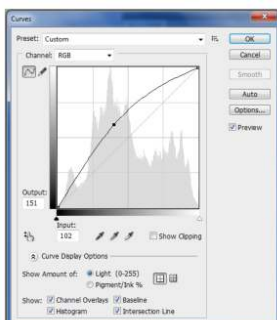
Incupimento

Per incupire un'immagine si abbassa la curva nella LUT



Schiarimento

Per schiarire un'immagine si alza la curva nella LUT



Trasformazione logaritmica (Operatore logaritmo)

La trasformazione logaritmica è una tecnica di elaborazione delle immagini utilizzata per **comprimere la gamma dinamica** delle intensità. Questo operatore è particolarmente utile quando l'immagine presenta una gamma dinamica molto ampia, ovvero quando ci sono

aree estremamente scure e altre estremamente luminose, rendendo difficile visualizzare i dettagli in entrambe le regioni.

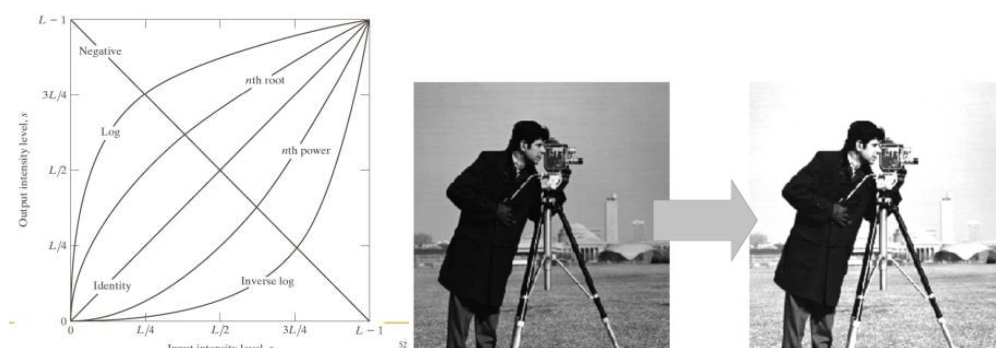
È espressa come:

$$g(x, y) = c \log_2(1 + f(x, y))$$

- c è una costante positiva che serve a normalizzare il risultato tra 0 e 255
- $c = \frac{255}{\log(1+255)}$ per un'immagine 8 bit

Intervalli:

- $f \in [0, 255]$
- $1 + f \in [1, 256]$
- $\log(1 + f) \in [0, 8]$
- $c \log_2(1 + f) \in [0, 255]$

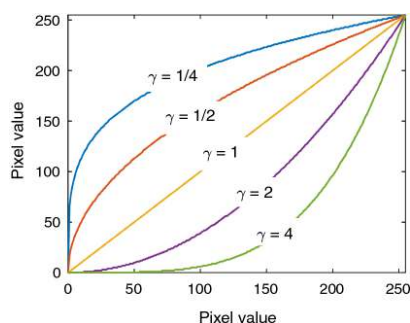


Se due immagini hanno lo stesso istogramma, applicare il logaritmo a entrambe produrrà due immagini con lo stesso istogramma, perché il logaritmo trasforma i valori dei pixel in modo identico per entrambe le immagini.

Trasformazione di potenza (gamma)

La **trasformazione di potenza**, o **correzione gamma**, è una tecnica di elaborazione delle immagini utilizzata per regolare l'intensità dei pixel in modo non lineare. Questa trasformazione è particolarmente utile per controllare il contrasto e la luminosità di un'immagine.

$$g(x, y) = c(f(x, y))^\gamma$$



- Per $\gamma = 1$ la trasformazione è lineare: l'immagine rimane invariata.
- Per $\gamma < 1$ (valori sopra la bisettrice): la trasformazione si comporta come la trasformazione logaritmica (espansione della dinamica per valori bassi di f , compressione della dinamica per valori alti di f). **L'immagine si schiarisce.**
- Per le curve con $\gamma > 1$ (valori sotto la bisettrice) accade l'opposto: **l'immagine si incupisce.**

c è una costante di normalizzazione che assicura che i valori di $g(x, y)$ rimangano nell'intervallo desiderato ($[0, 255]$ per immagini a 8 bit)

Intervalli:

- $f \in [0, 255]$
- $f^\gamma \in [0, 255^\gamma]$
- $c \cdot f^\gamma \in [0, 255]$ Applicando la potenza γ , l'intervallo dei valori di intensità cambia. La moltiplicazione per la costante c riporta i valori nell'intervallo desiderato, cioè tra 0 e 255.
- $c = \frac{255}{255^\gamma} = \frac{1}{255^{\gamma-1}}$

Una LUT (Look-Up Table) è un modo efficiente per implementare la trasformazione gamma, poiché mappa direttamente i valori di input ai valori di output senza dover ricalcolare la formula ogni volta.

Binarizzazione

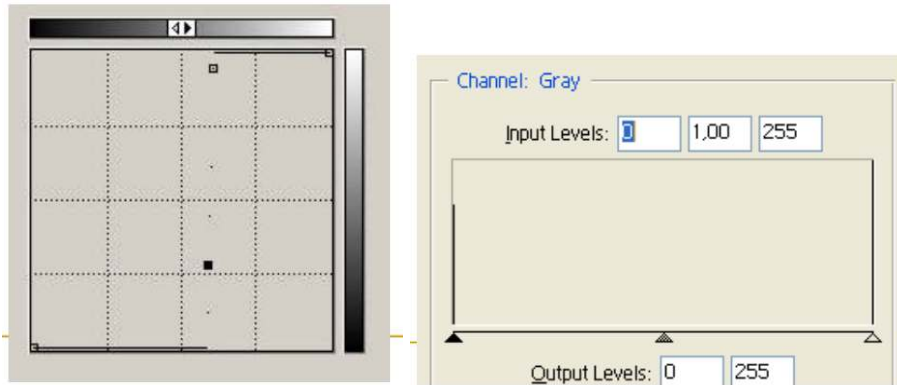
La **binarizzazione** è una tecnica per convertire un'immagine in scala di grigi in un'immagine **binaria** (cioè, con solo due livelli di intensità: **nero** e **bianco**).

Scelta una soglia T :

- **Pixel con intensità maggiore o uguale a T** vengono convertiti in **bianco** (valore 255).
- **Pixel con intensità minore di T** vengono convertiti in **nero** (valore 0).



Curva e Istogramma:



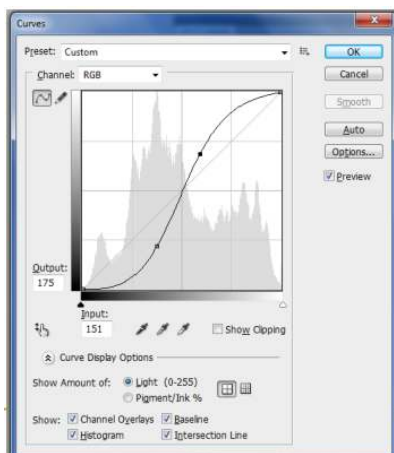
Variazioni di contrasto

Aumentare il contrasto è un'operazione fondamentale nell'elaborazione delle immagini che mira a enfatizzare le differenze di intensità luminosa tra i pixel chiari e quelli scuri, rendendo più evidenti i dettagli visivi.

Aumento di contrasto



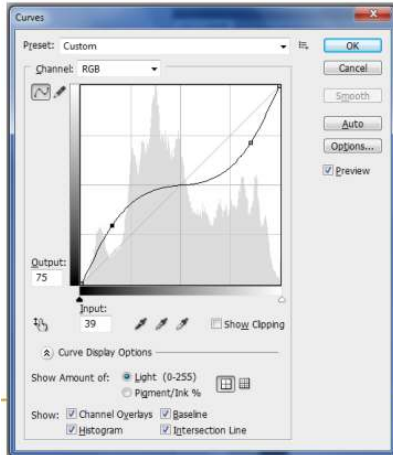
La curva cambia in:



Diminuzione di contrasto



La curva cambia in:



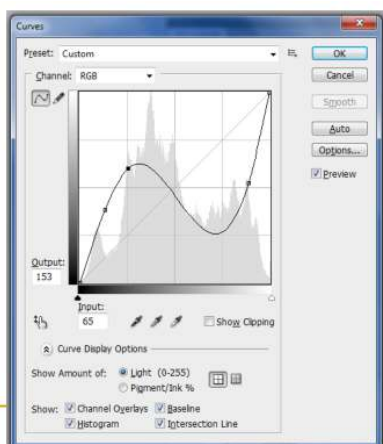
Curve non monotone

È possibile fare delle variazioni alle curve in modo che questa diventi non monotona, il che significa che i valori di intensità non aumentano o diminuiscono in modo uniforme, ma presentano delle **inversioni**.

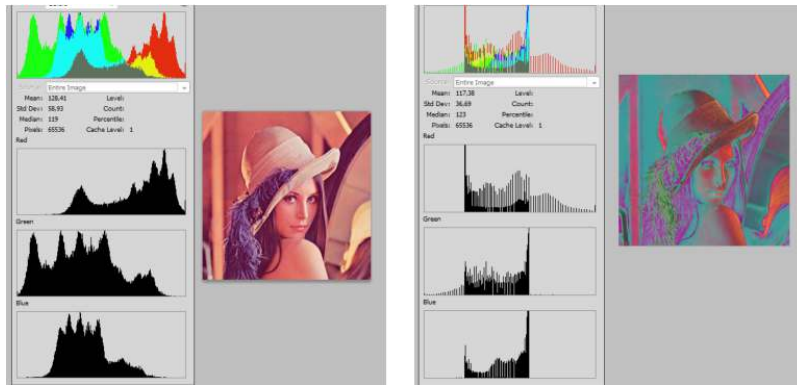
- Un esempio è la *solarizzazione*: effetto visivo che si ottiene invertendo parzialmente i valori di intensità di un'immagine.

Questo effetto crea una curva non monotona, in cui l'intensità dei pixel non segue una progressione uniforme, ma presenta delle inversioni che portano a risultati stilizzati e artistici.

La curva cambia in:



Solarizzazione su un immagine a colori:



10 Operatori locali e convoluzione

Base canonica

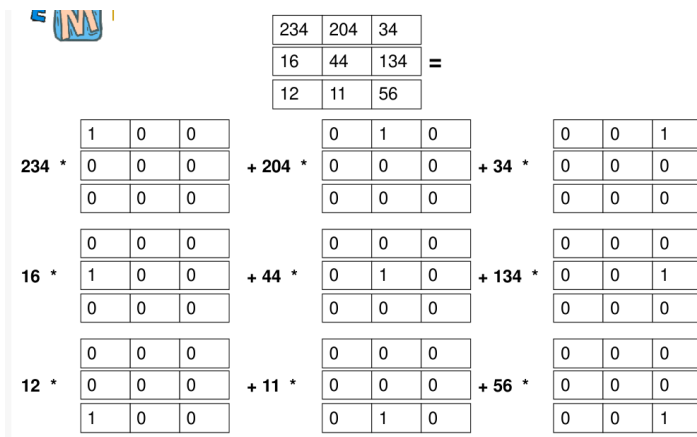
Un vettore di lunghezza N può essere pensato come un elemento di uno spazio N dimensionale

234	204	34	16	44	134	12	11	56
-----	-----	----	----	----	-----	----	----	----

Possiamo quindi scomporlo usando la base canonica di tale spazio:

$$\begin{array}{c}
 \begin{array}{|c|c|c|c|c|c|c|c|c|} \hline 234 & 204 & 34 & 16 & 44 & 134 & 12 & 11 & 56 \\ \hline \end{array} = \\
 \begin{array}{l}
 234 * \begin{array}{|c|c|c|c|c|c|c|c|c|} \hline 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \hline \end{array} + \\
 204 * \begin{array}{|c|c|c|c|c|c|c|c|c|} \hline 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \hline \end{array} + \\
 34 * \begin{array}{|c|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ \hline \end{array} + \\
 16 * \begin{array}{|c|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ \hline \end{array} + \\
 44 * \begin{array}{|c|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ \hline \end{array} + \\
 134 * \begin{array}{|c|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ \hline \end{array} + \\
 12 * \begin{array}{|c|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ \hline \end{array} + \\
 11 * \begin{array}{|c|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ \hline \end{array} + \\
 56 * \begin{array}{|c|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ \hline \end{array}
 \end{array}$$

Cioè la somma di tutti i valori scalari moltiplicati per un vettore che agisce da "maschera". Possiamo fare la stessa cosa con le immagini:



Un operatore si dice **invariante per traslazione (shift invariant)** quando il suo comportamento sulle immagini impulsive è sempre lo stesso, indipendentemente dalla

posizione in cui si trova il pixel.

Se il comportamento di un operatore non è lo stesso su tutti gli elementi della base canonica, quindi se il comportamento varia da elemento a elemento, allora l'operatore **NON è invariante per traslazione**.

Un **impulso** in un'immagine digitale è una matrice (o immagine) in cui tutti i valori sono zero tranne uno, che ha valore 1

Tutti gli operatori puntuali sono invarianti per traslazione (anche se non sono lineari).

Per descrivere questi operatori dovremo considerare il loro comportamento su ciascuna locazione delle immagini.

Operatori invarianti per traslazione

Gli **operatori puntuali** agiscono su ciascun pixel in modo indipendente, senza tenere conto dei pixel circostanti. Questa caratteristica li rende **invarianti per traslazione**: il risultato dell'operatore su un pixel è sempre lo stesso indipendentemente dalla posizione del pixel nell'immagine.

Caratteristiche degli Operatori Lineari e Shift Invariant:

1. Se un operatore F è **lineare** bisogna conoscere il comportamento su tutte le immagini impulsive;
2. Un operatore F è **shift invariant** se il suo comportamento non cambia quando l'immagine viene traslata (il comportamento è uguale per tutti gli impulsi dell'immagine);
3. **Risposta a un impulso - Point Spread Function (PSF):**
 - Se un operatore è **lineare e shift invariant**, può essere descritto completamente conoscendo la sua risposta a un singolo impulso, chiamata **Point Spread Function**.
 - L'impulso è una matrice con tutti i valori a 0 ed uno con valore 1.

Esempio

Si consideri l'operazione che preso un impulso:



Kernel finiti o infiniti e complessità

Un operatore **lineare e shift-invariant** è **completamente definito da una maschera** (kernel). Viceversa, ogni maschera rappresenta un operatore lineare e invariante per traslazioni, ottenuto tramite **convoluzione** con l'input.

Questa matrice rappresenta un "impulso" che viene trasformato da un operatore, producendo una risposta specifica.



La matrice che descrive la risposta all'impulso si chiama **kernel** o **maschera di convoluzione**, perché l'operazione di filtraggio è equivalente a una **convoluzione** tra l'immagine e il kernel.

- Le dimensioni della maschera (kernel) influenzano la complessità computazionale dell'operazione di convoluzione. Più grande è il kernel, maggiore sarà il numero di operazioni necessarie per applicare il filtro all'intera immagine.
- Teoricamente il kernel potrebbe avere dimensioni infinite, nella pratica si usano solo kernel di dimensioni finite.

Filtro vs Kernel

- **Kernel:** È la matrice di valori che definisce come i pixel vengono modificati.
- **Filtro:** È l'operatore che utilizza il kernel per applicare una trasformazione all'immagine. Quindi, il filtro è il concetto più generale, mentre il kernel è la sua implementazione pratica.

La **complessità** dell'operazione di filtraggio dipende da due fattori principali:

- La **dimensione** del kernel (più grande è il kernel, maggiore è la complessità).
- La **dimensione dell'immagine** (più pixel ci sono, maggiore sarà il numero di operazioni necessarie per applicare il filtro).

Filtri "convolutivi"

I filtri *lineari e invarianti* vengono anche detti **filtri convolutivi**.

La convoluzione è comunemente applicata nell'elaborazione dei segnali e delle immagini.

Convoluzione

È un'operazione che combina due funzioni per produrne una terza, incorporando l'interazione locale tra il segnale e il filtro. È una somma pesata dei valori del segnale, dove i pesi sono determinati dal kernel del filtro.

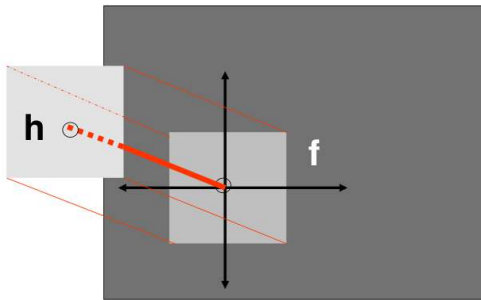
Nell'immagine processing viene usata per applicare filtri a un'immagine, eseguendo operazioni come sfocatura, rilevazione di bordi o l'estrazione di dettagli.

Utilizziamo la notazione:

$$g = f \otimes h$$

La convoluzione gode delle proprietà:

- Commutativa: $f \otimes h = h \otimes f$
- Associativa: $(f \otimes h) \otimes h_1 = f \otimes (h \otimes h_1)$



Rappresentazione della convoluzione $f \otimes h$ in cui:

- f è la funzione o **immagine principale** (rettangolo grigio)
- h è il **kernel (o filtro)**, rappresentato come il rettangolo più piccolo a sinistra con il punto rosso che rappresenta il centro del kernel

Convoluzione tra immagine e matrice (kernel)

Per convoluzione intendiamo l'applicazione di una matrice chiamata **kernel** su un'immagine rappresentata come griglia (o matrice) di pixel.

Il kernel è una matrice di dimensioni solitamente 3×3 , 5×5 , etc... che rappresenta il filtro da applicare.

- Il kernel viene sovrapposto a una porzione dell'immagine (di dimensione uguale al kernel)
- Si calcola il prodotto scalare tra i valori del kernel e i pixel dell'immagine su cui il kernel è posizionato
- Il risultato di questo prodotto scalare viene assegnato a un pixel della nuova immagine (immagine convoluta)
- Il kernel viene spostato di un pixel e il processo si ripete fino a coprire l'intera immagine

Caso finito

Se il kernel h ha dimensioni $s \times t$ la formula va riscritta come:

$$g_{m,n} = \sum_{i=-\lfloor \frac{s}{2} \rfloor}^{\lceil \frac{s}{2} \rceil - 1} \sum_{j=-\lfloor \frac{t}{2} \rfloor}^{\lceil \frac{t}{2} \rceil - 1} (h_{i,j} \cdot f_{m+i,n+j})$$

	-1	0	1
-1	a	b	c
0	d	e	f
1	g	h	i

Supponendo che gli indici del kernel siano disposti in modo da avere il punto $(0, 0)$ in posizione centrale

Spiegazione

Simboli:

- $g_{m,n}$ è il risultato della convoluzione nel punto (m, n) . Ovvero l'uscita dell'operazione di convoluzione;
- h è il kernel, o filtro, ovvero una matrice di valori $s \times t$. Definisce come combinare i valori dell'immagine in ingresso;
- f è la matrice di ingresso (segnale bidimensionale), su cui viene applicata la convoluzione;
- i e j sono gli indici di spostamento rispetto al punto centrale del kernel $(0, 0)$.

Indici della somma:

- Somma su i : itera da $-\lfloor \frac{s}{2} \rfloor$ a $\lceil \frac{s}{2} \rceil - 1$
 - Esempio: se $s = 3$, otteniamo $i = -1, 0, 1$
- Somma su j : itera da $-\lfloor \frac{t}{2} \rfloor$ a $\lceil \frac{t}{2} \rceil - 1$
 - Esempio: se $t = 3$, otteniamo $j = -1, 0, 1$

Questi intervalli di somma definiscono il range degli indici rispetto al centro

Passi dell'operazione:

1. **Centrare il kernel:** Si posiziona il kernel h in modo che il suo punto centrale (suo pixel $h_{0,0}$) sia sovrapposto al punto (m, n) della matrice di ingresso f
2. **Moltiplicazioni puntuali:** Moltiplica ogni valore del kernel $h_{i,j}$ per il corrispondente valore della matrice di ingresso $f_{m+i,n+j}$
3. **Somma:** somma tutti i prodotti ottenuti

La formula calcola così la convoluzione per tutti gli elementi i e j del kernel

Caso finito (2)

In questo caso abbiamo una diversa definizione degli indici del kernel e dell'immagine

Se il kernel h ha dimensioni $s \times t$ la formula va riscritta come:

$$g_{m,n} = \sum_{i=1, j=1}^{s, t} h_{i,j} \cdot f_{m+(i-s+\lfloor \frac{s}{2} \rfloor), n+(j-t+\lfloor \frac{t}{2} \rfloor)}$$

	1	2	3
1	a	b	c
2	d	e	f
3	g	h	i

Se gli indici del kernel sono disposti da 1 fino ad arrivare a s e t

Spiegazione

Simboli:

- $g_{m,n}$ è il risultato della convoluzione nel punto (m, n) ;
- $h_{i,j}$: valore del kernel alla posizione (i, j) , che variano da 1 a s e t rispettivamente;
- $f_{m+(i-s+\lfloor \frac{s}{2} \rfloor), n+(j-t+\lfloor \frac{t}{2} \rfloor)}$: valore della matrice in ingresso ai punti con spostamento di (i, j) dalla posizione del kernel
- s : altezza (numero di righe) del kernel
- t : larghezza (numero di colonne) del kernel

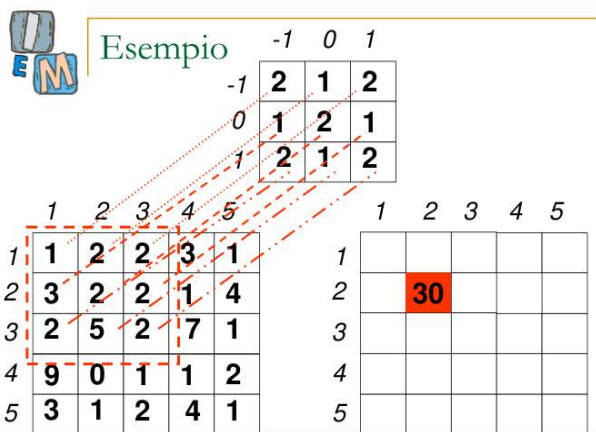
Indici della somma:

- Somma su indici i e j
- Si iterano tutti i valori da 1 a s e da 1 a t
- Il kernel non è centrato in $(0, 0)$, ma in $(\lfloor \frac{s}{2} \rfloor, \lfloor \frac{t}{2} \rfloor)$

Differenze tra le formule

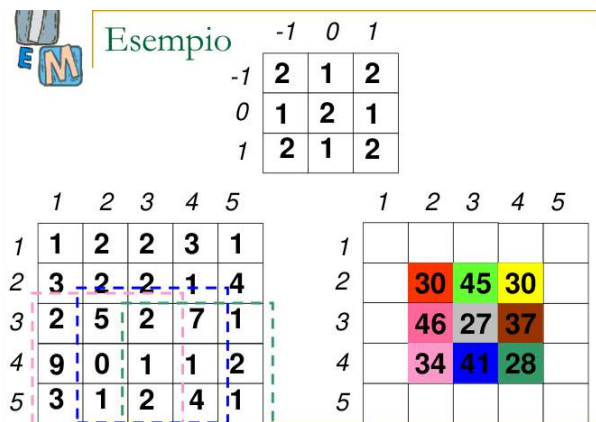
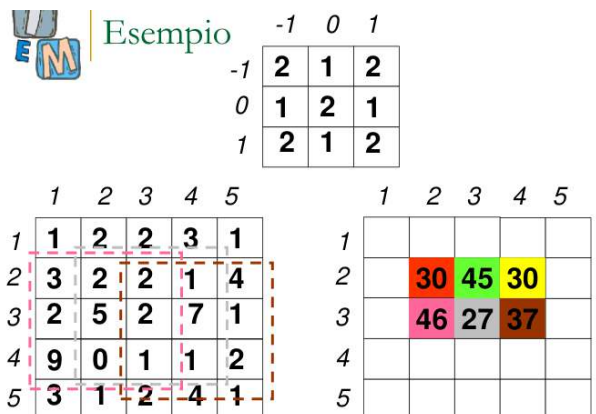
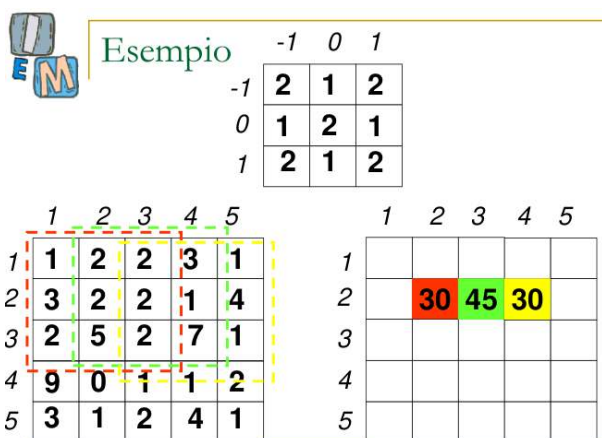
Nella prima formula gli indici del kernel sono centrati in $(0, 0)$; nella seconda formula gli indici partono da 1 fino a s e t (rispettivamente) e per essere centrato necessita di una traslazione esplicita.

Esempio



Per ottenere il valore 30 abbiamo moltiplicato ogni elemento del kernel con il corrispettivo elemento dell'immagine nella finestra 3x3.

Abbiamo quindi: $(2 + 2 + 4) + (3 + 4 + 2) + (4 + 5 + 4) = 30$, e così via per gli altri valori.



Convoluzione e filtraggio

Teorema di convoluzione e filtraggio:

L'applicazione di un filtro lineare e shift-invariant a un'immagine equivale a calcolare la convoluzione tra il kernel del filtro e l'immagine.

Linearità:

- Un filtro è lineare se l'output per una combinazione lineare di segnali è equivalente alla combinazione lineare degli output corrispondenti.
- La convoluzione soddisfa questa proprietà perché combina linearmente i valori del segnale con i pesi del kernel.

Shift-Invarianza:

- Un filtro è shift-invariante se l'output non cambia in risposta a una traslazione del segnale in ingresso.
- La convoluzione è shift-invariante perché il kernel si applica uniformemente, indipendentemente dalla posizione dell'immagine o del segnale.

Equivalenza tra convoluzione e filtro lineare-shift invariant:

- Qualsiasi **filtro lineare e shift-invariante** può essere espresso come una convoluzione con un kernel specifico, detto anche risposta impulsiva.
- Viceversa, ogni convoluzione è intrinsecamente una rappresentazione di un filtro lineare shift-invariante.

Problema dei bordi

Come effettuare la convoluzione ed il filtraggio ai bordi?

Vi sono diverse soluzioni possibili:

- Filtrare solo le zone centrali dell'immagine
- Supporre che tutto intorno all'immagine ci sia 0
- Assumere una topologia "toroidale" ("effetto pacman": sfiorando a destra si rientra a sinistra, sfiorando in basso si rientra dall'alto...)
- Aggiungere una riga all'inizio uguale alle riga precedente, una riga alla fine uguale all'ultima riga, una colonna all'inizio uguale alla colonna iniziale, e una colonna alla fine uguale alla colonna finale.

Ricapitolando:

- Un sistema **lineare e shift-invariant** (LSI) è un sistema in cui l'uscita a un ingresso è determinata da una convoluzione con una funzione fissa, detta **risposta all'impulso**.
- Questa risposta all'impulso è rappresentata da un **kernel**, che è la maschera o filtro usato per effettuare operazioni di convoluzione sull'immagine.

Operatori locali

Mediano

Filtro shift-invariant che fornisce in uscita il valore mediano dell'intorno di pixel. Si ordinano i valori dei pixel presenti nell'intorno in ordine crescente. Il **valore mediano** (quello al centro dell'insieme ordinato) sostituisce il valore del pixel centrale.

- È il miglior filtro per correggere il rumore "sale e pepe", poiché elimina i valori estremi.
- Non è lineare



Massimo e minimo

I filtri **minimo** e **massimo** sono esempi di **filtri basati su statistiche d'ordine** (_order statistics filters)

Selezionano un pixel dalla finestra di analisi in base alla sua posizione nell'ordinamento dei pixel, piuttosto che applicare una trasformazione lineare

- Entrambi non sono lineari
- Entrambi non sono applicabili per convoluzione
- Si usano per ridurre il rumore impulsivo

Massimo:

- Sostituisce il valore del pixel centrale con il valore **massimo** all'interno della finestra $m \times n$.
- Crea un effetto di **schiarimento**, usato per ridurre il **rumore pepe**



Minimo:

- Sostituisce il valore del pixel centrale con il valore **minimo** all'interno della finestra $m \times n$

- Crea un effetto di **incupimento**, usato per ridurre il **rumore sale**



N-box (Media)

Il filtro **N-box**, noto anche come filtro di **media uniforme**, è un tipo di filtro lineare utilizzato per sfocare un'immagine. Si basa su un kernel $N \times N$ i cui elementi sono tutti uguali a $\frac{1}{N^2}$

Se applicato a un'immagine di colore uniforme I , restituisce la stessa immagine (cioè $I'=I$)

Sfocatura (Blur):

- Ridistribuisce i valori dei pixel creando un effetto di sfocatura uniforme. La sfocatura è più evidente nelle direzioni orizzontale e verticale, mentre è meno pronunciata nelle direzioni diagonali
- Non preserva bene i bordi
- **Potrebbe introdurre valori** di grigio prima **non presenti**, questo per via del calcolo della media
- Può essere applicato per convoluzione
- È usato per correggere il rumore gaussiano

3-box

$\frac{1}{9} \cdot$

1	1	1
1	1	1
1	1	1

N-binomiale (filtri gaussiani)

I filtri n-binomiali sono basati su kernel i cui valori derivano dal binomio di Newton: $(x + y)^n$. La distribuzione binomiale, quando viene normalizzata e applicata a una griglia di pixel, diventa una **approssimazione discreta della distribuzione gaussiana**; per questo motivo vengono anche chiamati *filtri gaussiani*.

A differenza del filtro **N-box** (media uniforme), il filtro n-binomiale applica una **media pesata**, quindi i pixel centrali hanno un peso maggiore rispetto a quelli periferici.

Esempio:

$$(a + b)^2 = a^2 + 2ab + b^2$$

i cui coefficienti sono:

$$a^2 \rightarrow 1, \quad 2ab \rightarrow 2, \quad b^2 \rightarrow 1$$

che rappresentano i pesi utilizzati per la media pesata.

- È conservativo perché preserva meglio i dettagli dell'immagine, specialmente i bordi.
- Smussa l'immagine in tutte le direzioni
- Può essere applicato tramite convoluzione
- È lineare

3-binomiale			5-binomiale				
1/16 *			1/256 *				
1	2	1	1	4	6	4	1
2	4	2	4	16	24	16	4
1	2	1	6	24	36	24	6
			4	16	24	16	4
			1	4	6	4	1

Filtri energy preserving

Un filtro è **energy preserving** (o **conservativo**) se la **somma dei valori** nel suo **kernel** è uguale a 1. Questo garantisce che, quando il filtro viene applicato all'immagine, la **somma complessiva dei valori** (luminanza) non venga alterata.

I filtri N-box e N-binomiale, visti finora, sono conservativi, insieme ad altri filtri di smoothing.

Esistono anche filtri non energy preserving.

Noise cleaning e smoothing

I filtri N-box e N-binomiali **mediano** i pixel vicini a ciascun pixel dell'immagine, in modo che i valori "anomali" o i **picchi di rumore** vengano "smorzati". Più grande è il kernel, più saranno "uniformizzati" i valori dei pixel, riducendo il rumore, ma allo stesso tempo può portare a una maggiore sfocatura.

Per **ridurre il rumore** e ottenere un buon risultato, si cerca di trovare una **dimensione ottimale** del kernel, con la conservazione dei dettagli, quindi meno sfocatura.

Per un rumore molto diffuso è meglio applicare iterativamente lo stesso kernel piuttosto che applicare un kernel più grande, in questo modo si elimina il rumore con meno sfocatura.

Il Rumore

Ci sono due tipi di rumore:

- **Rumore impulsivo** ("sale e pepe"): caratterizzato dalla frazione dell'immagine modificata in percentuale %
- **Rumore gaussiano** bianco: caratterizzato dalla media e dalla varianza

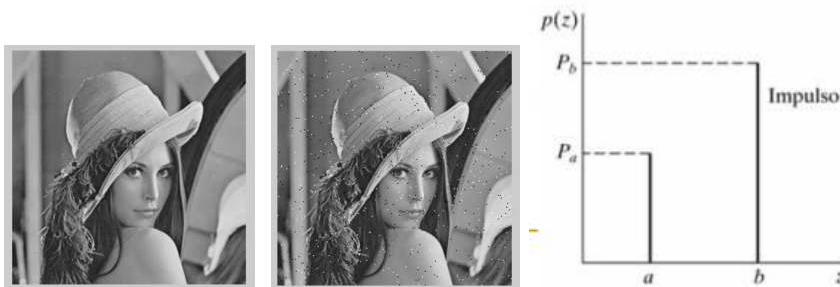
Rumore impulsivo (sale e pepe)

È un tipo di disturbo che si manifesta sotto forma di pixel "estremi" che assumono valori massimi o minimi, creando un aspetto simile a granelli di sale e pepe. Colpisce i pixel in posizioni casuali dell'immagine. La gravità del rumore dipende dal numero di pixel alterati rispetto al totale (ad esempio, 10%, 20%, ecc.).

- Per questo tipo di rumore il filtro mediano funziona meglio.

$$p(z) = \begin{cases} P_a & \text{per } z = a \\ P_b & \text{per } z = b \\ 0 & \text{altrimenti} \end{cases}$$

- z rappresenta il valore d'intensità del pixel
- a e b sono i valori saturi, cioè massimo e minimo dell'immagine (solitamente $a = 0$ e $b = 255$ per un'immagine in scala di grigi 8bit)
- P_a e P_b indicano la probabilità che il pixel assuma i valori a o b



Il grafico rappresenta dove e con quale intensità il rumore impulsivo altera i pixel.

Rumore gaussiano

Il **rumore gaussiano**, detto anche **rumore bianco gaussiano**, è un tipo di disturbo che si distribuisce su un'immagine seguendo la **distribuzione normale** (o gaussiana). Questo rumore è caratterizzato da una variazione casuale dei pixel, concentrata attorno a un valore medio, con deviazioni più probabili vicino alla media e meno probabili man mano che ci si allontana. Tutti i pixel dell'immagine possono essere influenzati, creando un aspetto "granuloso".

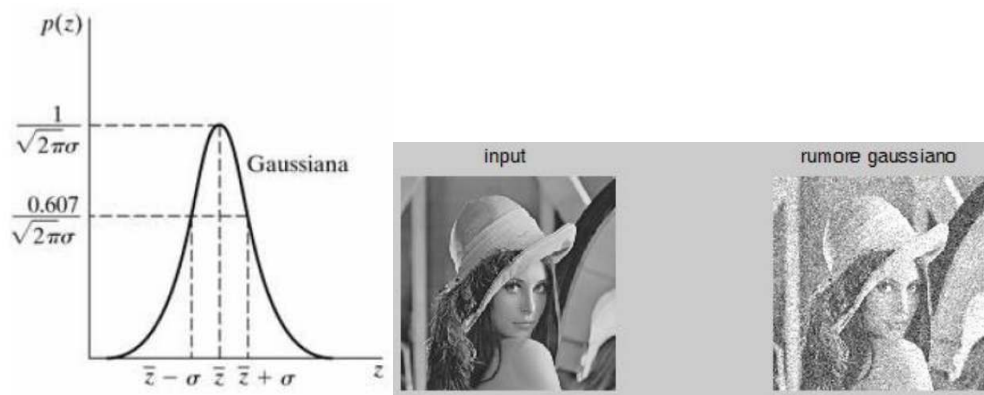
Il filtro media funziona bene per rimuovere il rumore gaussiano

$$p(z) = \frac{1}{\sqrt{2\pi} \cdot \sigma} e^{-(z-\bar{z})^2/2\sigma^2}$$

- z rappresenta l'intensità
- $\bar{z}$ rappresenta il valore medio di z Definisce il valore attorno al quale i pixel disturbati si distribuiscono
- σ è la sua deviazione standard (quanto il valore si allontana dalla media)
- σ^2 è detta varianza di z

Circa il 70% dei valori di z cade in $[\bar{z} - \sigma, \bar{z} + \sigma]$

Circa il 95% dei valori di z cade in $[\bar{z} - 2\sigma, \bar{z} + 2\sigma]$



Media vs mediano

I filtri mediani sono migliori dei filtri di media perché il filtro di media può introdurre livelli di grigio non presenti nell'immagine e sfocare indiscriminatamente le alte frequenze spaziali. Al contrario, il filtro mediano conserva i bordi e rimuove solo i picchi di rumore, preservando meglio i dettagli.

Altri filtri denoise

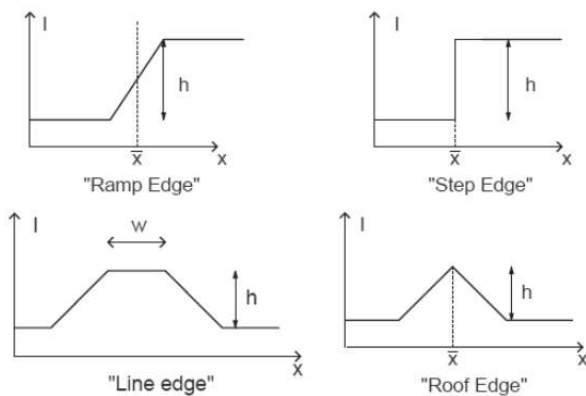
Altri filtri non lineari molto importanti:

- **Outlier:** il valore del pixel centrale viene confrontato con il valore della media dei suoi 8 vicini. Se il valore assoluto della differenza è maggiore di una certa soglia, allora il punto viene sostituito dal valore medio, altrimenti non viene modificato.
- **Olimpico:** scarta i valori estremi (massimo e minimo) all'interno di una finestra, quindi calcola la media dei valori rimanenti. Non è adatto a rimuovere rumore gaussiano, perché non produce valori estremi evidenti.

Edge detectors

Gli operatori locali aiutano ad estrarre i contorni da un'immagine. I contorni sono delle discontinuità locali della luminanza (intensità luminosa). Gli **edge detectors** sono algoritmi specifici progettati per rilevare i contorni. Il loro obiettivo è **preservare solo le variazioni significative di luminanza** e scartare le informazioni che non contribuiscono alla definizione dei bordi.

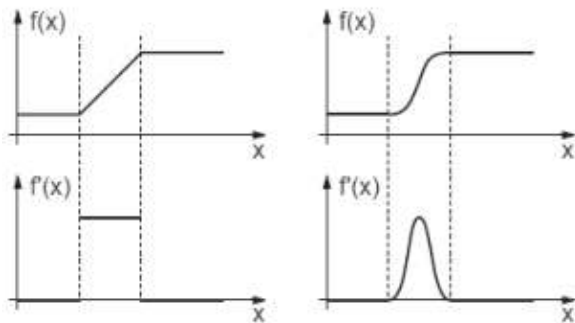
Esempi di lato in 1D



Edge detector basati sulla derivata prima

Se di un segnale monodimensionale calcoliamo la **derivata prima**, lungo le righe o colonne di un'immagine, possiamo individuare i bordi come i punti di massimo o minimo della derivata.

I filtri dovranno calcolare la derivata prima in direzione x , in direzione y e poi combinarle insieme.



- **Prewitt** è più semplice e con pesi uniformi. È meno sensibile ai dettagli rispetto a Sobel.
- **Sobel** ha pesi asimmetrici, con maggiore enfasi sui pixel centrali, ed è più efficace nel rilevare bordi netti e ridurre il rumore.

Entrambi si possono applicare per convoluzione e sono usati per rilevare bordi orizzontali e verticali, ma Sobel è generalmente preferito per immagini più complesse o rumorose.

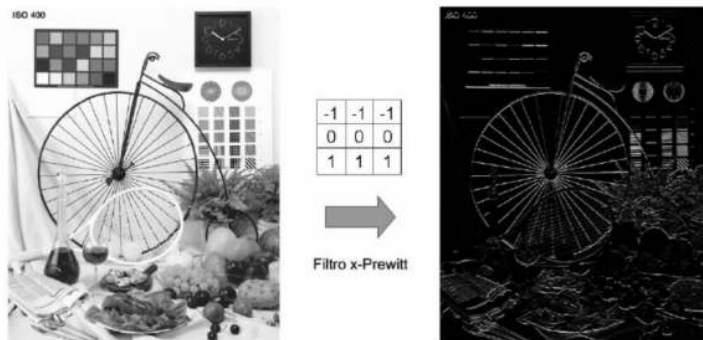
Kernel notevoli per i lati verticali

Rileva i bordi **verticali** (variazioni orizzontali della luminanza).

Sobel x



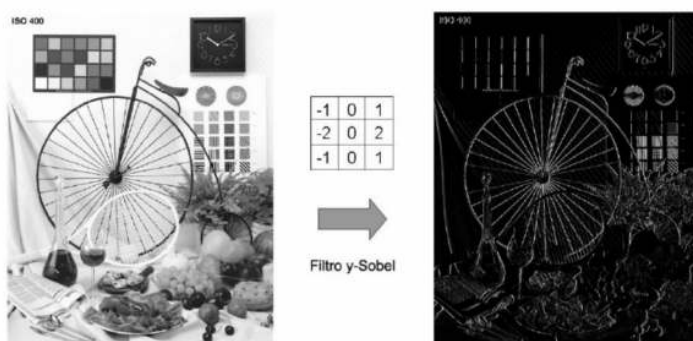
Prewitt x



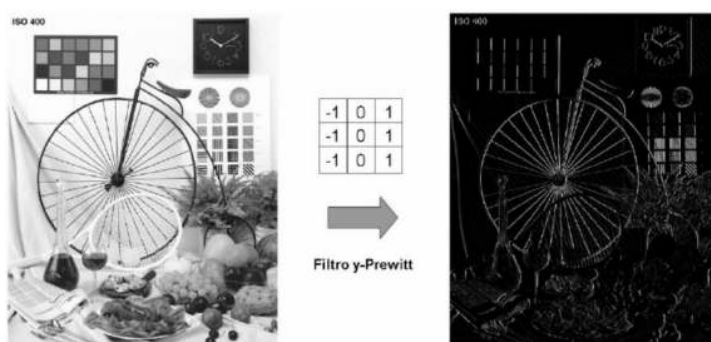
Kernel notevoli per i lati orizzontali

Rilevano i bordi orizzontali (variazioni verticali della luminanza).

Sobel y



Prewitt y



Magnitudo

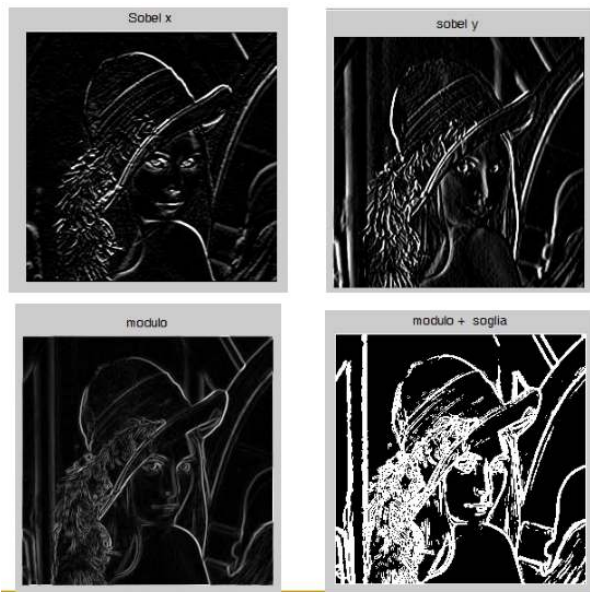
La **magnitudo** calcolata con i filtri Sobel o Prewitt rappresenta l'intensità complessiva della variazione di luminanza in un'immagine, combinando i contributi delle variazioni in direzione orizzontale e verticale.

- *sobel_x*: rileva i bordi **verticali**
- *sobel_y*: rileva i bordi **orizzontali**

$$magnitudo = \sqrt{sobel_x^2 + sobel_y^2}$$

La formula $\sqrt{sobel_x^2 + sobel_y^2}$ combina le variazioni orizzontali e verticali per ottenere un valore che rappresenta l'intensità del bordo, indipendentemente dalla sua direzione.

Si applica dunque una **soglia**: se la magnitudo è maggiore del valore soglia allora il pixel viene considerato facente parte di un bordo; in questo modo si crea una matrice **binaria** che indica quali pixel fanno parte dei bordi.

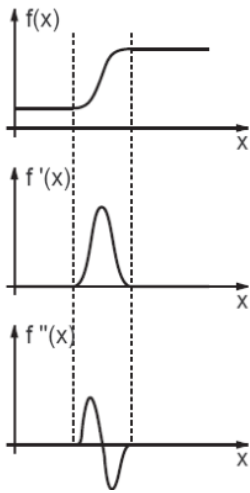


Migliori risultati

I migliori risultati nel rilevamento dei bordi si ottengono con **algoritmi non lineari** e tecniche avanzate come l'**algoritmo di Canny** e gli **algoritmi fuzzy**.

Edge detector basati sulla derivata seconda

La derivata seconda di un segnale $f(x)$, denotata come $f''(x)$, misura la **curvatura** del segnale. In un punto dove la derivata seconda è zero, significa che il segnale cambia la sua curvatura, indicando un possibile **cambio nella direzione** o una **discontinuità** (ad esempio, un bordo).



Kernel notevoli: Laplaciano

Filtro più diffuso per il calcolo della derivata seconda. È particolarmente utile per rilevare bordi e contorni.

Laplaciano è l'unico operatore che, quando applicato a un'immagine di colore uniforme, produce un risultato in cui tutti i valori sono uguali a **zero**. (Tutte le derivate seconde sono zero, poiché la curvatura dell'immagine è nulla (non c'è cambiamento tra i pixel))

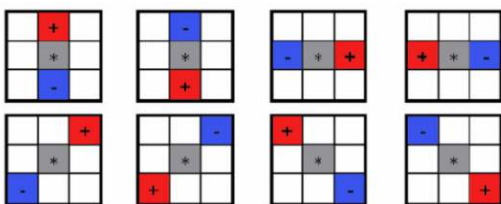
- Può essere applicato per convoluzione
- È lineare
- Non attenua rumori

$$\text{Laplaciano} = \begin{bmatrix} -1 & 0 & -1 \\ 0 & 4 & 0 \\ -1 & 0 & -1 \end{bmatrix}$$

Zero-crossing

Dopo aver applicato il Laplaciano, il **Zero-crossing** aiuta a identificare i **contorni** nell'immagine.

- La condizione di Zero-crossing implica che in un intorno di un pixel, ci deve essere un valore positivo e uno negativo, indicando una discontinuità nei valori di intensità, che rappresenta un bordo.



Filtri di sharpening

I **filtri di sharpening** migliorano la nitidezza di un'immagine aumentando il contrasto locale, operazione opposta allo sfocamento. Questo effetto si ottiene con una maschera derivata dal Laplaciano, che rinforza i bordi ma amplifica anche il rumore presente nell'immagine.

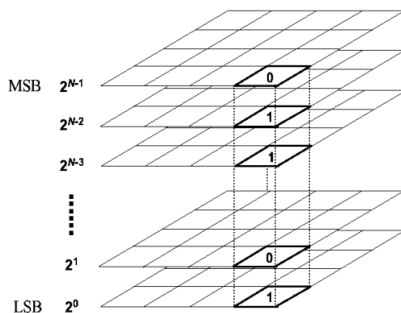
- È lineare
- Applicato per convoluzione

Filtro di Enhancing

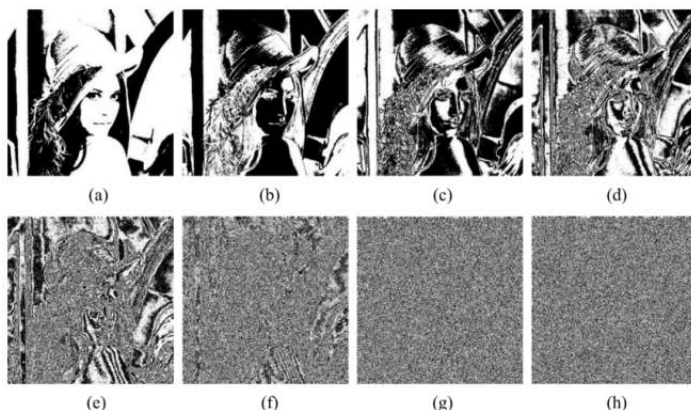


BitPlane

Un'immagine con profondità colore di **N bit** può essere suddivisa in **N piani di bit (bit-planes)**, ognuno rappresentabile come un'immagine binaria. I piani sono ordinati dal **bit più significativo (MSB)**, che contribuisce maggiormente al valore del pixel, al **bit meno significativo (LSB)**, che ha il minor peso nella rappresentazione del colore.



Il bit plane di un'immagine digitale a N bit, è un'insieme di N immagini binarie (piani), in cui l'immagine i-esima contiene i valori dell' i-esimo bit della codifica scelta.



(a) : most significant bit \ (b) : least significant bit

Con la **codifica in binario puro**, i **bit plane** più significativi (come il 7°) contengono informazioni sulla **struttura principale** dell'immagine, mentre quelli meno significativi rappresentano **dettagli meno significativi**, contengono infatti rumore gaussiano o errori di acquisizione dell'immagine.

Usi bit-planes binario puro:

Questo genere di scomposizione è molto utile per eliminare tutti i valori compresi in un certo range.

- Se vogliamo eliminare tutti i grigi tra 32 e 63, sappiamo che questi valori hanno **il 5° bit sempre a 1**:
 - $32_{10} = 00100000_2$
 - $63_{10} = 00111111_2$
- Ponendo a **0 il 5° bit-plane**, eliminiamo questi valori dall'immagine.
Se vogliamo una versione meno dettagliata di un'immagine, possiamo **ignorare i bit meno significativi** e usare solo i più significativi, ponendo a 0 i bit-plane meno importanti.

Bit-Planes - Problema

Un problema relativo alla rappresentazione in binario puro, è la seguente: due valori di grigio molto simili possono avere codifiche estremamente diverse: 127 (01111111) e 128 (10000000). Questa brusca transizione tra 127 e 128 si ripercuote su tutti i piani, creando dei bit-planes non molto coerenti tra loro. A fini di compressione, serve una codifica che associa a valori simili una codifica simile.

Codifica Gray

La codifica Gray risolve il problema delle **transizioni brusche** tra valori simili nella codifica binaria pura, rendendo i **bit-planes** più coerenti. La codifica Gray si ottiene partendo da una rappresentazione binaria tradizionale e applicando una trasformazione **basata sull'operazione XOR**. Riduce drasticamente le differenze tra numeri successivi, migliorando la coerenza tra i piani di bit e facilitando la compressione.

Il calcolo avviene in questo modo:

4. **Primo bit (più significativo)**: rimane lo stesso tra binario puro e Gray:

$$g_{m-1} = a_{m-1}$$

5. **Bit successivi**: ciascun bit della codifica Gray si ottiene calcolando l'XOR tra il bit corrente (a_i) e il bit successivo (a_{i+1}) nella rappresentazione binaria pura.

$$g_i = a_i \oplus a_{i+1} \text{ per } 0 \leq i \leq m-2$$

Quindi, 127 e 128 diventeranno:

$$127 = 01111111 \rightarrow 01000000$$

$$128 = 10000000 \rightarrow 11000000$$

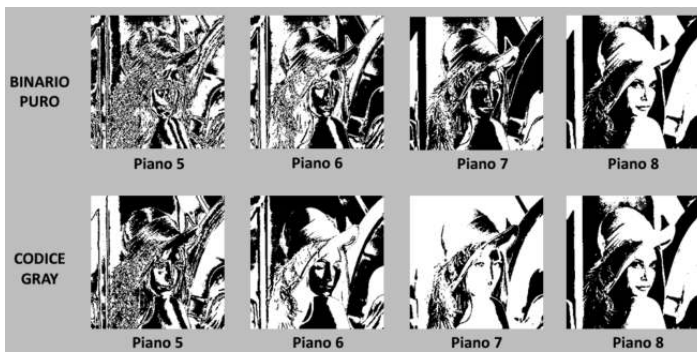


Image Enhancement nel Dominio delle Frequenze

Serie di Fourier: una **funzione periodica** può essere espressa come somma di seni e/o coseni di differenti frequenze e ampiezze. Questa rappresentazione permette di decomporre una funzione complessa in componenti semplici.

Trasformata di Fourier: anche una **funzione non periodica**, può essere espressa come integrale di seni e/o coseni, moltiplicati per opportune funzioni-peso. Questa trasformata descrive come le frequenze sono distribuite nel segnale originale, permettendo di lavorare con funzioni non periodiche in un "dominio delle frequenze".

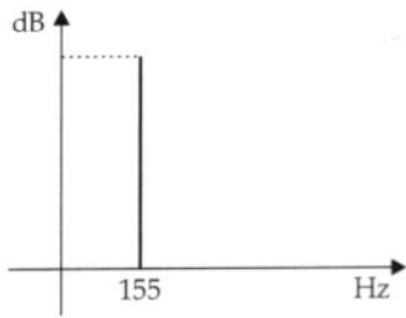
Sia la Serie di Fourier che la Trasformata di Fourier condividono una caratteristica cruciale: **la possibilità di ricostruire una funzione originale senza perdita di informazione.**

Dominio spaziale vs Dominio delle frequenze

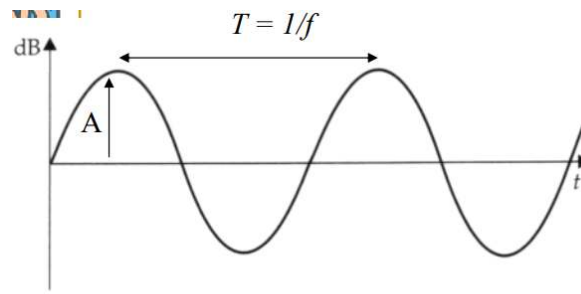
Dominio Spaziale: L'immagine originale è rappresentata come una matrice di pixel, dove ogni pixel ha un valore che corrisponde alla sua intensità luminosa.

Dominio delle Frequenze: La trasformata di Fourier scompone l'immagine nelle sue componenti sinusoidali di base. Le basse frequenze corrispondono a variazioni graduali nell'intensità (aree uniformi, grandi oggetti), mentre le alte frequenze corrispondono a variazioni rapide (bordi, dettagli fini)

- Ampiezza (A) espressa in decibel dB;
- Periodo (T) espresso in secondi;
- Frequenza (f) numero di cicli (onde) al secondo; si misura in Hertz Hz



Dominio della frequenza



Dominio del tempo

Formule della trasformata discreta di Fourier

Lavorando con segnali digitalizzati, e quindi discretizzati, andremo a lavorare in termini di sommatorie, avvalendoci così della trasformata discreta di Fourier.

Trasformata della funzione $f(x,y)$:

$$F(u, v) = \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-i2\pi(\frac{u}{M}x + \frac{v}{N}y)}$$

Antitrasformata della funzione $F(u,v)$:

L'antitrasformata, che riporta l'immagine dal dominio delle frequenze al dominio spaziale, è data da:

$$f(x, y) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} F(u, v) e^{i2\pi(\frac{u}{M}x + \frac{v}{N}y)}$$

per $x = 0, 1, \dots, M-1$ e $y = 0, 1, \dots, N-1$

- (u,v) : è la rappresentazione nel dominio delle frequenze.
- $f(x,y)$: è l'immagine ricostruita nel dominio spaziale.
- M e N : Sono le dimensioni dell'immagine. Determinano il numero di campioni presi nel dominio spaziale.

Secondo la **formula di Eulero**, relazione fondamentale tra le funzioni esponenziali complesse e le funzioni trigonometriche:

$$e^{ix} = \cos(x) + i \sin(x)$$

Per l'angolo negativo, abbiamo:

$$e^{-ix} = \cos(x) - i \sin(x)$$

Dove:

- e è la base dei logaritmi naturali.
- i è l'unità immaginaria ($i^2 = -1$).
- x è un numero reale (può rappresentare un angolo in radianti).

Formule della trasformata:

Spettro

Lo **spettro** della trasformata rappresenta l'**intensità delle frequenze** in un segnale o immagine.

- Frequenze basse: Variazioni lente (es. aree uniformi, sfumature).
- Frequenze alte: Variazioni rapide (es. bordi, dettagli).

- Si calcola dalla trasformata discreta di Fourier (**DFT**), che produce coefficienti complessi $F(u,v)$ con una parte reale $R(u,v)$ e una immaginaria $I(u,v)$.

Formula dello spettro:

$$|F(u, v)| = \sqrt{R^2(u, v) + I^2(u, v)}$$

Potenza spettrale

La potenza spettrale mostra **quanta energia è contenuta in una determinata frequenza**.

Frequenze basse: Energia associata a variazioni lente (gradienti, aree uniformi)

Frequenze alte: Energia associata a variazioni rapide (bordi, dettagli)

Formula per la potenza spettrale:

$$P(u, v) = |F(u, v)|^2 = R^2(u, v) + I^2(u, v)$$

Lo spettro e la potenza spettrale sono una il quadrato dell'altra!

- Esercizio: Quanto vale la potenza spettrale del coefficiente di Fourier $3 + 4i$?
 $R(u, v) = 3$ $I(u, v) = 4$
 $PotenzaSpettrale = 3^2 + 4^2 = 9 + 16 = 25$
- Esercizio: La potenza spettrale di un coefficiente ottenuto tramite DFT vale 18. Da quale coppia (parte reale, parte immaginaria) deriva tale potenza?
 $PotenzaSpettrale = R^2(u, v) + I^2(u, v)$
 $18 = 3^2 + 3^2$ quindi è dato dalla coppia (3,3).

Angolo di fase

L'**angolo di fase** indica la **posizione iniziale** di un'onda sinusoidale associata a una frequenza (u,v) .

È importante per applicazioni come il **riconoscimento di forme o bordi**, dove la struttura dell'immagine dipende dalla fase.

Formula per l'angolo di fase:

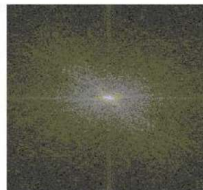
$$\phi(u, v) = \tan^{-1} \left(\frac{I(u, v)}{R(u, v)} \right)$$

Range dinamico

Quando si visualizza lo spettro di Fourier di un'immagine, la sua **gamma dinamica** può essere molto ampia, rendendo visibili solo le frequenze più luminose. Per visualizzare lo

spettro senza anomalie, si usa una **compressione logaritmica**, trasformando lo spettro con la formula:

$$D(u, v) = c \log(1 + |F(u, v)|)$$



Vantaggi della trasformata di Fourier

- **Soppressione di frequenze indesiderate:** consente di filtrare rumore o componenti non desiderate.
- Utilizzata nei formati di compressione come JPEG, MPEG, DivX e MP3 per ridurre lo spazio occupato dai dati, limitando la perdita di qualità.
- **Rigenerazione di segnali degradati**

Sintesi delle trasformazioni

Trasformazione diretta:

- Il segnale $f(x)$ viene scomposto nelle sue **componenti elementari**, che sono vettori di base (onde sinusoidali).
- I **coefficienti della trasformata** indicano **quanto ogni componente di base contribuisce al segnale**.

Trasformazione inversa:

- Il segnale viene ricostruito come somma pesata delle sue componenti di base.
- I pesi sono determinati dai **coefficienti della trasformata**, che rappresentano la correlazione tra il segnale e i vettori di base.

Proprietà della trasformata di Fourier Bidimensionale (DFT 2-D)

Separabilità

La trasformata di Fourier discreta può essere espressa in forma separabile.

La **proprietà di separabilità** della **DFT 2-D** permette di calcolare un'immagine bidimensionale $f(x,y)$ in modo **separabile**, applicando due volte la **DFT 1-D**:

- **Prima riga per riga** (trasformata sulle righe).
- **Poi colonna per colonna** (trasformata sulle colonne).

Invece di calcolare direttamente la **DFT 2-D** (che sarebbe computazionalmente costosa), si eseguono due **DFT 1-D**, riducendo il tempo di calcolo.

Formula generale:

$$F(u, v) = \frac{1}{M} \sum_{x=0}^{M-1} k(x, v) e^{-i2\pi ux/M}$$

Dove:

$$k(x, v) = \frac{1}{N} \sum_{y=0}^{N-1} f(x, y) e^{-i2\pi vy/N}$$

Basse e alte frequenze

All'interno delle immagini, le frequenze sono legate alla velocità di variazione dei toni. Per questo motivo, le basse frequenze sono quelle legate al tono generale dell'immagine, mentre le alte frequenze contengono informazioni relative a variazioni molto veloci, come nel caso del rumore.

Traslazione (Shift dell'origine della DFT 2-D)

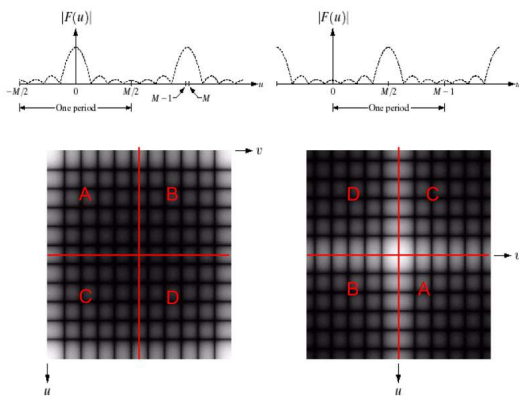
Nel caso della **trasformata di Fourier bidimensionale**, è utile applicare uno **shift dell'origine** al **centro** della matrice delle frequenze:

Di default, la **DFT 2-D** posiziona l'origine delle frequenze nel punto (0,0). Tuttavia, le frequenze più basse (quelle più rilevanti per l'analisi delle immagini) sono più utili se vengono spostate **al centro** dello spettro.

Per ottenere questo risultato, possiamo applicare uno **shift delle frequenze**, spostando l'origine da (0,0) a $(\frac{M}{2}, \frac{N}{2})$, dove:

- **M** e **N** sono le dimensioni dell'immagine.
- **Basse frequenze** (dettagli grossolani) vengono portate al **centro**.
- **Alte frequenze** (dettagli fini e rumore) vengono spostate verso i **bordi**.

Il vantaggio principale di questo shift è che **non modifica la magnitudo** della trasformata, ma rende più intuitiva la visualizzazione dello spettro, migliorando l'analisi delle frequenze. Questo approccio è utile in applicazioni come la **compressione** o il **filtraggio** delle immagini.



Valore Medio

Il valore della **trasformata di Fourier** nell'origine, cioè nel punto $F(0,0)$, rappresenta la **media** dei valori di grigio dell'immagine $f(x,y)$. Questo valore è noto come **componente continua (DC)** e descrive la componente a bassa frequenza dell'immagine.

$$F(0,0) = \frac{1}{N \times M} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y)$$

Fast Fourier Transform (FFT)

Nella sua forma classica, **implementare la trasformata di Fourier** richiederebbe un numero di operazioni proporzionale a N^2 , dove N è la dimensione del segnale o dell'immagine. Questo è dovuto a:

- N moltiplicazioni complesse per ogni valore di u e v ,
- $N-1$ addizioni per ciascuno degli N valori di u .

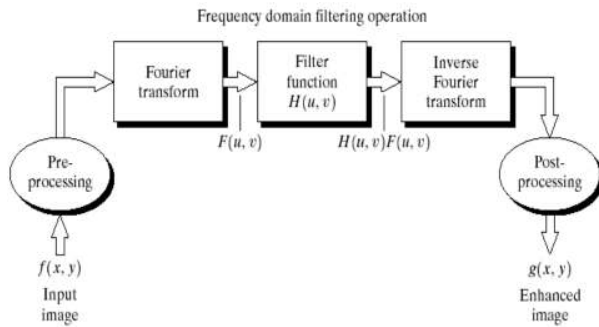
Tuttavia, utilizzando un algoritmo di **decomposizione** come la **Fast Fourier Transform (FFT)**, è possibile ridurre la complessità computazionale a $N \log_2 N$. La FFT sfrutta l'idea di **dividere il problema in sottoproblemi più piccoli** e quindi **unire i risultati** in modo efficiente. Questo consente di calcolare la trasformata di Fourier in tempi molto più rapidi.

$$F(u) = \sum_{x=0}^{N-1} f(x) \cdot \exp\left(-\frac{2\pi i u x}{N}\right)$$

Filtraggio nel Dominio della Frequenza

Filtro a sfasamento zero (Zero Phase Shift): un filtro è detto **zero phase shift** quando non introduce alcuno sfasamento nelle componenti di frequenza dell'immagine. In pratica, la funzione $H(u,v)$ moltiplica ciascuna componente della trasformata di Fourier (sia la parte reale che quella immaginaria) senza introdurre modifiche alla fase del segnale. Così facendo, il filtro ha effetto solo sull'ampiezza delle frequenze, lasciando invariata la fase e

quindi evitando lo sfasamento.



Teorema della convoluzione e Trasformata di Fourier

La convoluzione nel dominio spaziale è computazionalmente costosa, ma può essere semplificata trasferendo il problema nel dominio delle frequenze, dove la convoluzione diventa una moltiplicazione grazie alla proprietà del prodotto della trasformata di Fourier.

Enunciato del teorema

«La convoluzione di due segnali f e h nel dominio spaziale è equivalente al prodotto delle loro trasformate nel dominio di Fourier»

$$F(f \otimes h) = F(f) \cdot F(h)$$

Con f immagine, h kernel, $\otimes$ convoluzione, F trasformata di Fourier. Applicare questo metodo nel dominio di frequenze ha complessità $O(n \log n)$.

Allo stesso modo, la convoluzione nel dominio spaziale equivale a calcolare l'antitrasformata del prodotto delle trasformate di Fourier:

$$f \otimes h = F^{-1}(F(f) \cdot F(h))$$

Come ottenere un filtro a partire da una maschera spaziale:

Sfruttando il dominio delle frequenze, possiamo ottenere un filtro convolutivo H a partire da una maschera spaziale:

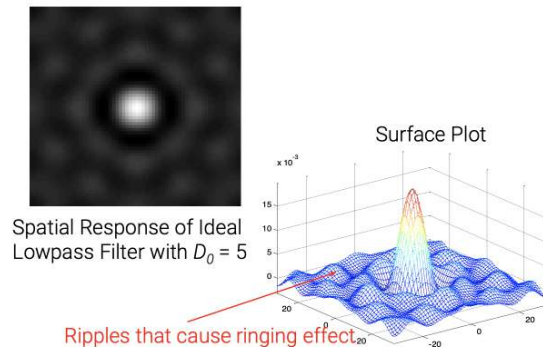
1. Creare un filtro H delle stesse dimensioni dell'immagine I .
2. Inserire la maschera spaziale in alto a sinistra di H , settando il resto a zero.
3. Eseguire uno shift di H per centrare la maschera nel dominio delle frequenze.
4. Calcolare la trasformata di Fourier di H .

Filtri ideali

Prima di parlare dei diversi tipi di filtri, introduciamo i filtri ideali. Il termine "ideale" indica che tutte le frequenze all'interno di un cerchio di raggio D_0 sono trasferite senza attenuazione,

mentre tutte quelle esterne sono, invece, annullate. Creano quindi un taglio netto tra le frequenze mantenute e quelle rimosse. Lasciano intatte solo le frequenze inferiori o superiori alla frequenza di taglio e annullano tutte le altre.

Tali filtri causano un forte fenomeno di sfocatura ad anello detto **ringing**, legato alla funzione $h(x,y)$ dell'anti-trasformata. Gli anelli generati hanno un raggio inversamente proporzionale alla frequenza di taglio.



Filtri di smoothing o passa-basso (low-pass)

I filtri di smoothing sono dei filtri low-pass (passa-basso) che eliminano il rumore. Ciò corrisponde nel dominio delle frequenze ad attenuare le componenti ad alta frequenza.

I principali filtri passa-basso sono:

Filtro ideale passa-basso: è un tipo di filtro che elimina completamente tutte le componenti di frequenza che si trovano a una distanza maggiore di D_0 dall'origine, dove D_0 è la **frequenza di taglio**. Il filtro lascia inalterate solo le frequenze inferiori a D_0 e annulla tutte le altre. La risposta di questo filtro è rappresentata da un **disco circolare** centrato nell'origine. Tutte le frequenze all'interno del disco vengono mantenute. L'uso di questo filtro può causare effetto di **ringing** (o effetto di Gibbs), che si manifesta come oscillazioni.

$D(u, v)$ è la distanza dall'origine ed è data da:

$$D(u, v) = \sqrt{\left(u - \frac{P}{2}\right)^2 + \left(v - \frac{Q}{2}\right)^2}$$

$$H(u, v) = \begin{cases} 1, & \text{se } D(u, v) \leq D_0 \\ 0, & \text{se } D(u, v) > D_0 \end{cases}$$

- $H(u, v)$: Risposta del filtro
- u, v : Coordinate di un punto nel dominio della frequenza
- P, Q : Dimensioni dell'immagine in input.

- $\frac{P}{2}, \frac{Q}{2}$: Coordinate del centro dell'immagine

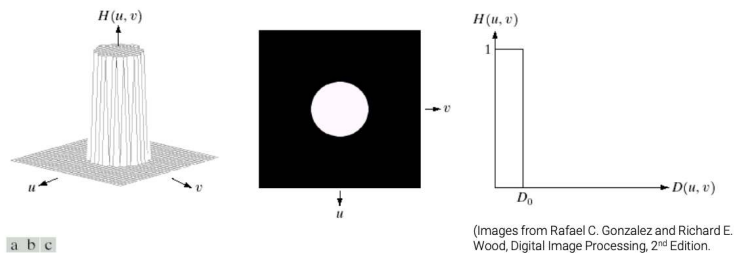


FIGURE 4.10 (a) Perspective plot of an ideal lowpass filter transfer function. (b) Filter displayed as an image. (c) Filter radial cross section.

- Esercizio: Sia H un filtro passa basso ideale di dimensione 20×20 e frequenza di taglio 4. Quanto vale $H(14, 13)$?

$$u = 14 \quad v = 13$$

$$\text{Il centro si trova in: } \left(\frac{20}{2}, \frac{20}{2}\right) = (10, 10)$$

$$D(14, 13) = \sqrt{(14 - 10)^2 + (13 - 10)^2} = \sqrt{4^2 + 3^2} = \sqrt{16 + 9} = \sqrt{25} = 5$$

La frequenza di taglio è $D_0 = 4$

Poiché $D(14, 13) = 5 > D_0$

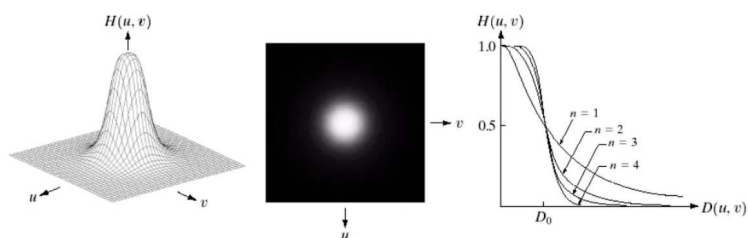
5 è **maggiore** di $D_0 = 4$, quindi il filtro **blocca** questa frequenza

$$H(14, 13) = 0$$

Filtri di Butterworth passa-basso: Introducono un'attenuazione graduale delle frequenze superiori a D_0 preservando le frequenze inferiori. **Regolando l'ordine n** , si controlla la transizione del filtro: **aumentando l'ordine n , il filtro si avvicina sempre di più a un filtro ideale**, assumendo gli stessi difetti.

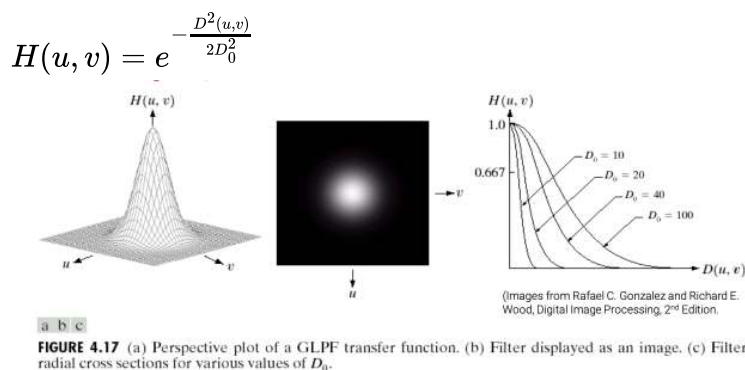
La funzione di trasferimento di tale filtro di **ordine n** e frequenza di taglio D_0 è:

$$H(u, v) = \frac{1}{1 + \left(\frac{D(u, v)}{D_0}\right)^{2n}}$$



Filtri gaussiani passa-basso: progettati per **attenuare le alte frequenze** (dettagli fini, bordi) e **preservare le basse frequenze** (aree uniformi o sfocate). L'effetto è simile a quello di un **filtro di sfocatura**, poiché riduce i dettagli e le variazioni rapide di intensità. Sono simili ai **filtri di Butterworth di piccolo ordine**, ma con una **realizzazione più semplice**. Il principale vantaggio è avere come trasformata di Fourier ancora una gaussiana.

La funzione di trasferimento di tale filtro di frequenza di taglio D_0 è:



Filtri di sharpening o passa-alto (high-pass)

Poiché i bordi e gli altri rapidi cambiamenti nei livelli di grigio sono legati alle componenti ad alta frequenza, è possibile attuare lo sharpening dell'immagine attraverso i filtri high-pass (passa-alto) che eliminano le componenti a bassa frequenza.

I principali filtri passa-alto sono:

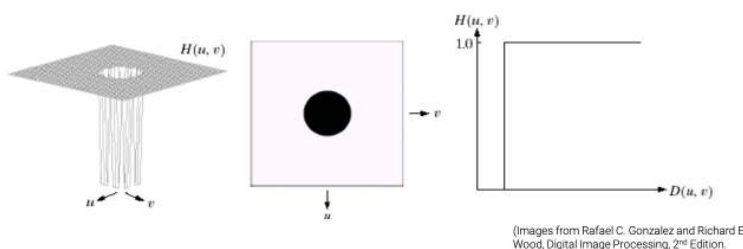
Filtro ideale passa-alto: elimina completamente tutte le componenti di frequenza che, nel dominio delle frequenze, si trovano a una distanza minore o uguale a D_0 dall'origine (D_0 è la **frequenza di taglio**). Il filtro lascia inalterate solo le frequenze superiori a D_0 e annulla tutte le altre. La risposta di questo filtro è rappresentata da un **disco circolare** complementare, dove il centro (frequenze basse) viene bloccato e solo le frequenze esterne al disco (frequenze alte) vengono mantenute. L'uso di questo filtro introduce un **effetto di ringing** (o effetto di Gibbs).

Dove $D(u, v)$ è la distanza dall'origine ed è data da:

$$D(u, v) = \sqrt{\left(u - \frac{P}{2}\right)^2 + \left(v - \frac{Q}{2}\right)^2}$$

$$H(u, v) = \begin{cases} 0, & \text{se } D(u, v) \leq D_0 \\ 1, & \text{se } D(u, v) > D_0 \end{cases}$$

- P, Q : Dimensioni dell'immagine in input.
- $\frac{P}{2}, \frac{Q}{2}$: Coordinate del centro dell'immagine in dominio di Fourier

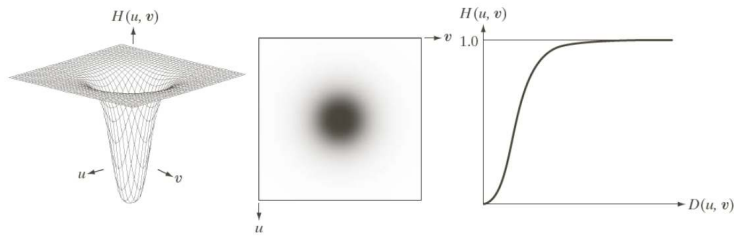


Filtri di Butterworth passa-alto: Introducono un'attenuazione graduale delle frequenze inferiori a D_0 preservando le frequenze superiori. **Regolando l'ordine n** , si controlla la transizione del filtro: **umentando l'ordine n , il filtro si avvicina sempre di più a un filtro ideale**, assumendo gli stessi difetti.

La funzione di trasferimento di un filtro passa-alto di Butterworth di ordine n e frequenza

di taglio D_0 è:

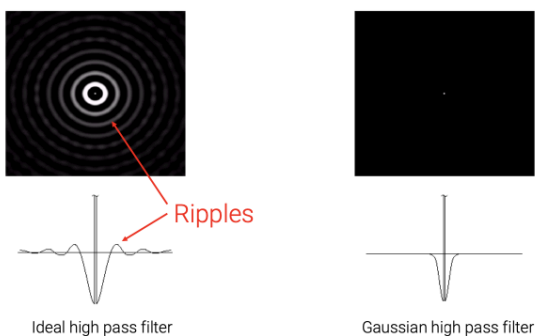
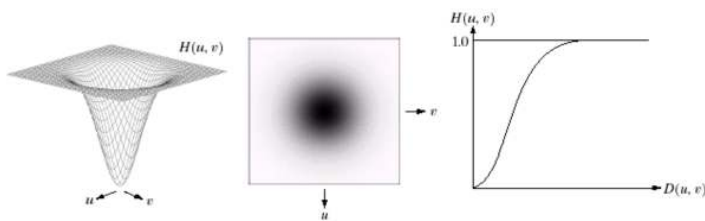
$$H(u, v) = \frac{1}{1 + \left(\frac{D(u, v)}{D_0}\right)^{2n}}$$



Filtri gaussiani passa-alto: progettati per **enfatizzare le alte frequenze** (dettagli fini, bordi) e **attenuare le basse frequenze** (aree uniformi o sfocate). L'effetto è simile a quello di un **edge detector**, poiché i bordi sono associati a transizioni rapide di intensità (alte frequenze). Sono simili ai **filtri di Butterworth di piccolo ordine**, ma con una **realizzazione più semplice**. Il principale vantaggio è avere come trasformata di Fourier ancora una gaussiana.

La funzione di trasferimento di un filtro passa-alto gaussiano di frequenza di taglio D_0 è:

$$H(u, v) = 1 - e^{-\frac{D^2(u, v)}{2D_0^2}}$$



Filtri Band Reject

I **filtri band-reject** attenuano una **specifica gamma di frequenze** (un "intervallo di banda") mantenendo inalterate le frequenze al di fuori di tale intervallo. In altre parole, un filtro band-reject elimina o riduce l'intensità delle frequenze comprese in un determinato intervallo, mentre lascia passare sia le frequenze basse che quelle alte. Sono definiti da una frequenza centrale D_0 e da una larghezza di banda W .

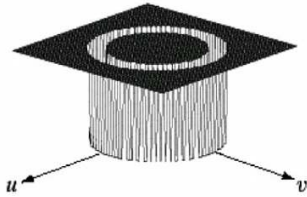
Filtri ideali band-reject: eliminano totalmente tutte le componenti di frequenza che nel rettangolo delle frequenze non entrano nel range desiderato di centro D_0 , la cosiddetta frequenza di taglio. La transizione è **istantanea**, senza graduale attenuazione, il che causa effetto ringing. Tale filtro ha una risposta in frequenza che è **1** (nessuna attenuazione) fuori

dall'intervallo di frequenze da rifiutare, e **0** (completa attenuazione) all'interno di tale intervallo.

$$H(u, v) = \begin{cases} 0, & \text{se } D_0 - \frac{W}{2} \leq D \leq D_0 + \frac{W}{2}, \\ 1, & \text{altrimenti.} \end{cases}$$

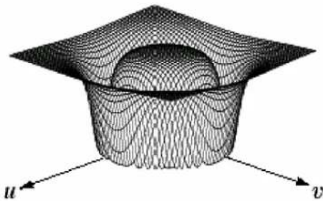
Dove $D(u, v)$ è la distanza dall'origine nello spazio delle frequenze, calcolata come:

$$D(u, v) = \sqrt{\left(u - \frac{P}{2}\right)^2 + \left(v - \frac{Q}{2}\right)^2}$$



Filtri Butterworth band-reject: attenua gradualmente le frequenze indesiderate, evitando discontinuità nette. L'ordine n del filtro controlla la transizione: più alto è l'ordine, più si avvicina a un filtro ideale ed ai suoi difetti di ringing.

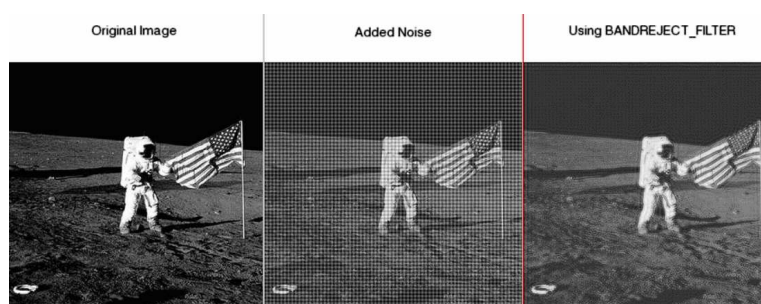
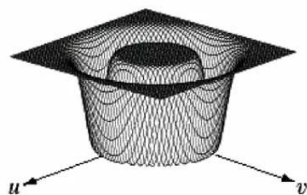
$$H(u, v) = \frac{1}{1 + \left(\frac{D^2 - D_0^2}{D \cdot W}\right)^{2n}}$$



Filtri gaussiani band-reject: attenuano gradualmente le frequenze indesiderate mantenendo una trasformata di Fourier gaussiana, il che li rende facili da implementare. Offrono transizioni morbide simili ai **filtri di Butterworth di piccolo ordine**. Sono ideali per rimuovere rumori senza compromettere la qualità del segnale.

La funzione di trasferimento di tale filtro di frequenza di taglio D_0 è:

$$H(u, v) = 1 - e^{-\left[\frac{D^2(u, v) - D_0^2}{D \cdot W}\right]^2}$$



La compressione

La **compressione dati** è una tecnica che, attraverso specifici algoritmi, riduce la quantità di bit necessari per rappresentare un'informazione in forma digitale. Questo consente di diminuire le dimensioni dei file, ottimizzando lo spazio di archiviazione e migliorando l'efficienza nella trasmissione dei dati, riducendo l'uso della banda.

Diverse quantità di dati possono essere usate per rappresentare la stessa quantità di informazione: le rappresentazioni che presentano informazioni ripetute o irrilevanti contengono *dati ridondanti*.

Un codice è un sistema di simboli (lettere, numeri, bit, ecc.) usati per rappresentare delle informazioni.

Ad ogni pezzo di informazione o evento è assegnata una sequenza di simboli codificati, detta *codeword*, la cui lunghezza è data dal numero di simboli che la compongono.

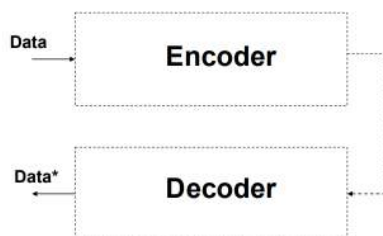
Ridondanza spaziale e temporale

La **ridondanza spaziale** si verifica quando i pixel di un'immagine sono simili ai loro vicini, causando una ripetizione inutile di informazioni.

La **ridondanza temporale** si manifesta nei video, dove i pixel di frame successivi sono spesso correlati, portando a una duplicazione dell'informazione.

Algoritmo di compressione

Un algoritmo di compressione è una tecnica che elimina la ridondanza di informazione dai dati e consente un risparmio di memoria.



Classificazione dei metodi di compressione

I metodi di compressione possono essere classificati in base a due criteri principali:

Basata sul tipo di dati:

Ogni tipo di dato ha algoritmi di compressione specifici:

- Generico (ZIP, RAR, ..)
- Audio (MP3, AAC, ..)

- Immagini (JPEG, PNG, GIF, ..)
- Video (H.264, H.265, ..)

Basata sul tipo di compressione

- **Lossless (Reversibile)**: non comporta perdita di dati. Esempi: ZIP, PNG.
- **Lossy (Irreversibile)**: comporta perdita di dati. Esempi: **JPEG**

Avendo come immagine originale **I1**; un'immagine compressa in formato lossy **I2** e infine una compressa in formato lossless **I3**:

Un formato **lossy** introduce una perdita di informazione durante la compressione, quindi l'immagine decompressa **I2** sarà diversa da **I1**, risultando in un **MSE > 0**.

Un formato **lossless** permette di ricostruire perfettamente l'immagine dopo la decompressione, quindi **I3** sarà identica a **I1**, e l'**MSE** sarà **0**.

Compressione LOSSLESS

Si parla di compressione lossless quando i dati possono essere trasformati in modo da essere memorizzati con risparmio di memoria e successivamente ricostruiti senza perdita di alcun bit di informazione.

Frequenza

La **frequenza** di un carattere a_i in una sequenza S di N caratteri, tratti da un alfabeto di M caratteri $(a_1, \dots, a_M)$, è definita come:

$$f_i = \frac{\# \text{ occorrenze di } a_i}{N}$$

Dove f_i indica la probabilità relativa del carattere a_i nella sequenza S .

Entropia

L'**entropia** **E** di una sequenza di dati S misura la quantità media di informazione per simbolo ed è definita come:

$$E = - \sum_{i \in S} f_i \log_2 f_i$$

Dove f_i è la frequenza del simbolo a_i nella sequenza.

Questa sommatoria ha segno complessivo negativo, quindi viene corretta con il - davanti

- **Massima entropia** → Per **massimizzare l'entropia** di una sequenza, i simboli devono essere **equiprobabili**, cioè devono avere la stessa probabilità di apparire. Questo perché l'entropia è massima quando ogni simbolo contribuisce in modo uguale all'incertezza totale del sistema.

Esempio:

Nel caso in cui ci siano 4 simboli (A, B, C, D), per massimizzare l'entropia, ciascun simbolo deve avere una probabilità di:

$$p_A = p_B = p_C = p_D = \frac{1}{4} = 25\%$$

In poche parole:

Entropia = quanto è disordinata la sequenza

- **Entropia alta** = sequenza **più casuale** e meno prevedibile
- **Entropia bassa** = sequenza **più ordinata** e prevedibile

Criterio per una buona compressione di tipo lossless: Teorema di Shannon

Una compressione senza perdita di dati (**lossless**) è efficace se si avvicina il più possibile al limite teorico stabilito dal **Primo Teorema di Shannon sulla codifica**.

«Per una **sorgente discreta e a memoria zero**, il **bitrate minimo** necessario per rappresentare l'informazione senza perdita è pari all'**entropia della sorgente**.»

Una sorgente è detta **a memoria zero** quando i suoi simboli sono **indipendenti**, cioè la probabilità di un simbolo non dipende da quelli precedenti.

I dati devono essere rappresentati senza perdere informazione (lossless) usando almeno un numero di bit pari a: $N * E$

Dove N è il numero di caratteri mentre E è l'entropia.

(La quantità può essere superiore, ma non può assolutamente essere minore, altrimenti la compressione diventerebbe di tipo lossy!)

Il teorema di Shannon stabilisce la lunghezza minima dei codici, ma non come trovarli. Occorre usare un algoritmo che permetta di codificare i caratteri usando il numero di bit ricavati con il teorema di Shannon.

Compressione lossless: Huffman

L'algoritmo di **Huffman** risolve questo problema, proponendo un algoritmo greedy che permette di ottenere un "dizionario" (cioè una tabella carattere codifica binaria) per una compressione quasi ottimale dei dati (cioè pari al limite di Shannon) con un eccesso di qualche bit.

Si tratta di una codifica a lunghezza variabile che creando un albero *binario* le cui foglie sono i caratteri da codificare, ed associa

- a **simboli meno frequenti i codici più lunghi** e quindi più bit
- a **simboli più frequenti i codici più corti** e quindi meno bit

Inoltre, nessun codice è prefisso di altri codici.

Facciamo un esempio:

Dati: AABABCAACAAADDDD.

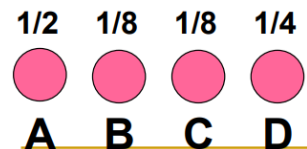
A : frequenza pari a $8/16 = \frac{1}{2}$

B : frequenza pari a $2/16 = \frac{1}{8}$

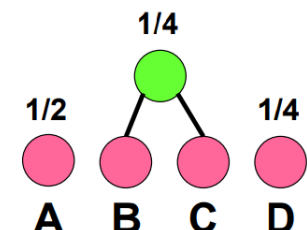
C: frequenza pari a $2/16 = \frac{1}{8}$

D: frequenza pari a $4/16 = \frac{1}{4}$

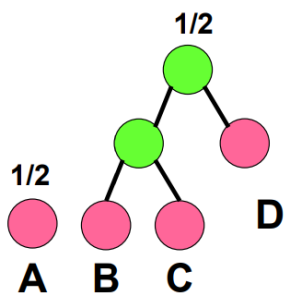
Inizio costruendo una foresta con 4 alberi, ciascuno composto da un solo nodo:



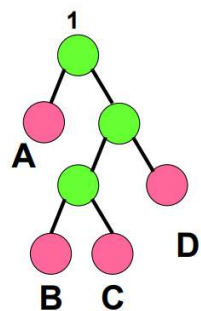
A questo punto, devo aggregare i due alberi della foresta con *minore frequenza* (B e C perchè erano entrambi $\frac{1}{8}$):



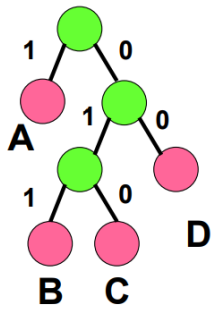
Continuo con questo procedimento fino ad avere un solo albero che aggreghi tutte le foglie. Quindi, i due alberi con minore frequenza ora saranno quello formato da B-C e D, entrambi con frequenza pari a $\frac{1}{4}$, di conseguenza li uniamo:



Ripeto l'operazione con l'albero B-C-D ed A:



Abbiamo unificato l'albero, a questo punto si devono etichettare con "1" tutti i rami sinistri e con "0" i rami destri. Il cammino dalla radice al simbolo fornisce la parola del dizionario che "codifica" in maniera ottimale il simbolo:



A : 1 B : 011 C : 010 D : 00

Codice per la sequenza AABABCAACAAADDDD:

1-1-011-1-011-010-1-1-010-1-1-1-00-00-00-00 → pari a 28 bit (ottimale)

Costo aggiuntivo: nella memoria verranno memorizzati i bit delle lettere e quelli necessari per la memorizzazione della *tabella caratteri-codici*. Se i caratteri sono tanti questo può essere costoso.

A : 1
B : 011
C : 010
D : 00

Se scambiassimo le cifre 0 ed 1 nei rami non cambierebbe nulla! Dovrei semplicemente etichettare in modo diverso i rami e ci sarebbe una codifica diversa; l'importante è non cambiare casualmente i numeri.

La codifica di Huffman distingue tra maiuscole e minuscole (es. "A" ≠ "a"). Maiuscole, spazi e simboli speciali vengono codificati come qualsiasi altro carattere.

Huffman viene usato per comprimere alcune informazioni nella fase finale della codifica **JPEG**.

- Esercizio: Data la stringa AAAAABBC, calcolare le tabelle di codifica tramite l'algoritmo di Huffman.

1. Calcolare le frequenze dei simboli:

- A: 5
- B: 2
- C: 1

2. Costruire l'albero:

Combiniamo i simboli meno frequenti, ovvero B-C, creando un albero di frequenza 3. Uniamo l'albero risultante(3) con A(5), ottenendo l'albero finale di frequenza 8

3. Assegnare i codici:

Partendo dalla radice, si assegna 0 ai rami sinistri e 1 ai rami destri.

Radice (8)

/

0/\1
A(5) Nodo BC(3)
/
0/\1
B(2) C(1)

La tabella corretta è **A:0 B:10 C:01**

Altri algoritmi di compressione LOSSLESS

Codifica Run-length

Il **metodo di compressione Run-Length (RLE)** è particolarmente efficace in presenza di sequenze ripetute di valori. Esso riduce la dimensione dei dati rappresentando queste sequenze, chiamate *run*, in modo più compatto.

Ogni *run* viene descritto da una coppia di valori:

- **Il valore** (l'intensità del pixel).
- **La lunghezza** della sequenza** (il numero di volte consecutive in cui compare quel valore).

La RLE è utilizzata nella compressione di immagini, dove i pixel spesso presentano intensità ripetute lungo le righe o le colonne. È impiegata in formati come **JPEG, BMP, M-JPEG**, ecc.

Facciamo un esempio di RLE avendo la sequenza:

00000111001011101110101111111

1. **Identificare le sequenze di simboli uguali (runs):** 5 volte 0; 3 volte 1; 2 volte 0; ecc..
2. **Memorizzare solo le lunghezze delle sequenze:** 5, 3, 2, 1, 1, 3, 1, 3, 1, 1, 1, 7
3. **Rappresentare le lunghezze in binario:** Ogni numero che rappresenta la lunghezza della run viene scritto in formato binario: - 5 → 101 ; 3 → 11 ; 2 → 10 ; ecc..
4. Risultato finale: 101, 11, 10, 1, 1, 11, 1, 11, 1, 1, 1, 111

La codifica RLE è vantaggiosa quando ci sono molte run lunghe, poiché permette di risparmiare spazio. Se le run sono brevi o la lunghezza delle run è più piccola del numero di bit necessari per rappresentarle, la compressione **non è vantaggiosa**.

- **Esercizio RLE:**
Sia **3, 5, 1, 3** la sequenza ottenuta codificando una codeword binaria tramite una codifica run-length. Quale, tra le seguenti è la codeword di partenza?
La sequenza run-length (3, 5, 1, 3) rappresenta una codifica in cui i numeri alternano tra la lunghezza delle sequenze di 0 e 1. Assumiamo che la codeword inizi con 0.
- **3:** Tre 0 → 0 0 0
- **5:** Cinque 1 → 1 1 1 1 1

- 1: Un 0 → 0
- 3: Tre 1 → 1 1 1

Quindi, la codeword di partenza è:

0 0 0 1 1 1 1 1 0 1 1 1

Run Length + bit planes

Facciamo adesso un esempio di questa codifica applicata sui bit planes.

Un'immagine in scala di grigi a 8 toni (3 bit di profondità) è rappresentata con valori da 0 a 7.

2	5	7	0
5	5	5	5
4	4	3	4
5	4	2	0

Ogni valore viene convertito in binario a 3 bit.

In binario

010	101	111	000
101	101	101	101
100	100	011	100
101	100	010	000

La matrice binaria risultante è suddivisa in tre **piani di bit**:

- **Primo piano di bit (MSB):** Contiene il primo bit (più significativo) di ogni valore.
- **Secondo piano di bit:** Contiene il secondo bit.
- **Terzo piano di bit (LSB):** Contiene il terzo bit (meno significativo).

Bit planes		
0 1 1 0	1 0 1 0	0 1 1 0
1 1 1 1	0 0 0 0	1 1 1 1
1 1 0 1	0 0 1 0	0 0 1 0
1 1 0 0	0 0 1 0	1 0 0 0
Primo bit	Secondo bit	Terzo bit

Ogni piano di bit viene letto riga per riga e trasformato in una sequenza lineare. Quindi il primo piano diventa 0110111111011100, il secondo 1010000000100010 ed il terzo 0110111100101000.

La sequenza viene compressa usando la tecnica del **Run-Length Encoding**, conta il numero di zeri e di uno consecutivi e li rappresenta in modo compatto.

Codifica differenziale

La **codifica differenziale** è una tecnica di compressione che sfrutta la ridondanza nei dati per ridurre il numero di bit necessari alla memorizzazione. Il principio alla base è che, in molti casi, i valori successivi in una sequenza variano di poco rispetto al precedente, quindi è più efficiente memorizzare le differenze.

Come funziona?

1. Si registra il **valore iniziale**.
2. Si memorizzano le **differenze successive** tra ogni valore e il precedente.

Dati i valori:

134, 137, 135, 128, 130, 134, 112, ...

Si salva il primo valore (**134**) e poi la sequenza delle differenze:

+3, -2, -7, +2, +4, -22, ...

La sequenza delle differenze tra pixel successivi in un'immagine contiene meno **informazione casuale** (cioè ha una **minore entropia**) rispetto ai valori assoluti dei pixel. Questo rende la sequenza più compressibile, cioè richiede meno bit per essere rappresentata.

- Esercizio:
Sia -2, 5, -2 la codifica differenziale di una quaterna. Sapendo che il primo valore non codificato è 100 quanto vale il quarto valore?

1. 100
2. $100 - 2 = 98$
3. $98 + 5 = 103$
4. $103 - 2 = 101$

Compressione Lossy

La **compressione lossy** è una tecnica di riduzione dei dati che permette un notevole risparmio di memoria a costo di una perdita di informazione. Questo approccio è particolarmente adatto a contenuti multimediali in cui una leggera perdita di qualità è accettabile e spesso impercettibile. Trattandosi di una compressione **irreversibile**, i dati eliminati non possono essere recuperati.

Ambiti di Applicazione della Compressione Lossy

1. **Immagini**: le immagini possono essere compresse con tecniche **lossy** (ad esempio, *JPEG*) poiché la perdita di qualità non è sempre visibile a occhio nudo.
2. **Audio**: formati come *MP3* e *AAC* eliminano suoni mascherati da altri più forti o fuori dalla gamma di percezione umana. La perdita di dettagli è generalmente poco evidente per l'ascoltatore medio.
3. **Video**: la compressione lossy per i video è comunemente accettata perché, come nel caso delle immagini, la perdita di qualità non è sempre percepibile.

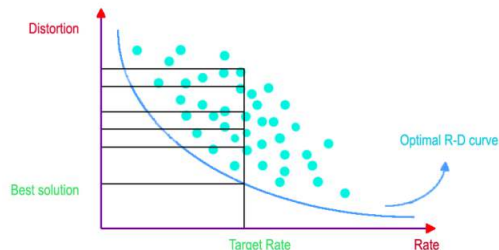
Testo: Il testo non dovrebbe mai essere compresso con una compressione lossy, perché ogni carattere è importante per l'interpretazione del contenuto.

- La **compressione lossless** (senza perdita di dati) è essenziale per il testo, in modo che non vengano persi dati cruciali.

Criterio per una buona compressione di tipo lossy

Un buon algoritmo lossy deve bilanciare **compressione e qualità**, seguendo due criteri fondamentali:

- Fissata la massima distorsione accettabile, l'algoritmo di compressione deve trovare la rappresentazione con il più basso numero di bit;
- Fissato il massimo numero di bit accettabile, bisogna trovare il miglior algoritmo di compressione che a parità di numero di bit fornisce la minima distorsione.



Algoritmo lossy: Requantization

La **requantization** è una tecnica di compressione lossy che riduce la quantità di informazioni codificate abbassando la precisione dei valori dei pixel. In pratica, si eliminano alcuni bit meno significativi, riducendo così il numero di livelli disponibili per ciascun colore.

Se ogni canale colore (RGB) è originariamente rappresentato con 8 bit, abbiamo:

- **Rosso (RED):** $2^8 = 256$ livelli $\rightarrow$ ridotti a 4 bit $\rightarrow 2^4 = 16$ livelli
- **Verde (GREEN):** $2^8 = 256$ livelli $\rightarrow$ ridotti a 6 bit $\rightarrow 2^6 = 64$ livelli
- **Blu (BLUE):** $2^8 = 256$ livelli $\rightarrow$ ridotti a 2 bit $\rightarrow 2^2 = 4$ livelli

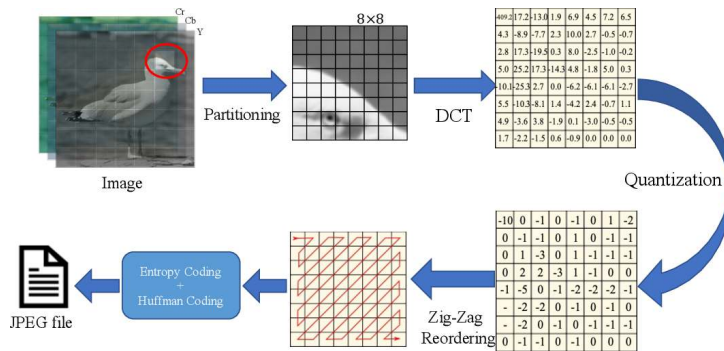
Effetti

- **Riduzione del numero totale di colori** disponibili nell'immagine, da 16.7 milioni a 4,096 colori.
- Risparmio del 50% della memoria.
- **Si riduce l'entropia del segnale** (meno simboli diversi da codificare).
- **Perdita visibile della qualità**, specialmente nelle sfumature e nei dettagli fini.

Potrebbe capitare all'esame un esercizio di requantizzazione!)



Lo standard JPEG



Storia:

JPEG è uno standard di compressione immagini sviluppato dal Joint Photographic Experts Group. Utilizza una tecnica di compressione "lossy" basata sulla DCT (Discrete Cosine Transform - simile alla Trasformata Discreta di Fourier (DFT), ma utilizza solo numeri reali). Proposto ufficialmente nel 1991, è diventato uno standard nel 1992.

I passaggi della compressione

La codifica JPEG è abbastanza complessa poiché viene eseguita in 3 passi, a loro volta costituiti da sotto-passi.

1. Il pre-processing

a) Trasformazione dei colori: RGB $\rightarrow$ YCbCr

L'immagine originale è in **RGB** (rosso, verde, blu).

Viene convertita in **YCbCr** (luminanza + due componenti di cromaticità):

b) Sotto-campionamento della cromaticità:

Nel formato JPEG, si preferisce lavorare su uno spazio di colore luminanza-cromaticità (YCbCr) perché l'occhio umano è più sensibile ai dettagli di luminanza rispetto ai dettagli di cromaticità. Di conseguenza, si mantengono tutti i valori della luminanza e si prende, invece, 1 valore ogni 4 per i canali C_b e C_r . **Questo passaggio è LOSSY ma comporta un gran risparmio in memoria.**

c) Partizione in blocchi 8×8 :

L'immagine viene scomposta in blocchi di dimensione 8 pixel x 8 pixel. Se l'immagine non può essere scomposta perfettamente in blocchi di dimensione 8×8 , vengono aggiunti ulteriori pixel (padding). Dalla partizione nasce la problematica della quadrettatura, artefatto

tipico delle immagini compresse in JPEG, specialmente se la compressione è elevata.



2. Trasformazione

Prima della DCT

Processo di shift dei livelli di grigio: prima della applicazione della DCT ai 64 pixel di ciascun blocco, viene sottratta una quantità pari a 2^{n-1} , dove 2^n rappresenta il numero massimo di livelli di grigio dell'immagine. Ai valori di pixel viene sottratto **128** (se l'immagine è a 8 bit). Questo porta i valori da **[0, 255]** a **[-128, 127]**.

52	55	61	66	70	61	64	73	-76	-73	-67	-62	-58	-67	-64	-55
63	59	66	90	109	85	69	72	-65	-69	-62	-38	-19	-43	-59	-56
62	59	68	113	144	104	66	73	-66	-69	-60	-15	16	-24	-62	-55
63	58	71	122	154	106	70	69	-65	-70	-57	-6	26	-22	-58	-59
67	61	68	104	126	88	68	70	-61	-67	-60	-24	-2	-40	-60	-58
79	65	60	70	77	68	58	75	-49	-63	-68	-58	-51	-60	-70	-53
85	71	64	59	55	61	65	83	-43	-57	-64	-69	-73	-67	-63	-45
87	79	69	68	65	76	78	94	-41	-49	-59	-60	-63	-52	-50	-34

a) La Trasformata Discreta del Coseno (Discrete Cosine Transform):

La compressione JPEG utilizza la **Trasformata Discreta del Coseno (DCT)**, che appartiene alla famiglia delle **trasformate di Fourier**, trasforma l'immagine da una rappresentazione spaziale a una rappresentazione nelle "basi" della DCT, che sono funzioni coseno. In altre parole, la DCT trasforma i dati dell'immagine dal dominio dello spazio a quello delle frequenze.

Che differenza c'è?

- Il dominio dello *spazio* mostra la variazione di intensità del colore nello spazio
- Il dominio delle *frequenze* mostra quanto rapidamente l'intensità del colore varia spostandosi da un pixel a quello adiacente

Ogni blocco di **8x8 pixel** può essere visto come un vettore in uno spazio a **64 dimensioni**, in altre parole, ogni immagine a 8x8 pixel può essere rappresentata come un vettore di **64 valori**. Questa immagine è la **somma pesata di 64 immagini impulsive**, che insieme formano una **base impulsiva**.

(Un'immagine impulsiva è una matrice di 64 pixel, con tutti i pixel uguali a **0**, tranne uno che ha valore **1**)

I **pesi** di questa combinazione sono i valori dei pixel originali, che moltiplicano le immagini impulsive per ricostruire il blocco. I pesi rappresentano l'effettivo livello di luminosità di ogni

pixel.

La trasformata del coseno esprime l'immagine in un'altra base, vediamo come.

La formula della DCT per un blocco di dimensioni NxN

DCT (Trasformata Discreta del Coseno):

Moltiplichiamo i pixel originali $f(x,y)$ per le basi della DCT e sommiamo i risultati per ottenere i coefficienti $F(u,v)$.

$$F(u, v) = \frac{2}{N} \left[\sum_{x=0}^{N-1} \sum_{y=0}^{N-1} C(u)C(v)f(x, y) \cos\left(\frac{(2x+1)u\pi}{2N}\right) \cos\left(\frac{(2y+1)v\pi}{2N}\right) \right]$$

IDCT (Trasformata Coseno Discreta Inversa):

Parte dai coefficienti $F(u,v)$ e ricostruisce l'immagine sommando i contributi delle basi coseno moltiplicate per i coefficienti DCT.

$$f(x, y) = \frac{2}{N} \left[\sum_{u=0}^{N-1} \sum_{v=0}^{N-1} C(u)C(v)F(u, v) \cos\left(\frac{(2x+1)u\pi}{2N}\right) \cos\left(\frac{(2y+1)v\pi}{2N}\right) \right]$$

$$C(h) = \begin{cases} \frac{1}{\sqrt{2}} & \text{se } h = 0 \\ 1 & \text{altrimenti} \end{cases}$$

Queste formule richiedono un tempo computazionale di $O(N^2)$, tuttavia esistono algoritmi più veloci, con prestazioni $O(N \log N)$, derivati dalla Fast Fourier Transform.

Adesso, facciamo un esempio su una matrice per vedere come cambiano i coefficienti dopo aver applicato la formula:

-76	-73	-67	-62	-58	-67	-64	-55	-415	-29	-62	25	55	-20	-1	3
-65	-69	-62	-38	-19	-43	-59	-56	7	-21	-62	9	11	-7	-6	6
-66	-69	-60	-15	16	-24	-62	-55	-46	8	77	-25	-30	10	7	-5
-65	-70	-57	-6	26	-22	-58	-59	-50	13	35	-15	-9	6	0	3
-61	-67	-60	-24	-2	-40	-60	-58	11	-8	-13	-2	-1	1	-4	1
-49	-63	-68	-58	-51	-60	-70	-53	-10	1	3	-3	-1	0	2	-1
-43	-57	-64	-69	-73	-67	-63	-45	-4	-1	2	-1	2	-3	1	-2
-41	-49	-59	-60	-63	-52	-50	-34	-1	-1	-1	-2	-1	-1	0	-1

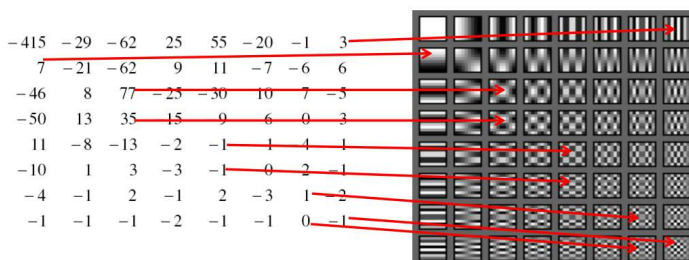


Per la prima riga si ottiene:

(Ognuna di queste basi è grande 8x8 pixel)

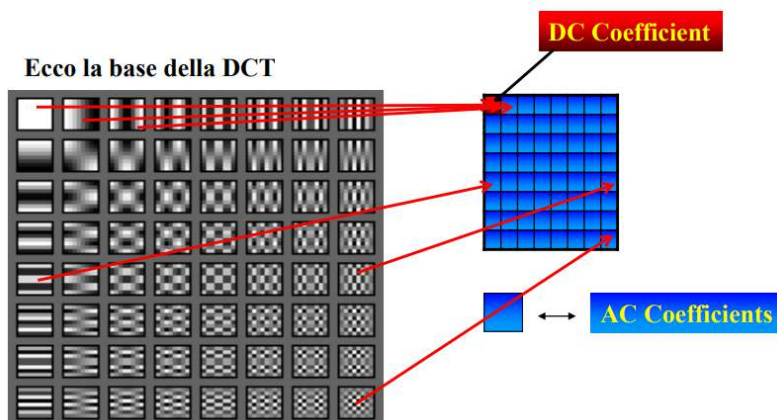
$$\begin{aligned} &-415 * \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} &-29 * \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} &-62 * \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} &+25 * \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \\ &+55 * \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} &-20 * \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} &-1 * \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} &+3 * \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} &+ \dots \end{aligned}$$

Lo stesso calcolo va ripetuto per tutti gli elementi della matrice:



Nella **Trasformata Discreta del Coseno (DCT)**, i coefficienti **DC** e **AC** rappresentano i componenti dell'immagine trasformata:

- **Coefficiente DC:**
 - È il valore **in alto a sinistra** nella matrice trasformata.
 - Rappresenta la **media dei valori di luminosità** dell'intero blocco **8×8**.
 - Ha tipicamente il valore più alto e varia più lentamente nell'immagine.
- **Coefficienti AC:**
 - Sono gli altri **63 valori** nella matrice della DCT.
 - Rappresentano le **variazioni di luminosità** all'interno del blocco.
 - Più un coefficiente è vicino allo zero, meno contribuisce all'immagine finale.



b) Quantizzazione

La quantizzazione è il processo di riduzione della precisione dei coefficienti AC, per ottenere una rappresentazione più compatta. (quindi ridurre il numero di bit necessari a rappresentare l'immagine compressa)

Tale operazione permette di rappresentare i diversi coefficienti incrementando il fattore di compressione, più precisamente avviene un processo di riduzione del numero di bit necessari per memorizzare un valore intero, riducendone la precisione. riducendo anche il numero di "livelli" su cui i coefficienti della DCT possono variare

- **I coefficienti a bassa frequenza** (vicini a $F(0,0)$) contengono dettagli importanti e devono essere meno quantizzati.
- **I coefficienti ad alta frequenza** (lontani da $F(0,0)$) contengono dettagli più sottili e possono essere quantizzati di più senza perdere qualità visibile.

Dato un fattore di quantizzazione Q e un numero F il valore $F_{quantizzato}$ si ottiene come:

$$F_{quantizzato} = \text{round}\left(\frac{F}{Q}\right)$$

Il valore ricostruito si ottiene moltiplicando $F_{quantizzato}$ per Q

Ovviamente la quantizzazione è un processo irreversibile (comporta perdita di informazione).

Usare un unico fattore di quantizzazione per tutti i coefficienti peggiora la qualità dell'immagine, perchè come abbiamo descritto prima, i diversi coefficienti rappresentano informazioni diverse!

Per questo motivo, si adotta una **tabella di quantizzazione** $Q(i,j)$, che assegna un valore di quantizzazione differente a ogni coefficiente $F(i,j)$. Questi valori possono essere:

1. **Predefiniti dallo standard:** JPEG definisce due tabelle di quantizzazione standard, una per la luminanza (Y) e una per le crominanze (Cb e Cr).
2. **Personalizzati dall'utente:** si può fornire una tabella personalizzata, che dovrà essere trasmessa insieme ai dati compressi per garantire la corretta decompressione.

Tabella di quantizzazione luminanza

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

Tabella di quantizzazione crominanza

17	18	24	47	99	99	99	99
18	21	26	66	99	99	99	99
24	26	56	99	99	99	99	99
47	66	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99

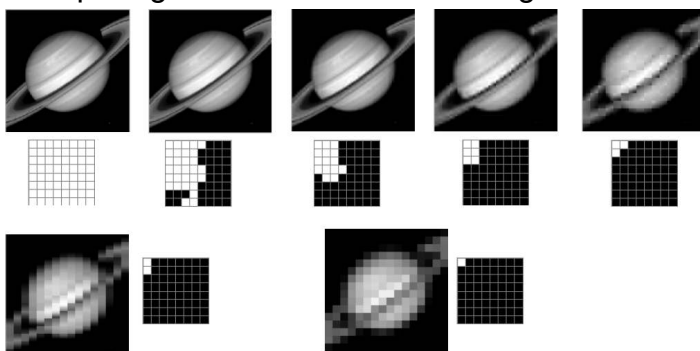
Un fattore di compressione maggiore comporta ovviamente una maggiore perdita di informazione. L'utente del JPEG può scegliere il "grado" di quantizzazione da adottare fornendo un "*quality factor*" **QF** che va da 1 a 100.

La tabella di quantizzazione adottata sarà una copia delle tabelle sopra, i cui elementi sono divisi per QF.

Maggiore il sarà QF, minore saranno i fattori di quantizzazione e la perdita di informazioni.

Effetto della quantizzazione:

I quadretti neri rappresentano i coefficienti della DCT che la quantizzazione ha portato a zero per ogni blocco 8x8 della immagine di Saturno.



Dopo la **DCT** e la **quantizzazione**, la maggior parte delle informazioni significative di un blocco **8x8** si trova nei coefficienti **a bassa frequenza** (in alto a sinistra della matrice). I coefficienti **ad alta frequenza** (in basso a destra) tendono a essere piccoli o nulli a causa della quantizzazione.

L'ordinamento **a serpentina** permette di **raggruppare i coefficienti significativi all'inizio** della sequenza, raggruppando gli zeri alla fine.

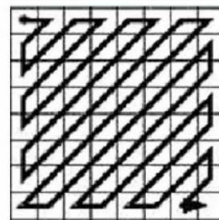
Matrice dei coefficienti DCT, dopo la quantizzazione:

-26	-3	-6	2	2	0	0	0
1	-2	-4	0	0	0	0	0
-3	1	5	-1	-1	0	0	0
-4	1	2	-1	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

(Si nota un triangularizzazione dei valori, cioè tutti gli 0 sono nel triangolo in basso a destra della matrice)

A questo punto, tutti i coefficienti vengono riordinati in un vettore 64 x 1 seguendo l'ordinamento "a serpentina" (per creare lunghe run di zeri) e codificati in un altro stream.

-26	-3	-6	2	2	0	0	0
1	-2	-4	0	0	0	0	0
-3	1	5	-1	-1	0	0	0
-4	1	2	-1	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0



ordinamento a zigzag

-26 -3 1 -3 -2 -6 2 -4 1 -4 1 1 5 0 2 0 0 -1 2 0 0 0 0 0 -1 -1 EOB

Utilizziamo un valore chiamato EOB per indicare che tutto il resto è zero.

Differenti codifiche

Dopo aver ordinato i coefficienti, si applicano due diverse codifiche:

- I **coefficienti DC**, nella posizione (1,1) del blocco 8x8 sono codificati con la *codifica differenziale*
- I **coefficienti AC**, vengono codificati usando la *codifica Run-length*

Codifica coefficienti DC

Il coefficiente DC è la differenza tra il coefficiente DC del blocco corrente e quello del blocco precedente (Δ).

Ex. 12,13,11,11,10,...  12,1,-2,0,-1,...

La codifica di questo coefficiente segue un certo schema:

- **Se $\Delta > 0$** : Si prendono i bit meno significativi di Δ in binario. Il numero di bit (n) dipende dalla rappresentazione scelta (es. 8 bit, 16 bit).
- **Se $\Delta < 0$** : Si prende il valore di Δ in **complemento a due** (per rappresentare i numeri negativi in binario) e poi si sottrae 1. La codifica dei bit risultanti è quella che viene

aggiunta alla sequenza.

- **Se $\Delta = 0$:** Non si aggiungono bit, poiché la differenza è zero e il valore è già rappresentato come "nessuna differenza".

Codifica del DC coeff. $\begin{cases} \rightarrow \text{Binary if positive} \\ \rightarrow \text{Complement if negative} \end{cases}$

Le tabelle mostrate di seguito forniscono codici di Huffman, ottenuti grazie a calcolatori statistici, fornite da uno standard.

TABLE 8.17
JPEG coefficient
coding categories.

Range	DC Difference Category	AC Category
0	0	N/A
-1, 1	1	1
-3, -2, 2, 3	2	2
-7, ..., -4, 4, ..., 7	3	3
-15, ..., -8, 8, ..., 15	4	4
-31, ..., -16, 16, ..., 31	5	5
-63, ..., -32, 32, ..., 63	6	6
-127, ..., -64, 64, ..., 127	7	7
-255, ..., -128, 128, ..., 255	8	8
-511, ..., -256, 256, ..., 511	9	9
-1023, ..., -512, 512, ..., 1023	A	A
-2047, ..., -1024, 1024, ..., 2047	B	B
-4095, ..., -2048, 2048, ..., 4095	C	C
-8191, ..., -4096, 4096, ..., 8191	D	D
-16383, ..., -8192, 8192, ..., 16383	E	E
-32767, ..., -16384, 16384, ..., 32767	F	N/A

TABLE 8.18
JPEG default DC
code (luminance).

Category	Base Code	Length	Category	Base Code	Length
0	010	3	6	1110	10
1	011	4	7	11110	12
2	100	5	8	111110	14
3	00	5	9	1111110	16
4	101	7	A	11111110	18
5	110	8	B	111111110	20

SSSS = categoria

Δ = differenza tra due
coefficienti DC
(evento)

Codifica coefficienti AC

Poiché i coefficienti quantizzati AC sono spessissimo nulli, si usa una trasformazione in skip-size. Cioè, data una sequenza di valori, si memorizza il numero degli zeri, seguito dal primo valore non zero che si incontra.

- Codificati come coppie (skip,value)
 - SKIP: quantità di 0 nella run
 - SIZE: Indica quanti bit servono per rappresentare il valore del coefficiente

Example:

$\xleftarrow{\text{Zig zag ordered}}$ 0.....0 0 0 2 2 2 2 3 3 3 7 6 12 $\xleftarrow{\text{DC}}$

(0,6) (0,7) (0,3) (0,3) (0,3) (0,2) (0,2)(0,2) (0,2) (0,0) $\nwarrow$

- block end
- Remaining coeff are null

Procedimento:

Si verifica a quale SSSS corrisponde il SIZE (ad esempio per una coppia (0,-3) il coefficiente AC il valore SSSS è 2)

SSSS	Coefficiente AC
1	-1, 1
2	-3 -2, 2 3
3	-7 ... -4, 4 ... 7
4	-15 ... -8, 8 ... 15
5	-31 ... -16, 16 ... 31
6	-63 ... -32, 32 ... 63
7	-127 ... -64, 64 ... 127
8	-255 ... -128, 128 ... 255
9	-511 ... -256, 256 ... 511
A	-1023 ... -512, 512 ... 1023

- Dalla tabella di Huffman associata ai coefficienti AC, si prende il codice corrispondente alla coppia (RUN, SSSS).

La classe dell'evento è espressa mediante una coppia del tipo (run, categoria).

Nel nostro caso quindi abbiamo (0,2)

(run, category)	Codice base	Lunghezza codice completo
(0, 0)	1010 (= EOB)	4
(0, 1)	00	3
(0, 2)	01	4
(0, 3)	100	6
....		
(F, 0)	111111110111	12
....		
(F, A)	1111111111111110	26

Alla coppia (0, 2) corrisponde il codice base 01, tale codice sarà completato dall'aggiunta di un numero di bit fino a raggiungere il numero totale di bit che è riportato nell'ultima colonna della tabella.

Aggiungere i bit supplementari (se necessari)

- Se il valore v del coefficiente AC è positivo, si aggiungono i bit meno significativi della rappresentazione binaria di v .
- Se v è negativo, si prende il complemento a due di $|v|$, e si sottrae 1 dal valore binario ottenuto.
- Se $v = 0$, nessun bit aggiuntivo viene inserito (SSSS = 0).

Nella ricostruzione

Si deve tornare indietro “ricostruendo” i dati originali (o le loro approssimazioni per i passi irreversibili). Esistono diverse “strategie” per la ricostruzione in modo da abilitare la ricostruzione progressiva, gerarchica o lossless. Tuttavia, noi non le tratteremo.

Confronto tra blocco originale e blocco ricostruito

Attenzione, il Jpeg è lossy! Quindi il **blocco ricostruito è diverso da quello in input**.

52	55	61	66	70	61	64	73	58	64	67	64	59	62	70	78
63	59	66	90	109	85	69	72	56	55	67	89	98	88	74	69
62	59	68	113	144	104	66	73	60	50	70	119	141	116	80	64
63	58	71	122	154	106	70	69	69	51	71	128	149	115	77	68
67	61	68	104	126	88	68	70	74	53	64	105	115	84	65	72
79	65	60	70	77	68	58	75	76	57	56	74	75	57	57	74
85	71	64	59	55	61	65	83	83	69	59	60	61	61	67	78
87	79	69	68	65	76	78	94	93	81	67	62	69	80	84	84

Compressione e immagini

Immagini “grafiche” con pochi colori e con testi non sono compresse con buona qualità dal JPEG! Inoltre, per il WEB, JPEG non gestisce la trasparenza (GIF lo fa).



JPEG2000

JPEG2000 è un formato di compressione che ha introdotto miglioramenti significativi rispetto al JPEG, ma non ha avuto successo commerciale a causa dell'**inerzia tecnologica** e della **compatibilità con i sistemi esistenti**.

- Usa la **Trasformata Wavelet Discreta (DWT)**.
- Alloca più bit nelle zone con più informazione e permette il controllo esplicito di tale allocazione.
- Raggiunge rapporti di compressione più elevati

La **decomposizione wavelet 2D** scompone un'immagine in più livelli di dettaglio utilizzando **filtri passa-basso e passa-alto** nelle direzioni orizzontale e verticale. L'immagine viene suddivisa ricorsivamente in **quattro sottobande**: **LL** (bassa frequenza, dettagli principali); **LH** (dettagli verticali); **HL** (dettagli orizzontali); **HH** (dettagli diagonali). La sottobanda **LL** viene ulteriormente suddivisa.

